RETRO DEV KIT APPLE 1 — THE ORIGINAL SIN

# Complete Manual

*The emulator, the ACI cassette interface, the GEN2 color card, the debugger — with complete guides to Integer BASIC and the 6502 assembler*




VERSION 1.0 NOVEMBER 2026 ENGLISH EDITION

# Contents

## THE EMULATOR


**1 About this manual 8**

1.1 How this manual is organized ..... 8
1.2 Conventions ..... 8
1.3 What you need ..... 9
1.4 The main paths ..... 10

**2 The window and the toolbar 11**

2.1 Overview ..... 11
2.2 The toolbar, button by button ..... 12
2.3 The status bar ..... 15
2.4 The Apple-1 keyboard ..... 15
2.5 Machine state ..... 16
2.6 The EPROM programmer ..... 17

**3 The screen: zoom, phosphor and vintage effect 19**

3.1 The original screen ..... 19
3.2 The display row ..... 19
3.3 The vintage screen ..... 20

**4 The library 23**

4.1 Folders ..... 23
4.2 Running a program ..... 24
4.3 A program's settings ..... 25
4.4 Recognized tapes ..... 26
4.5 GitHub projects ..... 27
4.6 The "Art ASCII" folder ..... 28

**5 The tape and the ACI card 29**

5.1 File formats ..... 29
5.2 The virtual deck ..... 29
5.3 Sound and oscilloscope ..... 30
5.4 Recording from the emulated Apple-1 ..... 30
5.5 How the ACI works ..... 31
5.6 ACI commands ..... 31
5.7 To a real Apple-1 ..... 32
5.8 What level is needed? The input models ..... 33
5.9 Digitizing a real tape or an Apple-1 recording ..... 33

**6 The graphics cards: GEN2 and P-LAB 34**

6.1 Plugging in the card ..... 34
6.2 The soft switches ..... 35
6.3 Modes and memory ..... 35
6.4 The test programs ..... 36
6.5 What is approximated ..... 37
6.6 The P-LAB graphics card ..... 37

**7 The editor 40**

7.1 Tabs and files 40
7.2 Find, replace and the right-click menu 41
7.3 A BASIC program 42
7.4 A 6502 source 43

**8 The debugger, the disassembly and the dump 45**

8.1 The debugger panel 45
8.2 Breakpoints 45
8.3 Debugging a BASIC program 46
8.4 Debugging a 6502 program 47
8.5 The dump 47
8.6 On the command line 48

**9 Photo → ASCII 49**

9.1 Opening a picture 49
9.2 Settings 50
9.3 The program produced 50
9.4 On the command line 51

**10 Panel layout 52**

10.1 Moving a panel 52
10.2 The initial layout 52
10.3 The start-up guide 52
10.4 Text size of each window 53
10.5 What is remembered between two launches 53
10.6 When the screen is bigger than its pane 53
10.7 Detaching a panel 53

**11 Preferences 55**

**12 The command line 58**

12.1 The terminal emulator 58
12.2 Conversions 60
12.3 BASIC and assembler 60
12.4 ASCII art 61
12.5 PROMs 61
12.6 The graphical interface 61

**13 Troubleshooting and FAQ 63**

13.1 The window does not open 63
13.2 Keys do not reach the Apple-1 63
13.3 “Integer BASIC not found” 63
13.4 A BASIC program stops with STOPPED AT 63
13.5 The program does not start after loading 63
13.6 The tape does not load on the real Apple-1 63
13.7 The screen is full of inverse @ 64
13.8 Characters come out slowly 64
13.9 The tape sound is muted 64
13.10 The GitHub project does not build 64
13.11 What the emulator does not do 64

**14** Installing a ROM or the original Integer BASIC **65**
14.1 What is already built in ... 65
14.2 Original Integer BASIC ... 65

## INTEGER BASIC

**15** First steps in BASIC **67**
15.1 Two BASICs: RDK G2 Basic and the original ... 67
15.2 Starting BASIC ... 68
15.3 Direct mode and program mode ... 68
15.4 Correcting a line ... 69
15.5 A first program ... 69
15.6 Commands ... 69
15.7 Three programs to type in ... 70

**16** Statement and function reference **72**
16.1 Statements ... 72
16.2 Functions ... 73
16.3 Operators ... 73
16.4 Extensions of RDK G2 Basic ... 74
16.5 Keywords that do nothing ... 74

**17** Variables, strings, arrays and memory **75**
17.1 Numbers ... 75
17.2 Variable names ... 75
17.3 Strings ... 75
17.4 Arrays ... 76
17.5 BASIC's memory ... 76
17.6 Mixing BASIC and machine language ... 78

**18** Error messages and limits **79**
18.1 Messages ... 79
18.2 Limits ... 80
18.3 Saving memory ... 80

**19** Known bugs and their workarounds **81**
19.1 Leaving a FOR loop with GOTO ... 81
19.2 The ^ operator hangs the machine ... 81
19.3 The "PPED AT" message ... 82
19.4 IF ... THEN governs only the first statement ... 82
19.5 −32768 ... 82
19.6 ASC( does not work ... 83
19.7 Graphics keywords that do nothing ... 83
19.8 The Apple II 251-character bug ... 83
19.9 STOPPED AT 65535 ... 83
19.10 USR crashes the machine ... 84
19.11 HIMEM= cannot go down ... 84
19.12 The colon refused in direct mode ... 84
19.13 INPUT always adds "?" ... 84
19.14 The FOR loop runs at least once ... 84
19.15 A key press stops the program ... 84
19.16 Summary ... 85

**20** Porting an Applesoft or Microsoft BASIC program **86**
20.1 Equivalents ... 87
20.2 The method ... 87
20.3 A porting example ... 88
20.4 Lessons from the ports ... 89

## 6502 ASSEMBLY

**21** The 6502 in brief **91**
21.1 Registers ... 91
21.2 Addressing modes ... 92
21.3 Instructions by family ... 92
21.4 An NMOS 6502, not a 65C02 ... 93
21.5 The memory map at a glance ... 93

**22** The devkit assembler **94**
22.1 The work cycle ... 94
22.2 The listing and the symbols ... 96
22.3 The command line ... 96
22.4 Syntax ... 97
22.5 Directives ... 98
22.6 Macros, .repeat and conditional assembly ... 99
22.7 Error messages ... 99
22.8 Compatibility with ca65 and xa ... 100

**23** The keyboard, the display and the Woz Monitor **101**
23.1 The four PIA registers ... 101
23.2 Reading a key ... 102
23.3 Writing a character ... 102
23.4 The Woz Monitor routines ... 103
23.5 Starting and ending a program ... 103
23.6 The PIA, the tape and the color card ... 104

**24** Annotated examples **105**
24.1 `hello.s`: printing a message ... 106
24.2 `clavier.s`: echoing keys ... 107
24.3 `hexa.s`: a monitor-style dump ... 107
24.4 `decimal.s`: printing a number in decimal ... 108
24.5 `jeu.s`: guess the digit ... 110
24.6 `macros.s`: letting the assembler do the work ... 112
24.7 `gen2-lores.s`: low-resolution color ... 113
24.8 `gen2-hires.s`: high resolution ... 115

**25** Tips, traps and limits **118**
25.1 Where to put a program ... 118
25.2 Living within the branch range ... 118
25.3 Strings and bit 7 ... 118
25.4 Waiting a little ... 119
25.5 Debugging efficiently ... 119
25.6 On to the real Apple-1 ... 120
25.7 Limits of the built-in assembler ... 120

## APPENDICES


| | | |
|---|---|---|
| A | Memory map | 122 |
| B | Woz Monitor commands | 124 |
| C | ACI timings | 125 |
| D | The 2513 character set | 126 |
| E | Keyboard shortcuts | 127 |
| F | Pitfalls and behaviours to know | 128 |
| G | Licence and credits | 130 |

PART I

# The emulator

*The window, the screen, the library, the ACI cassette interface, the GEN2 color card, the debugger and the command line*

CHAPTER 1

# About this manual

***Retro Dev Kit Apple 1*** — *"The Original Sin" — reproduces the 1976 Apple-1 cycle for cycle: the 6502, the video terminal, the keyboard, the ACI cassette interface and, as options, Uncle Bernie's GEN2 color card and the P-LAB TMS9918A graphics card. It comes with its own free BASIC,* ***RDK G2 Basic****. Around the machine it adds a complete workshop: a program library, an editor, an assembler, a debugger and export to a real Apple-1.*

This manual describes the whole emulator, version 1.0 (the first public release; the 7.x numbers were the test versions). It complements the *Quick Start Guide*, which is enough to install the program and play a first tape. Here every panel, every button and every command is explained, with examples you can type as they are.

## 1.1 How this manual is organized

**Part I — The emulator**

The window and its panels, the vintage screen, the library, the tape and the ACI, the GEN2 and P-LAB graphics cards, the editor, the debugger, ASCII art, the preferences, the command line and troubleshooting.

**Part II — Integer BASIC**

Steve Wozniak's BASIC and its free clone RDK G2 Basic: every statement, function and operator, memory, the error messages and the known bugs, with their workarounds.

**Part III — The 6502 assembler**

The 6502 from the ground up, the built-in assembler and its directives, the Woz Monitor routines, complete examples and the pitfalls to avoid.

**Appendices**

Memory map, Woz Monitor commands, the ACI format, the 2513 character set, keyboard shortcuts, and the command index.

## 1.2 Conventions

Keys are written like this: Ctrl+R, RETURN. Interface buttons are in bold: ▶ **Run**. Addresses are in hexadecimal, prefixed with `$` as in the documents of the time: `$FFEF`. The Woz Monitor itself writes them without `$`: `FFEF`.

Sessions on the Apple-1 screen have a dark background. What **you** type is white, the machine's answer is green:

```
300R                                                    WOZ MONITOR

0300: A2
HELLO, APPLE-1!
```

Four kinds of boxes appear along the way:

> ✦ TIP
> A shortcut, or a better way of doing something.

> ! CAUTION
> A trap that costs time or data.

> ▣ ON THE REAL MACHINE
> What happens on a real Apple-1, and what differs from the emulator.

> KNOWN BUG
> A defect of the original ROM, with its workaround. Each bug has a status: VERIFIED (reproduced in the emulator with the real Integer BASIC) or NOT VERIFIED (quoted from a source, not tried).

Every example in this manual was tried in the emulator. The 6502 programs of Part III are in the `docs/exemples/en/asm/` folder of the distribution. They produce exactly the same bytes with the built-in assembler and with ca65.

## 1.3 What you need

The emulator runs on Linux with nothing to install. The Woz Monitor and the ACI ROM are built in, and so is **RDK G2 Basic**, a free BASIC that is compatible with Integer BASIC (chapter 14): the **BASIC** button works at once, on a fresh install, with no file to find. The 1976 **Integer BASIC** of Apple is **not** supplied, because it is still under copyright. If you want the original, use your own copy (for example the `BASIC.wav` tape from the "Apple1Cassettes" collection on archive.org) and pick it in ⚙ **Preferences → Advanced → BASIC**.

> ! ROMS AND TAPES STAY WITH YOU
> The emulator ships neither Apple's Integer BASIC nor commercial tapes. Its recognition table holds only sizes, fingerprints and parameters. Keep your copies in your personal library.

> LICENCE
> Retro Dev Kit Apple 1 is free software under the **GNU GPL 3.0** (no warranty). Some parts keep their own MIT licence: RDK G2 Basic and Uncle Bernie's GEN2 tests and glyphs. See `LICENSE` and `NOTICE` in the archive, and the appendix "Licence and credits".

## 1.4 The main paths

Four ways of using the emulator come up again and again. Each is described later with its own diagram:

| I want to… | The path | See |
|---|---|---|
| play a program of the time | library → ▶ | chapter 4 |
| write a BASIC program | 📝 **Editor** → ▶ **Run** → 📼 **Export WAV** | Part II |
| write a 6502 program | 📝 **Editor** ( `.s` ) → 📃 **Listing** → 🐛 **Debugger** | Part III |
| send a program to a real Apple-1 | 📼 **Export WAV** ( `.aiff` ) → smartphone → ACI | chapter 5 |

CHAPTER 2

# The window and the toolbar

## 2.1 Overview

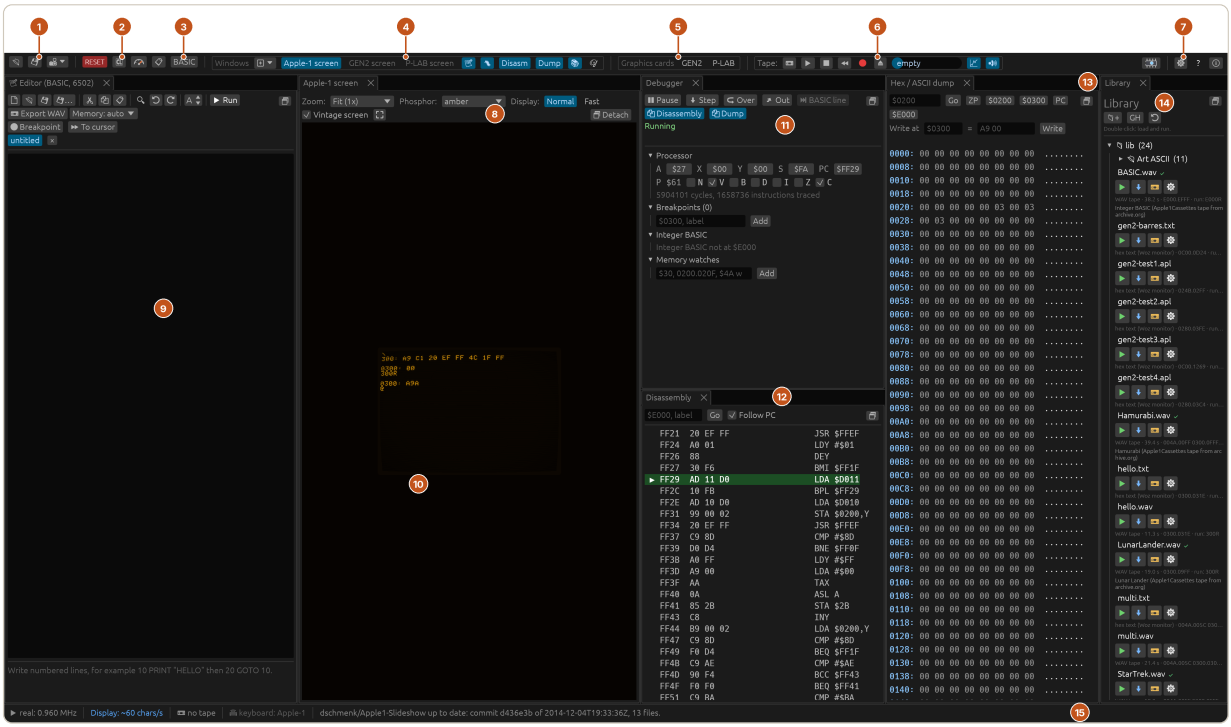



**Figure 1 —** The default layout: every panel open and docked.

1. **Load, Save, State**: `.txt` hex dumps, `.bin` and `.prg` binaries; Save also writes `.wav` / `.aiff` tapes; State saves and restores the machine.
2. **RESET, Clear screen, Speed (gauge), Paste**: the machine's buttons (Ctrl+R, Ctrl+L, F3, Ctrl+V).
3. **BASIC**: loads the built-in RDK G2 Basic at `$E000` (or your Integer BASIC, if you chose it).
4. **Windows**: the ⊞ **Layout** ▾ menu, then one button per panel (Monochrome screen, Colour screen, Editor, Debugger, Disassembly, Dump, Library, ASCII).
5. **Graphics cards**: **GEN2** and **P-LAB** plug the cards in or out.
6. **Tape**: 📼 **Insert** a tape, then play, stop, rewind, record, eject; oscilloscope and sound.
7. **EPROM**, ⚙ **Preferences**, **?** (detailed help bubbles, also F1) and **i** (about, shortcuts).
8. **Display row of the screen**: zoom (1x to 8x or Fit), phosphor, display speed **Normal** or **Fast**, vintage screen, full screen, **Detach**.
9. **Editor**: BASIC programs and 6502 sources in tabs.
10. **Apple-1 screen**: 40 × 24 characters; click it to type, right-click for Copy, Paste, Clear screen and Reset.
11. **Debugger**: pause, single step, registers, breakpoints, variables.
12. **Disassembly**: the code around the program counter (PC).
13. **Dump**: memory in hexadecimal and ASCII, editable.
14. **Library**: your programs, each with its small icons ▶ ⬇ 📼 ⚙.

15 **Status bar**: speed, display rate, tape, keyboard target; below it, the latest message.

Every panel (screen, editor, debugger, library…) is a tab that you can move, group or detach into its own window: that is the subject of chapter 10. On first launch the window opens maximized and a **Your workspace** window proposes five layouts (chapter 10); the size and position of the window and the arrangement of its panels are remembered, and found again as they were at the next launch.

## 2.2 The toolbar, button by button

The toolbar is a single row: the machine and its tools on the left, the tape deck in the middle, the panels in the Windows group, the cards, and the settings on the right. When the window is too narrow, the buttons keep only their icon (the text stays in the tooltip), or the bar wraps onto several lines if you untick **Compact bar** in the preferences.




**Figure 2 —** The toolbar up to the Graphics cards group: files, machine, windows and cards. The Tape strip follows (chapter 5), then the settings.

| Button | What it does |
|---|---|
| Load | Opens a program and puts it into memory: a `.txt` (Woz Monitor dump) or a `.hex`; for a `.bin`, the address is asked for. Tapes (`.wav`, `.aiff`) do not go through here: they are inserted with 📼 **Insert**, in the Tape strip. |
| Save | Asks for one or more ranges (`0300.03FF`, `004A.00FF 0300.0FFF`…), then a file. The extension selects the format: `.wav`, `.aiff`, `.txt` or `.bin`. |
| **RESET** (red button) | The Apple-1 RESET button (Ctrl+R). The 6502 and the PIA restart; memory and screen are kept. |
| 🆑 **Clear screen** | The CLEAR SCREEN button (Ctrl+L). |
| **Gauge (Real speed / Fast speed)** | Toggles between the original 1 MHz and maximum speed (F3). A speedometer dial: lit blue while fast speed is on. |
| 📋 **Paste** | Types the clipboard on the Apple-1 keyboard, at keyboard pace (Ctrl+V). Ideal for a hex dump. |
| **BASIC** | Loads the built-in **RDK G2 Basic** at `$E000` (7,933 bytes, then `E000R` starts it). If you chose the original Integer BASIC in ⚙ **Preferences → Advanced → BASIC**, it loads that one instead: a tape, a `.txt` dump or a 4096-byte `.bin`, looked for in the library (`BASIC.wav`, `integer_basic.bin`, any name containing "basic"), otherwise asked for. Right-click: choose another file. |
| **Windows** (group: ⊞ **Layout** ▾ and one button per panel) | Shows or hides each panel: **Monochrome screen** (the Apple-1 terminal), **Colour screen** (the screen of the graphics card that is plugged in, GEN2 or P-LAB: the same button serves both; with no card plugged in, it opens a choice to plug one, and **P-LAB** then shows its screen), 📝 **Editor** (chapter 7), 🐛 **Debugger**, **Disassembly**, **Dump** (chapter 8), 📚 **Library** and 🎨 **ASCII** (chapter 9). A lit button means the panel is visible; a panel that is already shown can also be closed with the cross of its tab. |
| **Graphics cards: GEN2 and P-LAB** | A named group. **GEN2** plugs in or unplugs Uncle Bernie's color card (chapter 6); **P-LAB** does the same for the P-LAB TMS9918A card (section 6.6). A button is lit while its card is plugged in; the screen of a card appears in its own panel, shown by the **Colour screen** button. |
| 📸 **State** ▾ | A single button with a menu: **Save state…** (Ctrl+Shift+S), **Load a state…** (Ctrl+Shift+O), then the recent states, each with its title and date (section 2.5). |
| **EPROM** (chip with a UV window, on the right of the bar, before ⊞ **Layout**) | Opens the EPROM programmer: read, write or verify a chip with a `minipro` programmer (section 2.6). |
| **Tape:** 📼 **Insert** ▶ ■ ⏪ ● ⏏ | The deck, gathered in one strip. 📼 **Insert** picks a `.wav` or `.aiff` and puts it in the deck as a virtual tape; then play, stop, rewind, record, eject. The strip shows the name, position and length. |
| 📈 and 🔊 | The tape oscilloscope, and the sound of the signal (chapter 5). |
| ⊞ **Layout** ▾ (first button of the Windows group) | A menu: **Default** (every panel, as at the first launch), **Game** (Apple-1 screen and library), **BASIC** (editor and screen), **Assembly** (editor, screen and debugger), **Debugging** (editor, debugger, disassembly and dump); then **Save the current layout…**, which asks for a name, and your saved layouts in alphabetical order (right-click: Rename, Delete). A tick marks the last choice; everything is kept in `gui.json`. |
| ⚙ | The preferences (chapter 11). |
| **?** | Help mode (F1): detailed bubbles on every control, in the 7 interface languages. |

| Button | What it does |
|---|---|
| ℹ | “About”, with the list of keyboard shortcuts. |

**File windows.** Open, Save as and Export use the emulator’s own file window. A blue 📁+ **New folder** button, at the top left, creates a folder in the folder being shown, with a tooltip to explain it; type the name, press Enter, and the new folder is ready to receive your file.

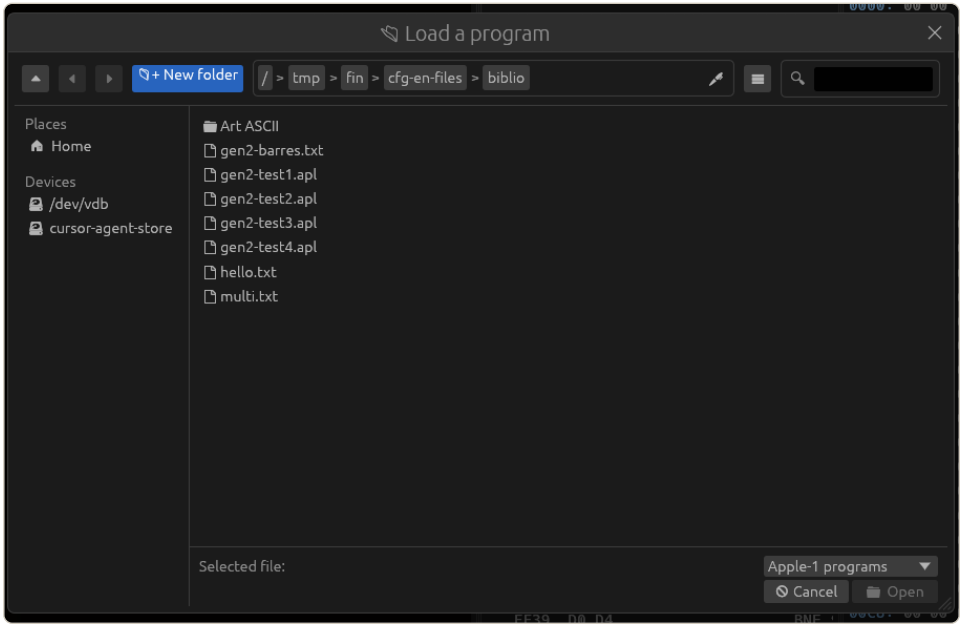



**Figure 3 —** The Load window, with the 📁+ New folder button at the top left.

The display settings are not in the toolbar: each screen has its own **display row**, at the top of its panel (section 3.2).




**Figure 4 —** The display row of the Apple-1 screen: zoom, phosphor, display speed, vintage screen and full screen; Detach is at the right end of the row.

> ✦ **HELP MODE**
>
> When you explore a panel, switch **?** on: every button explains what it does when you hover over it. Off, only the usual short tooltips remain (they can be turned off in ⚙ → **Interface**). The setting is remembered.

## 2.3 The status bar

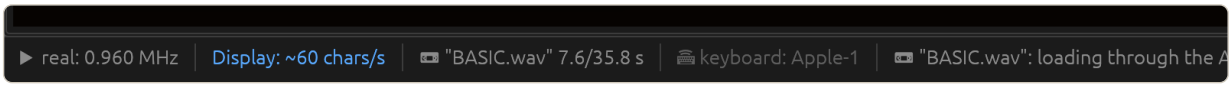



**Figure 5 —** The status bar while BASIC.wav is playing: speed, display limit, tape, keyboard, and the latest message below.

- **Measured speed**: about 0.960 MHz at real speed. This is the effective clock of the Apple-1, which halts the 6502 for four cycles out of sixty-five to refresh its dynamic memory.
- **Display ~60 chars/s**: the Apple-1 terminal displays only one character per frame, about 60 per second. Click to switch between this limit and instant display, like the **Normal** / **Fast** buttons of the display rows.
- **Tape**: name, position and length, or "no tape".
- **Keyboard**: where the keys go (the Apple-1, or a text field of the interface).

## 2.4 The Apple-1 keyboard

Click the screen: the keys go to the Apple-1. The original keyboard has upper case only, and the emulator converts lower case.

| PC key | For the Apple-1 |
|---|---|
| letters, digits, symbols | the character, in upper case |
| Enter | RETURN (code `$8D`) |
| Backspace | `_`: the Woz Monitor and BASIC rub out the previous character (it stays on screen, followed by `_`) |
| Esc | ESC: cancels the current line in the Woz Monitor |
| Ctrl+letter | the control character (`Ctrl+D` = `$84`...) |
| Ctrl+R / Ctrl+L | the RESET and CLEAR SCREEN buttons |

You can **select text with the mouse** on either screen: press and drag over the characters, and the selection is highlighted. **Right-clicking** either screen (the Apple-1's or the GEN2 monitor) opens a small menu: **Copy** puts the selected text on the clipboard (or the whole text of the screen when nothing is selected), **Paste (type the text)** types the clipboard on the Apple-1 keyboard, as if you were typing it, then **Clear screen** and **Reset**.

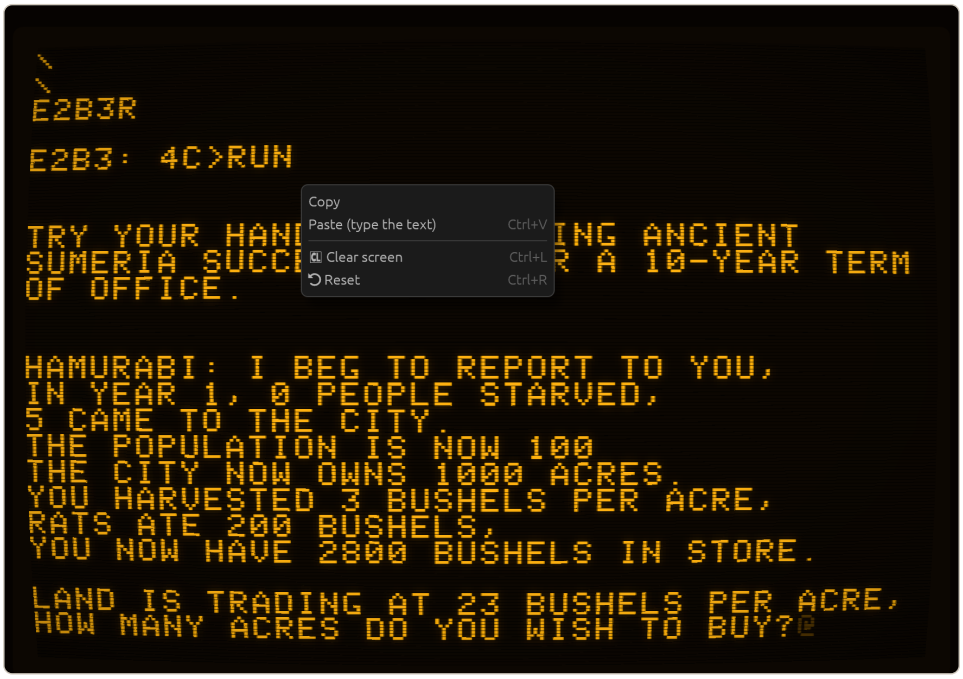



**Figure 6 —** The right-click menu of the Apple-1 screen, during a game of Hamurabi.

> ▣ **THE APPLE-1 SCREEN ERASES NOTHING**
>
> The terminal can only write a character and move to the next line. There is no movable cursor and no way to erase a character: the rub-out key prints `_` and the character stays on screen. This is normal.

## 2.5 Machine state

The 📸 **State** button of the toolbar takes a complete snapshot of the machine and puts it back later, exactly at the same point: you can stop a game or a program that is running, close the emulator, and resume it the next day. The button opens a menu:

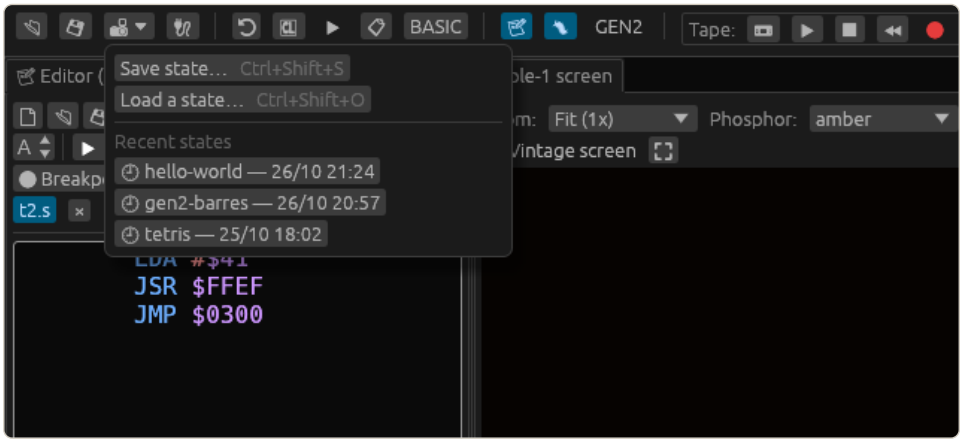



**Figure 7 —** The State menu: save, load, then the recent states with their date.

- **Save state...** (Ctrl + Shift + S) asks for a file name and writes a `.a1state` file. The proposed folder is `etats`, next to the library folder.

- **Load a state…** (Ctrl + Shift + O) replaces the machine by the chosen state. The status bar shows the file and the value of PC.
- **Recent states** lists the last eight, with their title and the date of the save; a click loads one at once. If the file has been moved or deleted, a message says so.

A state contains the memory, the 6502 registers, the PIA, the screen, the GEN2 card if it is plugged in, and the virtual cassette with its position. It does not contain your preferences or the library. A file that cannot be read (damaged, or not a state) is refused with a message, and the machine is left untouched.

> ✦ **TYPICAL USE**
>
> Save a state just before a difficult passage of a game, or at the end of a long BASIC session: a state is also a convenient way to share a running program with a friend who uses the same version.

## 2.6 The EPROM programmer

**EPROM** in the toolbar opens a window to read, write and verify an EPROM or EEPROM chip from the emulator, for example to burn the BASIC ROM, a program for the GEN2 expansion, or the contents of your own memory.

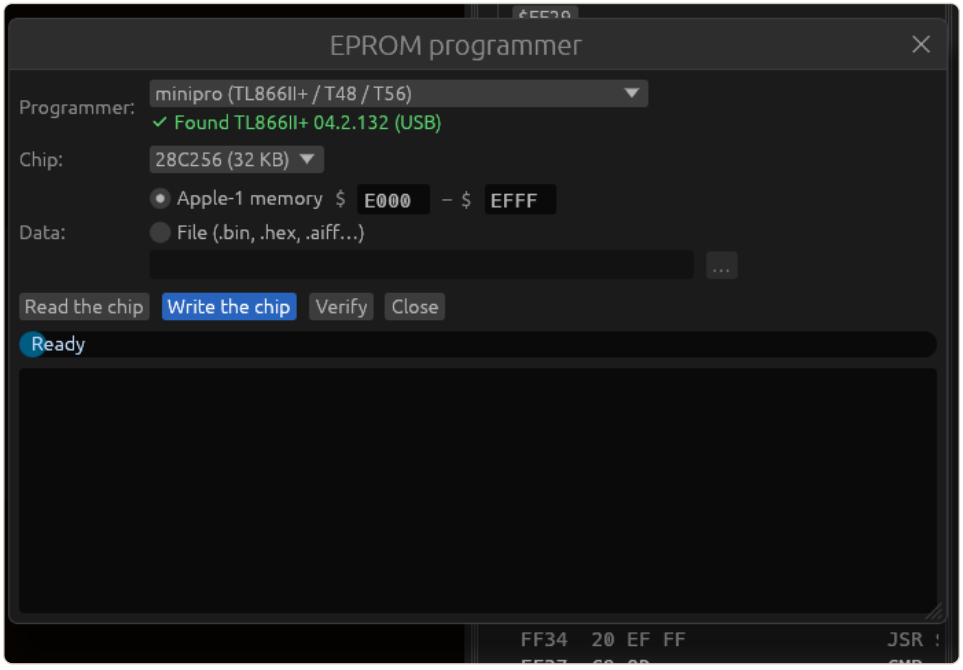



**Figure 8 —** The EPROM programmer: programmer, chip, data source and the three operations.

Only the programmers driven by the free `minipro` tool are supported: the XGecu **TL866II+**, **T48** and **T56**. On Debian, Ubuntu and Linux Mint, install it with `sudo apt install minipro`, plug the programmer into a USB port, then press **Detect**. If `minipro` is missing, the window says so and explains what to do; nothing else is needed on the emulator's side.

| Setting | Role |
|---|---|
| **Programmer** | the detected programmer, with its firmware (for example “Found TL866II+ 04.2.132 (USB)”) |
| **Chip** | the common chips of the Apple-1 world: 2716, 2732, 2764, 27C64, 27C128, 27C256, 27C512, 28C64, 28C256, SST27SF256, SST27SF512, and the 23128 mask ROM (read only) |
| **Data** | the **Apple-1 memory** (an address range, `$E000` – `$EFFF` by default), or a **file** ( `.bin` , `.hex` , `.aiff` …) |

Three buttons act on the chip, and a progress bar and a log show what is happening:

- **Read the chip** reads the whole chip. With the memory or an existing file as the data, the content is **compared** and the differences are listed: an existing file is never overwritten. With a new file name, the content is saved in it.
- **Write the chip** programs the chip with the chosen data. The chip must be blank or erasable, and a confirmation of the result is shown at the end.
- **Verify** compares the chip with the chosen data, byte for byte, and reports each difference ( `$E010: chip $4C, data $A9` ).

**Close** closes the window.

The window never changes the emulated memory: writing and verifying read it, and reading only compares it or fills a file.

CHAPTER 3

# The screen: zoom, phosphor and vintage effect

## 3.1 The original screen

The Apple-1 displays 40 columns × 24 lines of upper-case characters. Each character is a 5 × 7 dot matrix in a 7 × 8 cell, read from the **Signetics 2513** character generator ROM: the emulator uses the contents of that ROM, dot for dot (Appendix D). The picture is therefore 280 × 192 dots.

The cursor is a blinking `@`, driven by an NE555: 2.18 Hz, visible about two thirds of the time. On some boards the capacitor tolerance gives 1.7 Hz instead; the `--cursor-hz 1.7` option reproduces it.

## 3.2 The display row

Each screen has its own row of settings at the top of its panel: the Apple-1 screen, and the color monitor when the GEN2 card is plugged in (chapter 6).

| Setting | Role |
|---|---|
| **Zoom** | a list: `1x` to `8x` magnify the picture by an integer factor, so the dots stay square and sharp; **Fit** chooses the largest factor that fits in the panel (the chosen factor is shown in brackets). From the keyboard: Ctrl + +, Ctrl + −, Ctrl + 0 (fit). |
| **Phosphor** | Apple-1 screen only: the green of the monitors of the time, the white of a TV set, or amber. |
| **Display: Normal / Fast** | **Normal** shows the characters at the pace of the real terminal, about 60 per second; **Fast** shows everything at once. This setting is shared: changing it on one screen changes it on the other. |
| **Vintage screen** | the 1970s tube effect (next section); each screen has its own checkbox. |
| ⛶ | full screen (F11). |
| **Detach** | puts the screen in its own window (chapter 10). |

Each screen is **centered** in its panel, horizontally and vertically, with its caption centered below the picture. If the chosen zoom is larger than the panel, the screen is shown at the largest zoom that fits instead of being cut off. When the window is narrower than the display row, the row wraps onto a second line, and **Detach** stays at the right end of the last line.

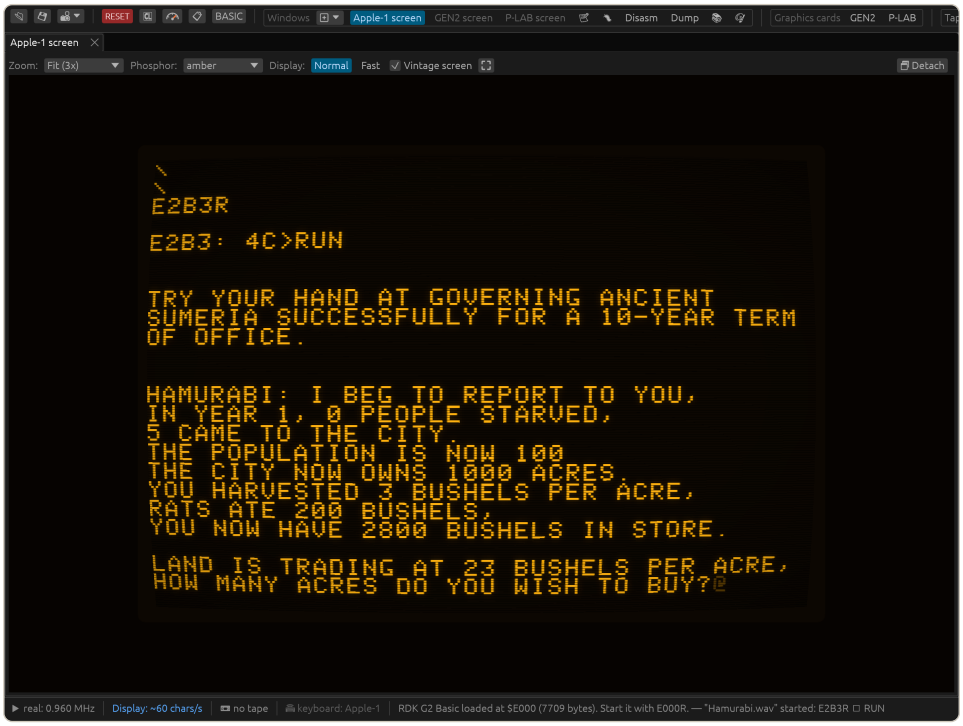



**Figure 9 —** Amber phosphor, with the vintage screen and a slight curvature.

## 3.3 The vintage screen

The **Vintage screen** checkbox of the display row imitates a 1970s CRT monitor. The main effect is the thin dark line between two scan lines: each row of dots becomes a horizontal band, as in a photograph of a real Apple-1. This line keeps the same thickness in pixels of your display at every zoom: it stays thin and sharp, even full screen.

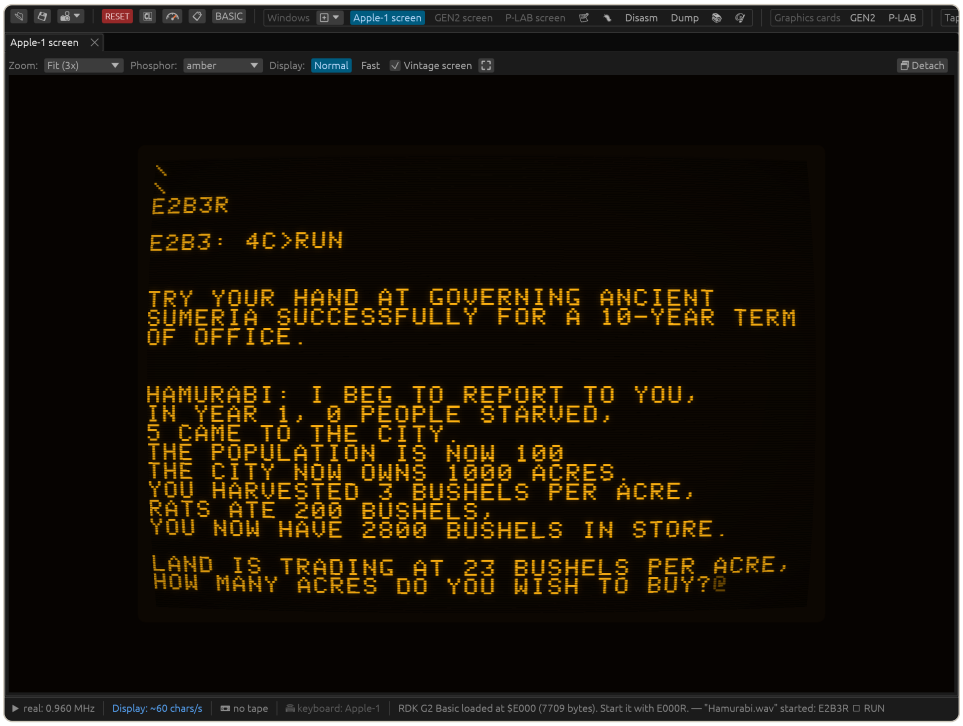



**Figure 10 —** The vintage screen, with a slight tube curvature (0.15) and dark corners, during a game of Hamurabi.

In ⚙ **Preferences → Advanced → Vintage screen: fine settings**, you set the **Intensity** and choose the effects; these settings are shared by both screens. Each slider shows its **numerical value** next to it (curvature, intensity…):

| Effect | Default | Role |
|---|---|---|
| gaps between scan lines | on | the dark line between two rows of dots |
| horizontally softened dots | on | a slight blur, horizontal only, like the beam |
| phosphor afterglow | short | dots fade in less than a tenth of a second (a little more slowly in amber); far left: none |
| glow | on, subtle | a halo around the dots, which stay sharp |
| curved tube and dark corners | off | with a **Curvature** slider |
| flicker | off | a very slight brightness variation |
| RGB phosphor mask | on | colour screen of the GEN2 card only: fine vertical red, green and blue stripes separated by thin dark lines, as on a colour TV tube. It is visible from zoom 3 up, and its strength follows the **Intensity** |

The effect can also be applied to the color monitor of the GEN2 card, with the checkbox of its own display row. It is computed by the CPU, at the Apple-1 resolution, and only when the picture changes: it stays smooth even without graphics acceleration.

On the command line: `--screening`, `--sans-screening`, `--gen2-screening` (GEN2 monitor), or for example `--screening-options intensite=0.7,remanence=0.5,courbure=0.4,sans-halo` (values from 0 to 1; the option names are French).

> ✦ **DETACHED SCREEN AND SECOND MONITOR**
>
> **Detach**, at the end of the display row, puts the Apple-1 screen in its own window. Place it on a second monitor, full screen, with the vintage effect: the keyboard goes to the Apple-1 as soon as that window has the focus. With **Fit**, the picture follows the window size, in integer steps (**Integer scale**, in the window's title bar, the sharpest) or filling it.

CHAPTER 4

# The library

*The library is the starting point of almost everything: it keeps your tapes, dumps and sources, recognizes the tapes of the time, and runs a program in one click, BASIC included.*

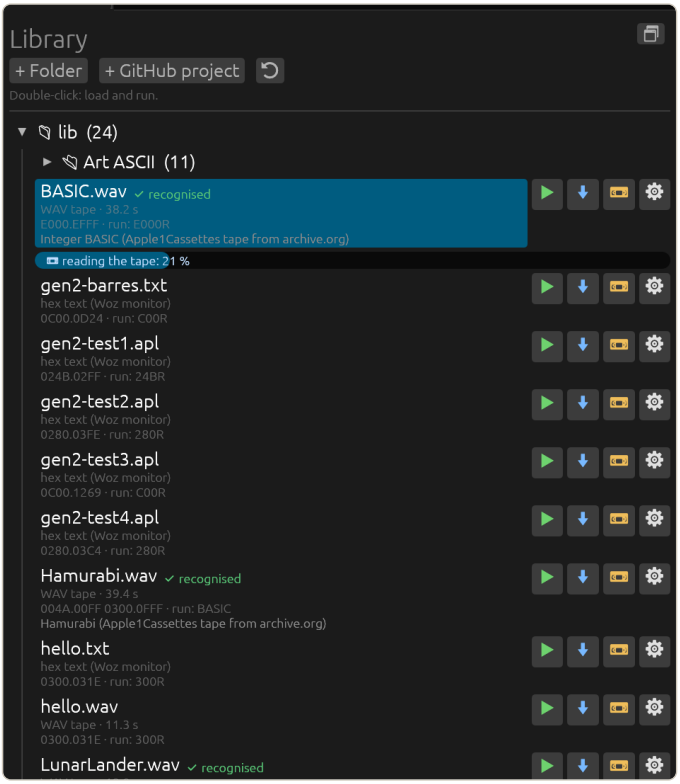



**Figure 11 —** The library panel: the main folder, the "Art ASCII" folder, recognized tapes and a GitHub project. BASIC.wav is playing.

## 4.1 Folders

The main folder is `~/apple1/programmes`. It is created on first launch, with the examples sorted into five folders: **Art ASCII** (eleven 40 × 24 ASCII-art examples, chapter 9), **BASIC** (`.bas` and BASIC `.hex.txt` programs), **6502** (`.s` / `.asm` sources with their `.bin` / `.hex`), **GEN2 COLOR** (Bernie's GEN2 card tests, `gen2-*.apl`, and the colour games for G2 Basic) and **WAV** (all the cassettes). In the panel those five folders are listed directly at the top level, in the order **BASIC**, **6502**, **GEN2 COLOR**, **WAV**, **Art ASCII**: the root folder (the directory's name on disk) is not shown above them, and files left loose in that folder are not listed. Put them into one of the five folders. A folder you add, or a GitHub project, keeps its own title. A library that already exists is never reorganised, except that its

**Assembleur** and **Uncle Bernie** folders (the names of versions 7.8 to 7.9.1) are renamed **6502** and **GEN2 COLOR**, with their content kept. To choose another one: ⚙ **Preferences** → **Library folder** → 📂 **Browse...** (⟲ **Default** goes back to `~/apple1/programmes`).

**+ Folder**, at the top of the panel, adds other folders; the main folder always stays. The library is a tree: folders and subfolders fold up, and their state is remembered. Each folder keeps its own settings in a `programs.json` file. Right-click an added folder to open it in the file manager or remove it from the list (nothing is deleted on disk).

**Narrow panel.** The library can be as narrow as you like (down to a minimum width, below which the divider stops). Each entry then adapts: the name takes the whole width, cut with "..." if it is too long (the tooltip shows the full name), the small icons ▶ ⬇ 📼 ⚙ move to the next line, and the description is limited to one or two lines, also cut with "...", the full text being in the tooltip. In the header, **+ Folder** and **+ GitHub project** become icons with tooltips.

The files shown are: `.wav .aiff .bin .hex .txt .apl .bas .s .asm .prg`. The list updates itself every two seconds: copy a file into the folder and it appears.

For each program the list shows the name, the type, the address ranges, the tape length, the run command and your notes.

### The games of Claude IA

The **BASIC** and **6502** folders also receive five small games written by Claude IA, each in a French and an English version: *Hangman* (`pendu.bas`, `hangman.bas`), *Matches* (`allumettes.bas`, `matches.bas`) and *Battleship* (`bataille.bas`, `battleship.bas`) in BASIC; *Tunnel* (`tunnel.s`, `tunnel-en.s`) and *Tic-tac-toe* (`morpion.s`, `tictactoe.s`) in 6502 assembly. Double-click a BASIC game to run it on RDK G2 Basic (the emulator starts BASIC, loads the listing and types `RUN`); assembly sources are assembled by the built-in assembler. The assembly sources include `apple1.inc`, which is installed with them. The **GEN2 COLOR** folder also holds *Connect 4 in colour* (`puissance4-couleur.bas`), a game for G2 Basic and the GEN2 card, credited to Claude IA. Q and S move the counter above the grid, Space drops it. The games are part of the project and under its licence (appendix "Licence and credits").

## 4.2 Running a program

Each program has four small icons on its right:

| Icon | Action |
|---|---|
| ▶ | **Load and run**: loads the program if needed, then runs it: RESET, then its command typed on the keyboard (`300R`, or `E2B3R` and `RUN` for a BASIC program). If the program needs Integer BASIC and it is not at `$E000`, the emulator loads it first. A BASIC listing is typed into Integer BASIC, a 6502 source is assembled. |
| ⬇ | **Load**: puts the program into memory, instantly, without starting it. |
| 📼 | **Through the tape**: as on the real machine. The emulator presses RESET, types `C100R` then the ACI read command (for example `0300.031ER`), and the tape plays at real speed, with its sound and a progress bar. `.txt` and `.bin` files are converted to a tape first. Greyed out for a source (`.bas`, `.s`), which has no recording. |
| ⚙ | **Parameters**: ranges, load address of a `.bin`, run address and command, type, whether BASIC is needed, notes. |

Loading is **instant** unless you choose the tape with 📼. A **double-click** on a program loads it, and runs it if it has a run command (like ▶). The cassette is used only when you ask for it; the deck of the toolbar then shows the tape and its position.

When the editor is open, ▶, ⬇ or a double-click on a BASIC listing or a 6502 source also **opens it in a tab of the editor**, marked 📚; if it is already open, its tab is simply shown.

**Right-clicking** a program offers the rest: load, load and run, insert as a tape without typing anything, parameters, rename, delete (with confirmation), open the folder.

## 4.3 A program's settings

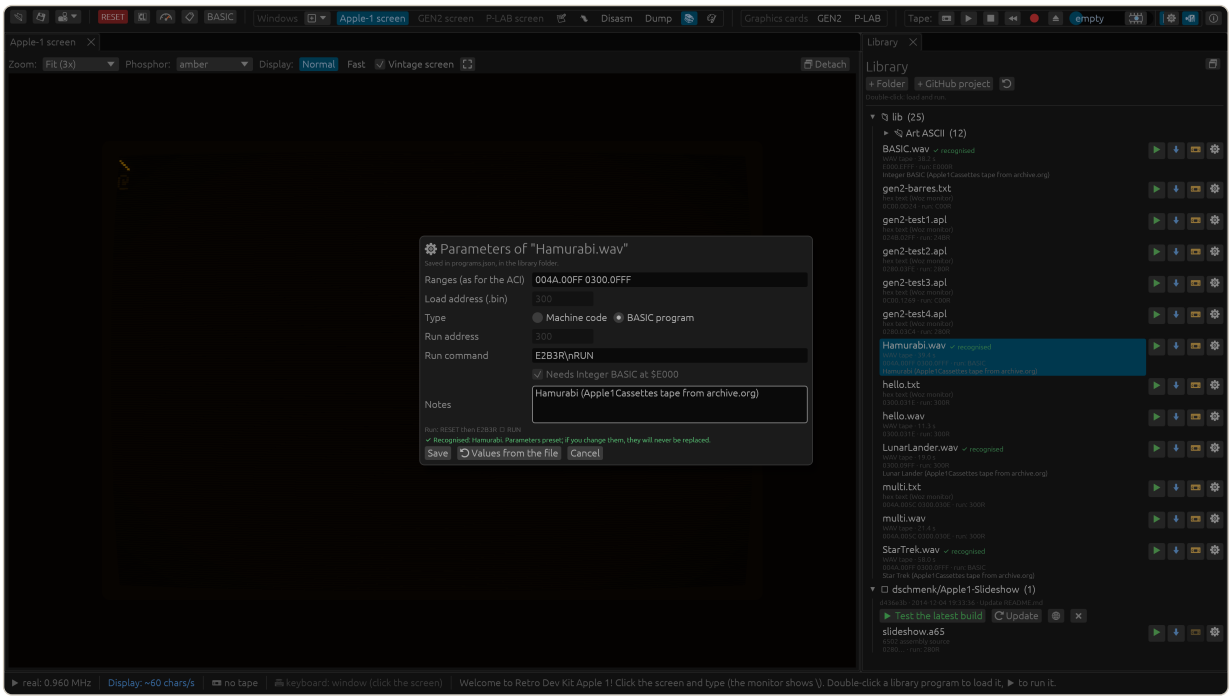



**Figure 12 —** The ⚙ settings of Hamurabi: two ranges (BASIC zero page, then the program), started with E2B3R then RUN.

| Field | Role |
|---|---|
| **Ranges** | The memory areas, written as for the ACI: `0300.031E` , or `004A.00FF 0800.0FFF` for a BASIC program. Each record on the tape goes into its range, in order. |
| **Load address** | For a `.bin` , which holds no address. |
| **Run** | The address and the command typed by ▶. `\n` stands for RETURN between two lines: `E2B3R\nRUN` . |
| **Type** | **Machine code**, **G2 Basic program** (the default for a BASIC listing, since the emulator provides it) or **Integer BASIC program** (for those who prefer Apple's original: ▶ then loads the original Integer BASIC you chose in the preferences). The type chosen for a program wins over the general BASIC setting of the preferences. |
| **Needs a BASIC** | ▶ will load the BASIC of the program's type first. |
| **Notes** | Free text, shown in the list. |

Values are prefilled from the file and checked when saved; ↻ **Values from the file** reads them again.

> **! ALWAYS FOUR DIGITS: 004A.00FF, NEVER 4A.FF**
>
> The settings refuse `4A.FF`. This is not a whim: the ACI ROM does not clear its hex accumulator at the period, so the digits keep shifting in and it would read `$004A-$4AFF` (see section 5.6). The emulator therefore always writes addresses with four digits.

## 4.4 Recognized tapes

When a file is **recognized**, its settings fill themselves in and the list shows ↻ **recognised**. The emulator decodes the records and compares their sizes and SHA-256 fingerprints with a built-in table: the 24 tapes of the "Apple1Cassettes" collection on archive.org. The file name then does not matter at all. Failing that, it recognizes the **name** (case, spaces and dashes ignored) with the same record sizes; the help bubble then points out that the contents differ.

| Tape | Ranges | Run |
|---|---|---|
| `BASIC.wav` (Integer BASIC) | `E000.EFFF` | `E000R` |
| `Hamurabi.wav`, `StarTrek.wav`, `Wumpus.wav`, `Slots.wav`, `Craps.wav`, `Bowling.wav`, `Buzzword.wav`, `WordCross.wav`, `AppleiEnhancedCheckers.wav`, `LunarlanderasciiGraphics.wav` | `004A.00FF 0300.0FFF` | `E2B3R ⏎ RUN` |
| `AceyDucey.wav`, `Blackjack.wav`, `HiLo.wav` | `004A.00FF 0800.0FFF` | `E2B3R ⏎ RUN` |
| `Football.wav` | `004A.00FF 0400.2FFF` | `E2B3R ⏎ RUN` (12 KB) |
| `DealOrNoDeal.wav`, `StarTrek2003.wav` | `004A.00FF 0300.3FFF` | `E2B3R ⏎ RUN` (16 KB) |
| `APPLE30TH.wav` | `0280.0FFF` | `280R` |
| `ASMmchess.wav` | `0300.0BFF` | `300R` |
| `Microchess.wav` | `1000.18FF` | `1000R` |
| `LunarLander.wav` | `0300.09FF` | `300R` |
| `Mastermind.wav` | `0300.03FF` | `300R` |
| `Pasart.wav` | `0300.05FF` | `300R` |
| `Life.wav` | `2000.21FF` | `2000R` (12 KB) |

Your own settings are never overwritten: a program you have set up yourself keeps its settings, only the "recognised" mark appears. Programs that go beyond the basic 8 KB automatically get the RAM they need when started, as with an expansion card.

**Unknown tape.** The emulator guesses what it can: a 4096-byte record that starts with `JMP` is Integer BASIC; two records of which the first is 182 bytes long (`$4A-$FF`) are a BASIC program, placed according to its LOMEM and HIMEM pointers. The list then shows "(guessed)"; check with ⚙. A machine-code program without addresses is not guessed: give its range.

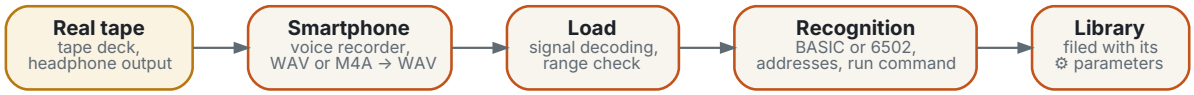



**Figure 13 —** Digitizing a real tape: from the cassette to a file, then to the library.

## 4.5 GitHub projects

**+ GitHub project** accepts a URL ( `https://github.com/jefftranter/6502/tree/master/asm/jmon` ) or simply `owner/repository` . The useful files are copied to `~/.cache/apple1-emu/github/<owner>/<repository>/<commit>/` , and the project becomes a node of the tree, with its commit, date and message. ⟳ **Update** fetches the latest commit; 🌐 opens the repository page. All network access happens in the background.

Test the latest build tries, in order:

1. the files of the repository's latest **Release** ( `.wav` / `.aiff` , then `.bin` , then `.hex` / `.txt` );
2. the latest **GitHub Actions artifact** (a token is required);
3. **building from source**: a BASIC listing is typed into Integer BASIC then started with `RUN` ; a 6502 source is assembled with **ca65/ld65** (package `cc65` ), with `-D APPLE1` by default; a Woz dump is loaded.

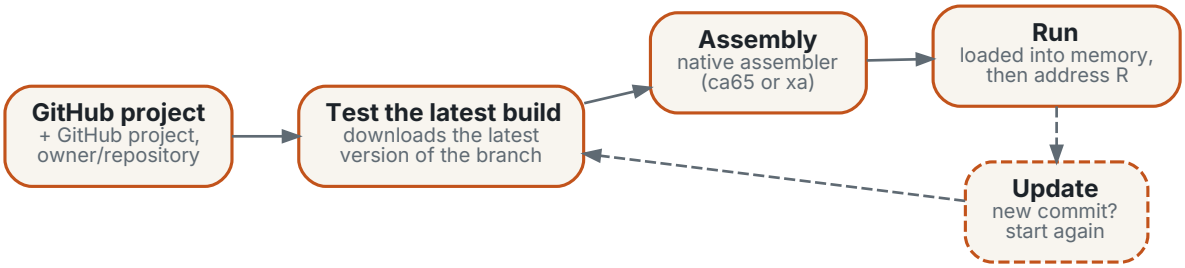



**Figure 14 —** A GitHub project, from the repository URL to the running program.

An `apple1.toml` (or `apple1.cfg` ) file at the root of the repository says what to build:

apple1.toml — TOML

```toml
file    = "src/game.s"    # program to run
load    = $0300           # load address (otherwise the source's .org)
run     = $0300           # run address (otherwise the load address)
ld65    = "apple1.cfg"    # ld65 configuration (otherwise -t none)
defines = "APPLE1"        # -D symbols for ca65
```

✦ THE GITHUB TOKEN IS OPTIONAL

Public repositories do not need one. A token ( `GITHUB_TOKEN` variable, or ⚙ **Preferences**) raises the request limit and gives access to your private repositories and to artifacts. It is never displayed or written into messages, and `gui.json` then becomes readable by your account only.

## 4.6 The “Art ASCII” folder

The library also contains an **Art ASCII** folder: drawings in the screen format (apple, Apple-1 board, 6502 chip, cassette, rocket…) and portraits of Steve Wozniak and Steve Jobs converted from freely licensed photos. An ASCII-art `.txt` shows a preview when selected, with → **BASIC**, → **6502** and 🎨 **Open in the tool**; ▶ runs its 6502 version right away. Chapter 9 describes the tool.

CHAPTER 5

# The tape and the ACI card

*The Apple-1 had neither floppy disk nor BASIC in ROM: everything went through an audio tape and the ACI card (Apple Cassette Interface). The emulator runs the real ACI ROM, so its WAV and AIFF files load on a real Apple-1, and recordings made on a real Apple-1 load in the emulator.*

## 5.1 File formats

| Extension | Contents | Read | Write |
|---|---|---|---|
| `.wav` | ACI tape, 8- to 32-bit PCM or float, mono or stereo, any sample rate | yes | 44.1 kHz mono 16-bit by default |
| `.aif`, `.aiff` | ACI tape (AIFF and AIFF-C) | yes | smartphone profile: 48 kHz, 2 identical channels, 16-bit |
| `.txt`, `.hex`, `.apw` | Woz Monitor dump: lines `0300: A9 C1 20 …` | yes | yes |
| `.bin` | raw bytes, no address ( `FILE.bin@300` ) | yes | yes |
| `.bas` | Integer BASIC listing (text) | in the editor | in the editor |
| `.s`, `.asm`, `.a65` | 6502 source (ca65, or xa) | in the editor | in the editor |

Tapes exported by the emulator carry their addresses in a comment chunk of the file ( `APPLE1 ACI 0300.031E` ), which audio players ignore. A tape from elsewhere has no addresses: the library recognizes or guesses them (chapter 4), and the command line accepts `--addr` or `--range`.

The decoder tolerates noise, low levels, inverted polarity, a tape speed error of ±10 % and several records in one file. WAV files made by Egan Ford's **c2t** tool are read too. A **mains hum** (50 or 60 Hz, even as strong as the tape signal itself) is filtered out before the edges are detected, so a long recording made with a poorly shielded cable still reads correctly.

## 5.2 The virtual deck

The **Tape:** strip of the toolbar gathers every tape control: 📼 **Insert**, ▶ play, ■ stop, ⏪ rewind, ● record, ⏏ eject, then the tape's name and position, the oscilloscope 📈 and the sound 🔊. A tape is inserted with 📼 **Insert** (a `.wav` or `.aiff` ), or by right-clicking a library program ("Insert as a tape"): you then type `C100R` and the range yourself. The 📼 icon of a library program goes one step further: it inserts the tape and types the commands for you.

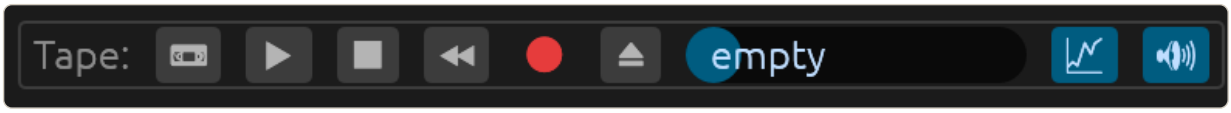



**Figure 15 —** The Tape: strip while BASIC.wav is being read: 7.6 seconds out of 35.8.

The tape follows **emulated** time: at fast speed (F3), loading takes far less than the 10-second header, and it still works. Without the tape, library loads are instant (chapter 4).

Loading Integer BASIC "the old way" goes like this:

```
WOZ MONITOR
C100R
C100: A9*
E000.EFFFR
E000R

E000: 4C
>
```

`C100R` starts the ACI program, which answers with its `*` prompt. The line `E000.EFFFR` asks it to **read** the tape into `$E000-$EFFF`. When the tape ends, the ACI returns to the monitor ( `\` ); `E000R` starts BASIC, whose prompt is `>`.

## 5.3 Sound and oscilloscope

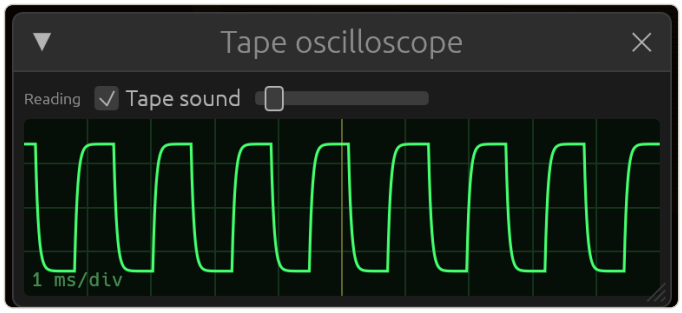



**Figure 16 —** The oscilloscope while BASIC.wav plays: the "Tape sound" checkbox, the volume, and the signal at 1 ms per division.

During playback or recording, 🔊 lets you hear the signal, like the speaker of a tape recorder, at low volume by default. 📈 opens the **oscilloscope**: a green trace on a dark graticule, 1 ms per division, showing 10 ms around the playback head, or the last 10 milliseconds written by the Apple-1. You can make out the header (≈ 800 Hz), the 0 bits (≈ 2 kHz) and the 1 bits (≈ 1 kHz). Sound is muted at fast speed.

## 5.4 Recording from the emulated Apple-1

To produce a tape as in 1976, arm the recorder, then ask the ACI to write:

1. Click ● and choose the file ( `.wav` or `.aiff` ).
2. Type `C100R`, then the range followed by `W`, for example `0300.03FFW`, then RETURN.

3 The ACI writes a 10-second header, then the data. The file is written 1.5 s after the end, or on ■.

The simplest way is still **Save** in the toolbar, or 📼 **Export WAV** in the editor: the signal is the same, cycle for cycle, and it is produced instantly.

## 5.5 How the ACI works

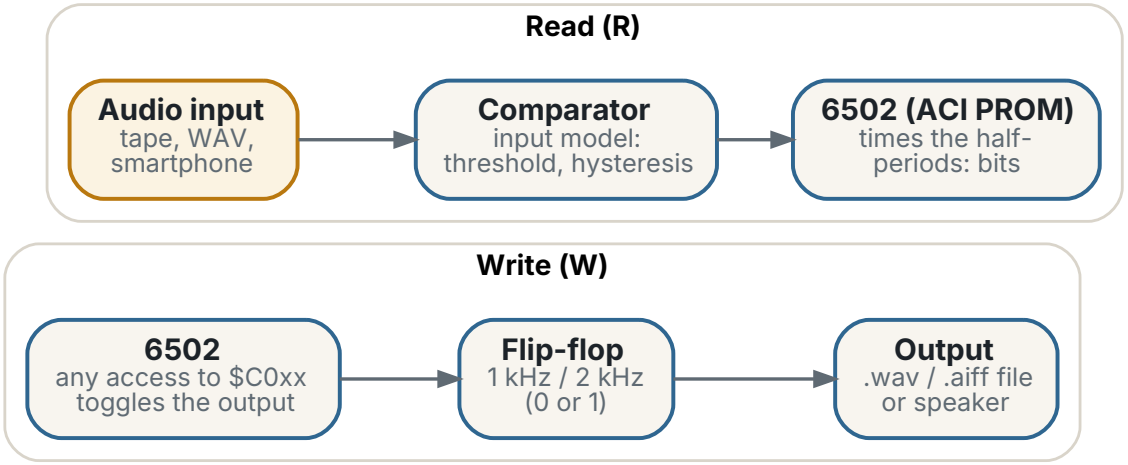



**Figure 17 —** The two directions of the ACI: when writing, any access to $C0xx toggles the output; when reading, the comparator drives address line A0 of the PROM.

The card is remarkably simple: a 256-byte PROM visible at `$C000` and `$C100`, a flip-flop and a comparator.

- **Writing.** Every access to an address in `$C000-$C0FF` toggles the output (74LS74). The ROM program sets the time between two accesses: that time encodes the bits.
- **Reading.** The level of the tape input, after the LM311 comparator, drives PROM address line A0 when A7 = 1. The same instruction therefore reads `$C080` or `$C081` depending on the level: the ROM measures the half-periods with a 12-cycle loop.
- **Encoding.** A "1" lasts about twice as long as a "0": 1 kHz for a 1, 2 kHz for a 0. Bytes go out most significant bit first, with no parity and no checksum.

Appendix C gives the exact timings, in cycles and in microseconds.

## 5.6 ACI commands

After `C100R`, the ACI waits for a line in monitor format, ending with `R` (read) or `W` (write):

| Line | Effect |
|---|---|
| `0300.03FFR` | reads the tape into `$0300-$03FF` |
| `0300.03FFW` | writes `$0300-$03FF` to tape, with a 10 s header |
| `004A.00FFR 0300.0FFFR` | reads **two** consecutive records: the BASIC zero page, then the program |
| `004A.00FFW800.FFFW` | the run-together form from the 1976 BASIC manual, also accepted |

> **! THE ZEROS OF 004A.00FF ARE MANDATORY**
>
> The ROM parser does not clear its hex accumulator at the period (“Doesn’t clear HEX1!!!”, says the listing). The end digits are shifted in behind those of the start: `4A.FF` means `$004A-$4AFF`, almost 19 KB! `0300.0FFF` works because the four digits push the 3 of `0300` out. Always write both addresses with four digits. The emulator reproduces this behavior, since it runs the real ROM.

A BASIC program is saved as two records: the zero page `$004A-$00FF` (which holds the LOMEM and HIMEM pointers and the end of the program), then the program itself, which always ends at HIMEM. To load it back, BASIC must already be in memory; you then re-enter BASIC “warm” with `E2B3R`, which keeps the program.

WOZ MONITOR

```
C100R
C100: A9*
004A.00FFR 0300.0FFFR
E2B3R

E2B3: 20
>RUN
```

## 5.7 To a real Apple-1

This is what justifies the emulator: prepare a program on the PC, then play it into the ACI of a real machine, from a smartphone or a computer’s headphone output.

1. **Prepare the file.** 📼 **Export WAV** in the editor, or `apple1-emu export prog.txt -o prog.aiff`. Choose `.aiff` for a smartphone. Note the line displayed, for example `C100R` then `0300.031ER`.
2. **Set up the player.** Volume at maximum (accept the “high volume” warning), every audio effect off (equalizer, Dolby, “audio enhancement”), airplane mode, no Bluetooth. A simple player such as VLC is fine.
3. **On the Apple-1**, type `C100R`: the monitor answers `C100: A9*`.
4. Type the read line, for example `0300.031ER`, but **do not press RETURN yet**.
5. Start playback, then press RETURN **during the header** (it lasts about 10 s, after one second of silence).
6. At the end, the ACI returns to the monitor ( \ ). Run the program: `300R`, or `E2B3R` then `RUN` for a BASIC program.

**Why this AIFF profile?** Uncompressed 16-bit, no lossy codec: MP3 or AAC would round off the edges. 48 kHz, Android’s native rate, to avoid any resampling. Two strictly identical channels, because a cable that ties left and right together onto FROM TAPE must never receive two different signals. A square wave at −1 dBFS: the input of the original ACI is a high-pass network (0.01 µF) that sees little more than the edges, and a square wave with steep edges suits it best. One second of silence at the start and at the end, because a phone sometimes clips the beginning of a sound.

> ▣ **DO NOT TRUST THE LED**
>
> The 1976 manual asks for a level at which the ACI LED is “just on”. Yet it only lights at about 1.2 V, far more than the comparator needs. An unlit LED therefore does not prove that the signal is too weak. If loading never ends, the ROM is still waiting for edges: press RESET and turn the volume up.

> ! **NEVER MP3 OR AAC**
>
> A lossy format shifts the edges and makes the tape unreadable. Use WAV, AIFF, or FLAC if your player decodes it.

## 5.8 What level is needed? The input models

The `info --aci-input` command simulates the analog input of the card (coupling capacitor, bias, LM311 hysteresis) for a series of peak voltages, and tells you from what level the file would be read:

TERMINAL

```
apple1-emu info prog.aiff --aci-input stock     # 1976 card, 0.01 µF
apple1-emu info prog.aiff --aci-input mimeo     # 0.1 µF (Mimeo guide advice)
apple1-emu info prog.aiff --aci-input gen2      # Uncle Bernie's ACI Gen2
apple1-emu info prog.aiff --aci-input stock --volts 0.5 --series 1200
```

| Model | What it represents |
|---|---|
| `stock` | the original ACI: 0.01 µF into the comparator biased at 2.5 V, 3k/47k feedback |
| `mimeo` | Mike Willegal’s modification: 0.1 µF, more reliable |
| `gen2` | Uncle Bernie’s ACI Gen2: matched 100 nF, 300k instead of 47k (less hysteresis) |

`--volts` sets the peak voltage at the jack for digital full scale; `--series` adds a series resistance (for example the 1.2 kΩ of a smartphone cable). With the `stock` model, the emulator’s files are read from **about 0.25 V peak**; a smartphone headphone output at maximum usually gives 0.5 to 1 V. This model is simplified: no noise, no drift of the bias point.

## 5.9 Digitizing a real tape or an Apple-1 recording

To recover a program typed on a real Apple-1, record the ACI’s **TO TAPE** output through a microphone input (a phone, or a PC sound card with Audacity at 44.1 or 48 kHz), with automatic gain control and noise reduction turned off. On the Apple-1: `C100R`, then `0300.03FFW` and RETURN. Stop a few seconds after the `\` comes back, then:

TERMINAL

```
apple1-emu import recording.aiff --addr 300 -o prog.txt
apple1-emu import startrek.aiff --range "004A.00FFR 0300.0FFFR" -o startrek.txt
```

> ▣ **TO TAPE IS A MICROPHONE-LEVEL OUTPUT**
>
> It delivers only about 50 mV peak to peak. Check the waveform in Audacity: no clipping, no level jump during the header. The emulator’s import tolerates any gain and a DC offset.

CHAPTER 6

# The graphics cards: GEN2 and P-LAB

*In 2026, Uncle Bernie designed a color graphics card for the Apple-1, the "GEN2", which brings the Apple II display modes to it. The emulator simulates it as an option: its soft switches, its memory, its video scan and its color screen.*

## 6.1 Plugging in the card

The card is **unplugged by default**. There are three ways to plug it in:

- **GEN2**, in the "Graphics cards" group of the toolbar: the card is plugged in or unplugged at once; the running program carries on (press RESET if it was using the card);
- ⚙ **Preferences → Uncle Bernie GEN2 colour card**, with a choice of 60 or 50 Hz;
- on the command line, `apple1-emu-gui --carte-couleur` (`--couleur-50hz`, `--sans-carte-couleur`).

On the real machine, the card has its own video output for a second monitor, in color; the Apple-1 terminal stays on its original output and works as before. The window therefore shows **two screens**: the Apple-1's, and the **color monitor** below or beside it. A caption shows the current mode. The color monitor has its own display row: zoom, display **Normal** / **Fast** (shared with the Apple-1 screen), its own **Vintage screen** checkbox, full screen and **Detach**, to put it on a second monitor.

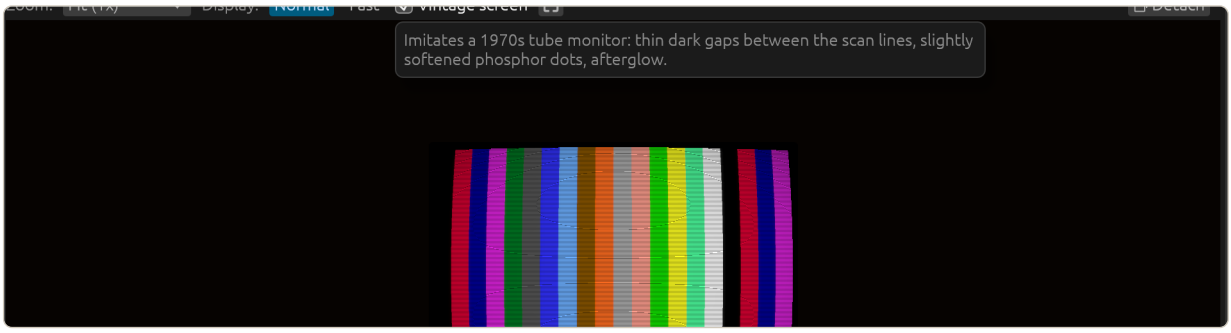



**Figure 18 —** The color monitor: the LORES bars of the gen2-lores.s example, and the mode caption.

> ▣ **THE CARD IS ALSO A MEMORY EXPANSION**
>
> With the GEN2, RAM covers `$0000-$BFFF` (plus `$E000-$EFFF`). The graphics pages live in that memory: a program must not load itself there.

## 6.2 The soft switches

The card reproduces the Apple II soft switches `$C050-$C057`, moved to `$C250-$C257`. They respond **to reads only**: a write has no effect. In assembly language they are therefore triggered with `BIT` or `LDA`, never with `STA`.

| Address | Name (`gen2.inc`) | Effect |
|---|---|---|
| `$C250` | `SS_GR` | graphics mode |
| `$C251` | `SS_TEXT` | text mode |
| `$C252` | `SS_FULL` | full screen |
| `$C253` | `SS_MIX` | mixed: 4 lines of text at the bottom |
| `$C254` | `SS_PAGE1` | page 1 |
| `$C255` | `SS_PAGE2` | page 2 |
| `$C256` | `SS_LORES` | low resolution |
| `$C257` | `SS_HIRES` | high resolution |

Bit 7 of a read returns **HSTo**: 1 during horizontal and vertical retrace, 0 during the visible part. A program can use it to synchronize with the scan (TEST4 does). Bits 0 to 6 are floating. The card also responds on its mirrors (`$C258`, `$C3F1`...) and leaves `$C0xx` and `$C1xx` free for the ACI. RESET does not touch the switches; the emulator starts in text mode, page 1.

In Integer BASIC a read is enough too: `X=PEEK(-15792)` reads `$C250` (−15792 = `$C250` − 65536).

## 6.3 Modes and memory

| Mode | Resolution | Page 1 | Page 2 |
|---|---|---|---|
| TEXT | 40 × 24 characters | `$0400-$07FF` | `$0800-$0BFF` |
| LORES | 40 × 48 blocks, 16 colors | `$0400-$07FF` | `$0800-$0BFF` |
| HIRES | 280 × 192 dots, 6 colors | `$2000-$3FFF` | `$4000-$5FFF` |
| MIXED | graphics, with the last 4 lines in text | | |

**Row addresses.** As on the Apple II, rows are not consecutive in memory. Text row *r* (0 to 23) starts at:

```
$0400 + (r & 7) × $80 + (r >> 3) × $28
```

Each byte of the LORES page holds **two** blocks: the low nibble for the upper block, the high nibble for the lower one.

In HIRES, each scan line *y* (0 to 191) is 40 bytes of 7 dots (bit 0 on the left, bit 7 selects the palette) and starts at:

```
$2000 + (y & 7) × $400 + ((y >> 3) & 7) × $80 + (y >> 6) × $28
```

**LORES colors** (in the order of values 0 to 15):

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| black | magenta | dark blue | purple | dark green | gray | medium blue | light blue |
| **8** | **9** | **10** | **11** | **12** | **13** | **14** | **15** |
| brown | orange | gray | pink | light green | yellow | aquamarine | white |

**HIRES colors** (NTSC artifacts): with bit 7 clear, violet on even columns and green on odd ones; with bit 7 set, blue and orange; two adjacent dots lit give white. So alternating `$55` / `$2A` gives violet, `$2A` / `$55` green, `$D5` / `$AA` blue.

**Text-mode characters**: codes `$00-$3F` inverse, `$40-$7F` flashing, `$80-$FF` normal; `$E0-$FE` are lower case and `$FF` a checkerboard.

## 6.4 The test programs

The `exemples/` folder (and the library, when it is created from scratch) holds a demo and Uncle Bernie's four tests:

| File | Run | Contents |
|---|---|---|
| `gen2-barres.txt` | `C00R` | the 16 LORES colors as bars, plus a line of text in mixed mode |
| `gen2-test1.apl` | `24BR` | TEST1: test of the card's RAM, with HIRES stripes |
| `gen2-test2.apl` | `280R` | TEST2: LORES diagonal, then mixed mode and a "typewriter" (a key to start, ESC to leave) |
| `gen2-test3.apl` | `C00R` | TEST3: the 256 character codes, then the full-screen editor |
| `gen2-test4.apl` | `280R` | TEST4: HST0 measurement, then a 4-line text window scrolling over LORES |

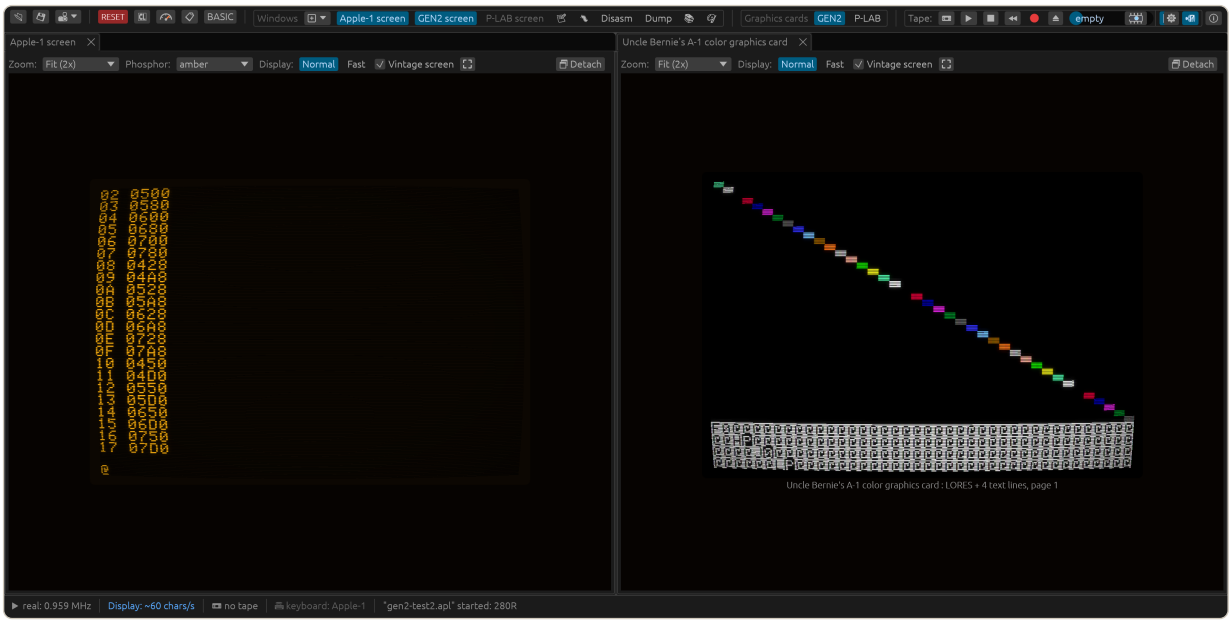



**Figure 19 —** Uncle Bernie's TEST2: the LORES diagonal on the color monitor.

The `gen2-lores.s` and `gen2-hires.s` sources of Part III (chapter 23) show how to write into the graphics pages yourself.

## 6.5 What is approximated

The exact hues depend on NTSC, on the monitor and on the card's trimmer capacitor: the emulator uses values close to the Apple IIgs, in the documented order. The lower-case glyphs are drawn by hand (the card's EPROM has not been published). In HIRES, bit 7 of a byte selects the blue/orange pair instead of violet/green, and the rightmost column of a line stays orange (a test covers it); the half-dot shift that bit 7 also causes on the real card is not modelled. A mode change takes effect from the next scan line. At power-up memory is zero, which gives a screen of inverse `@` until the program fills it. The 2023 GEN1 card (at `$C050`) is not emulated.

## 6.6 The P-LAB graphics card

*The P-LAB "Apple-1 Graphic Card" gives the Apple-1 a real video chip, the Texas Instruments* ***TMS9918A*** *(the one of the ColecoVision, the MSX and the TI-99/4A): 256 × 192 pixels, 16 colors, 32 sprites and a 16 KB video memory. The emulator reproduces it as an option, next to GEN2.*

### Plugging in the card

The card is **unplugged by default**. Click **P-LAB** in the "Graphics cards" group of the toolbar (it lights up), or choose **Plug the P-LAB card** in the **Colour screen** button when no card is plugged in, or start with `apple1-emu-gui --plab` (`--sans-plab` to remove it). The **P-LAB screen** appears as a panel (the **Colour screen** button of the Windows group shows or hides it); like every panel it can be docked, tabbed or detached into its own window.

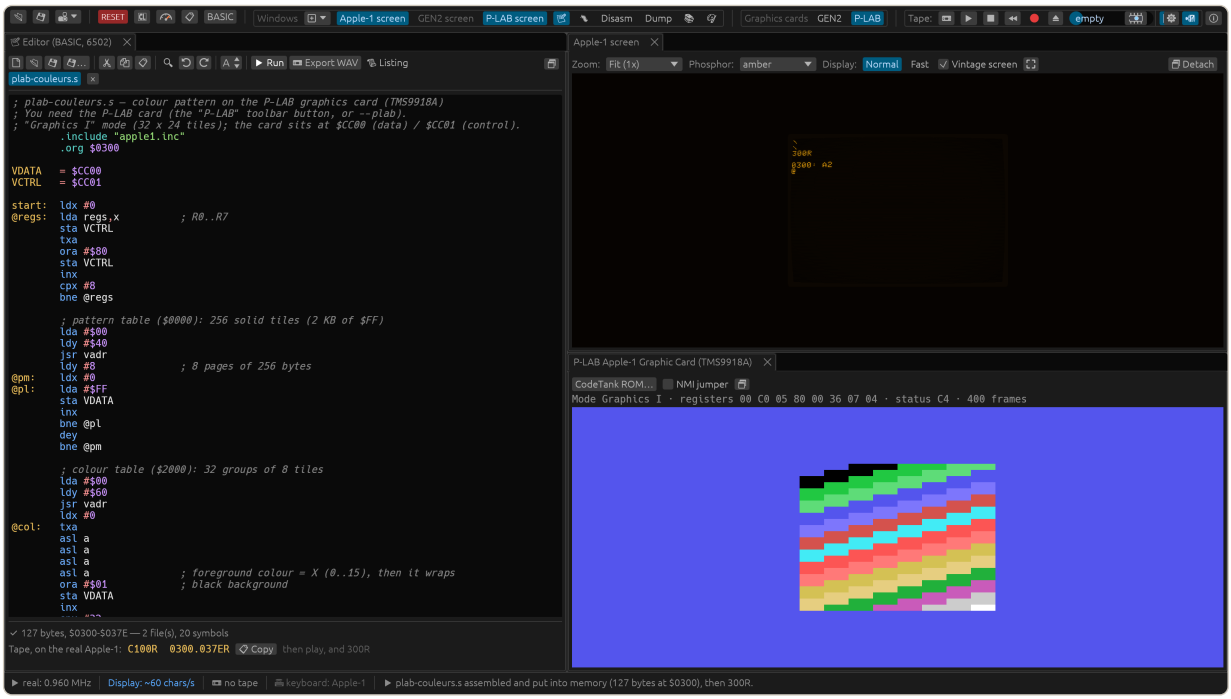



**Figure 20 —** The P-LAB screen below the Apple-1 terminal, drawing the colour pattern of the plab-couleurs.s example. The line under the toolbar of the panel shows the display mode, the eight registers and the status.

The Apple-1 terminal keeps working as usual. The card takes the addresses `$CC00-$CFFF`: **`$CC00` is the data port** (read or write the video memory) and **`$CC01` the control port** (video address and registers on write, status on read). The emulator decodes only address line A0 inside the `$CC00-$CFFF` window, so the pair repeats over the whole 1 KB; the card's manual does not specify this, it is an assumption (see the end of this section).

The panel has two controls and one information line:

- **CodeTank ROM…** plugs a 16 KB image into the CodeTank daughterboard: it then reads at `$4000-$7FFF`, read-only, over the RAM. Type `4000R` on the terminal to start it. **Remove ROM** takes it out;
- **NMI jumper** is the IRQ/NMI jumper of the card. Unchecked (the normal setting), the end-of-frame interrupt of the TMS9918A (60 times a second) reaches the 6502 as IRQ; checked, as NMI. The interrupt only happens when the program enables it (bit 5 of register 1);
- the **information line** shows the mode (Graphics I, Graphics II, Text, Multicolor), registers R0 to R7 in hexadecimal, the status register and the number of frames drawn.

### Programming it

The TMS9918A is reached through two ports only. To write the video memory, send the address to the control port (low byte, then high byte **or** `$40`), then write the bytes to the data port: the address increments by itself. To write a register, send the value, then the register number **or** `$80`, to the control port. Reading the status register (bit 7 = frame flag, bit 6 = fifth sprite, bit 5 = collision) clears it.

The `plab-couleurs.s` example (in the library, folder **6502**; also in `docs/exemples/en/asm/`) sets Graphics I mode, fills the pattern, color and name tables and leaves a colour pattern on the card's screen. Run it from the editor (▶ **Run**, then `300R`, or assemble and run in one click):

plab-couleurs.s (excerpt) 6502

```
VDATA   = $CC00
VCTRL   = $CC01

start:  ldx #0
@regs:  lda regs,x          ; R0..R7
        sta VCTRL           ; value
        txa
        ora #$80
        sta VCTRL           ; register number + $80
        inx
        cpx #8
        bne @regs
```

### Loading the public demos

The demos of the `nippur72/apple1-videocard-lib` project (written in KickC) come in four forms; the emulator knows all of them:

| File | What it is | How to load it |
|---|---|---|
| `*.bin` | raw program that starts at `$0280` | **Load** (the address dialog proposes `280`), then `280R` on the terminal |
| `*.woz` | the same, as text for the Woz Monitor | **Load**, then `280R` |
| `*.prg` | a 2-byte start address (little-endian), then the code | **Load**: the address is read from the file |
| `*_jukebox.bin` | 16 KB EPROM image for the CodeTank | **CodeTank ROM...**, then `4000R` |

Those demos are not distributed with this kit (their author has not published a licence); download them from the project's page and keep them in your library. The emulator was checked against four of them (the demo menu with its screens and the bouncing balloon, the Amiga hand, the photo viewer and Tetris), in the RAM and in the jukebox forms.

### What is emulated, and what is not

▣ **WHAT THE EMULATOR DOES**

The four display modes (Graphics I and II, Text, Multicolor); the 16 KB video memory; the 32 sprites, 8 × 8 or 16 × 16, magnified or not, with the five-sprites-per-line limit and the collision flag; the frame flag and the end-of-frame interrupt, IRQ or NMI; the CodeTank at `$4000-$7FFF`. The TMS9918A palette is the usual one for NTSC.

! **WHAT IS NOT EMULATED**

A register changed in the middle of a frame takes effect at the next frame (raster effects are not reproduced); the 4 KB DRAM mode; the exact timing of sprites and of video-memory accesses; the external video input; the composite-video artefacts. The state of the video chip is **not** saved in the machine states.

**WHERE THE DOCUMENTATION IS SILENT**

The card's manual (revision 0.4) does not say how the window is mirrored, what the bus timing is, what the video memory contains at power-up, whether the CodeTank is write-protected, or exactly how the IRQ/NMI jumper is wired. The emulator makes reasonable choices on those points (as described above); if you own the card and see a difference, please tell us.

CHAPTER 7

# The editor

*The built-in editor writes Integer BASIC programs and 6502 sources, checks them at every keystroke, runs them in the emulated Apple-1 in one click, and exports them as tapes for the real machine.*

## 7.1 Tabs and files

📝 **Editor** opens the panel, docked to the left of the screen. Its toolbar is compact, to leave as many lines as possible to the text. It starts with icons, explained by their tooltips:

| Icon | Action |
|---|---|
| **New** (first icon) | New tab: “BASIC program” or “6502 assembler source” |
| 📂 / 💾 / 💾… | Open ( `.bas` , `.txt` , `.s` , `.asm` , `.a65` , `.inc` ) / Save (Ctrl+S) / Save as… |
| ✂, copy, 📋 | Cut, copy, paste |
| 🔍 | Find and replace (Ctrl+F, Ctrl+H) |
| ⟲ ⟳ | Undo (Ctrl+Z), redo (Ctrl+Shift+Z) |
| A▲▼ | Text size: one compact control, the letter A with two small stacked triangles; ▲ enlarges, ▼ reduces |

After them, on the same row (or the next one when the panel is narrow), come the actions of the tab: ▶ **Run**, 📼 **Export WAV**, and **Memory** for BASIC, or 📃 **Listing**, • **Breakpoint** and ⏩ **To cursor** for a 6502 source.

Each file is a tab. A dot • marks a modified tab; × closes it (a second click is required if it is not saved). A tab marked 📚 was opened from the library: opening a listing or a source of the library (▶, ⬇ or a double-click) opens its tab when the editor is visible, or shows it if it is already open. Tabs, unsaved text included, come back at the next launch. A right-click on a tab offers **Save as…** and **Close**. In the text, a **middle click** pastes the primary selection (Linux): the cursor goes where you click, then the text last highlighted in any application is inserted.

The text size is set with the **A▲▼** control, Ctrl+wheel, or Ctrl++ / Ctrl+− / Ctrl+0 when the text has the keyboard focus.

**Detach** puts the editor in its own window, for example on a second screen; ⊟ **Dock** puts it back. Keys typed in that window go to the editor, never to the Apple-1.

## 7.2 Find, replace and the right-click menu

🔍 (or Ctrl+F) opens a search bar above the text: Enter goes to the next match, Shift+Enter to the previous one, **Aa** matches the case, **Word** finds whole words, and the counter shows “3 of 7”. Ctrl+H adds the **Replace with** field, with **Replace** (the current match) and **Replace all**. Esc closes the bar.

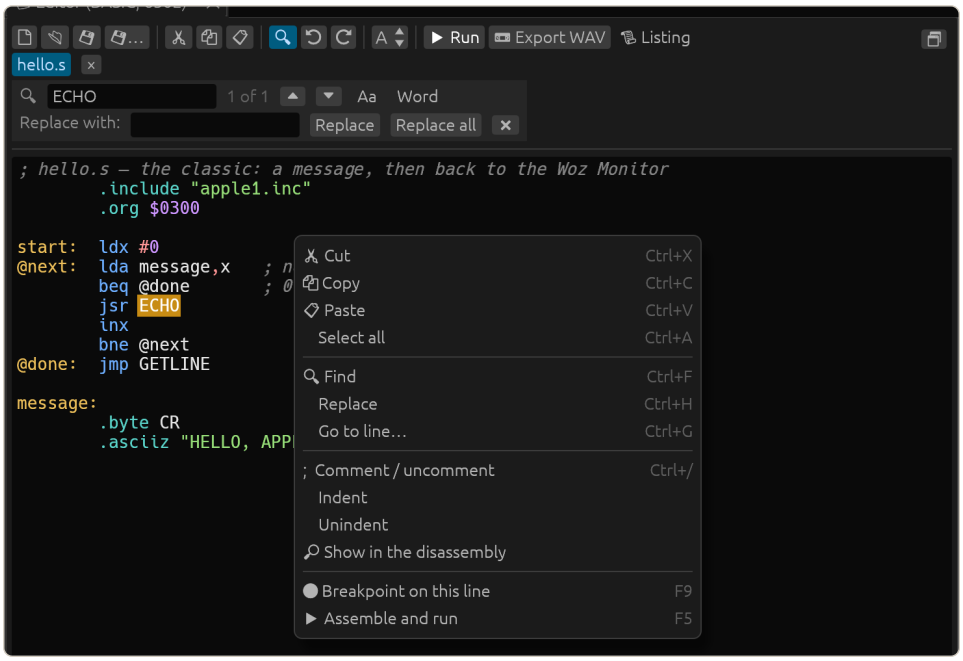



**Figure 21 —** The search bar open on ECHO, and the right-click menu of a 6502 source.

**Right-clicking** the text opens a menu with every command and its shortcut. The menu acts on the **selection**, which is kept when you right-click on it: **Cut** and **Copy** work on the selected text. The word-based commands (go to the definition, help) use the **word under the click**, not the one under the cursor.

| Command | Shortcut |
|---|---|
| Cut, Copy, Paste, Select all | Ctrl + X, C, V, A |
| Find, Replace | Ctrl + F, Ctrl + H |
| Go to line… | Ctrl + G |
| Comment / uncomment (6502), REM / un-REM (BASIC) | Ctrl + / |
| Indent, Unindent | |
| Go to the definition of a label (6502) or of a line number (BASIC) | F12 |
| Help on *word* (on a keyword, an instruction or a directive: opens its entry in the reference) | |
| Show in the disassembly (6502) | |
| Renumber… (BASIC: `GOTO`, `GOSUB` and `THEN` follow) | |
| Breakpoint on this line | F9 |
| Assemble and run (6502), Send and run (BASIC) | F5 |

**Colors.** The colors of the text follow the interface theme. ⚙ **Preferences → Editor** offers, for BASIC and for the 6502 assembler separately, the schemes *Automatic*, *Classic* (*dark*), *Light*, *Amber*, *High contrast*, or your own colors, one swatch per kind of word (chapter 11).

## 7.3 A BASIC program

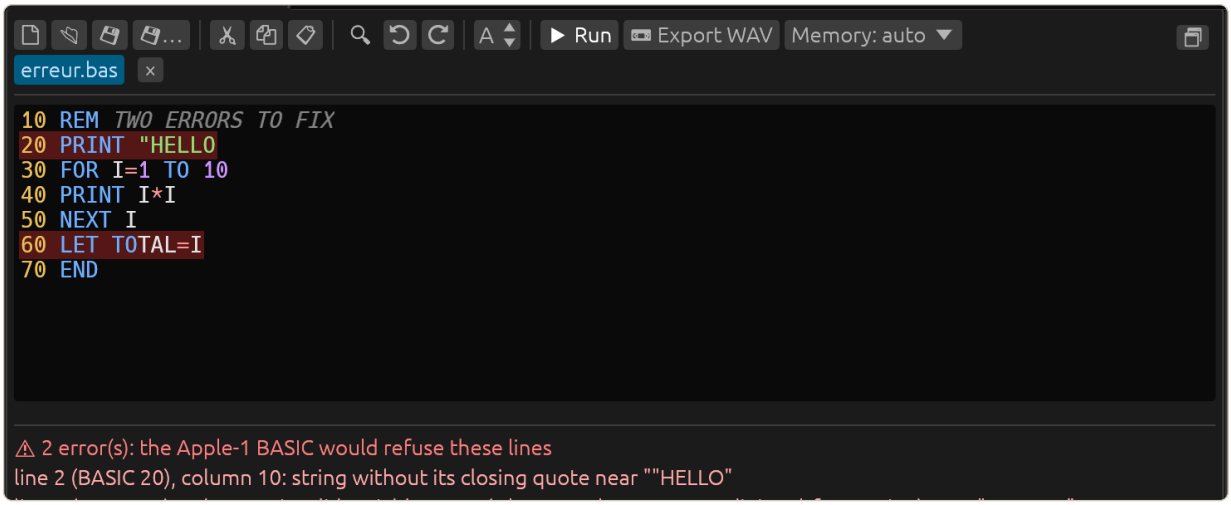



**Figure 22 —** The BASIC editor: errors shown on a red background, with their line number and column at the bottom of the panel.

The text is colored: line numbers, keywords, numbers, strings, `REM`, operators. Each line is **tokenized** as Integer BASIC would do it, by a native tokenizer checked byte for byte against the real one. A line that BASIC would refuse turns red, and the error gives the text line, the BASIC line number

and the column; a click puts the cursor there. Errors detected: syntax, invalid variable name, number greater than 32767, direct-mode command ( `LIST` , `RUN` ...) inside the program, unterminated string, line longer than 255 bytes, memory full.

- **Memory**: automatic (8 KB with LOMEM `$0800` , or `$0300` if the program is large, otherwise 12, 16 or 32 KB), or forced.
- ▶ Run: loads Integer BASIC if it is missing (like the BASIC button), puts the tokenized program into memory, adds RAM if needed, then types `E2B3R` and `RUN` . It is instant: nothing is typed line by line.
- 📼 Export WAV: a tape ( `.wav` , or `.aiff` for a smartphone) of two records, with its settings in the library. The bottom of the panel shows what to type on the real Apple-1, for example `C100R` then `004A.00FFR 0C3B.0FFFR` , then `E2B3R` and `RUN` ; 📋 **Copy** puts it on the clipboard.

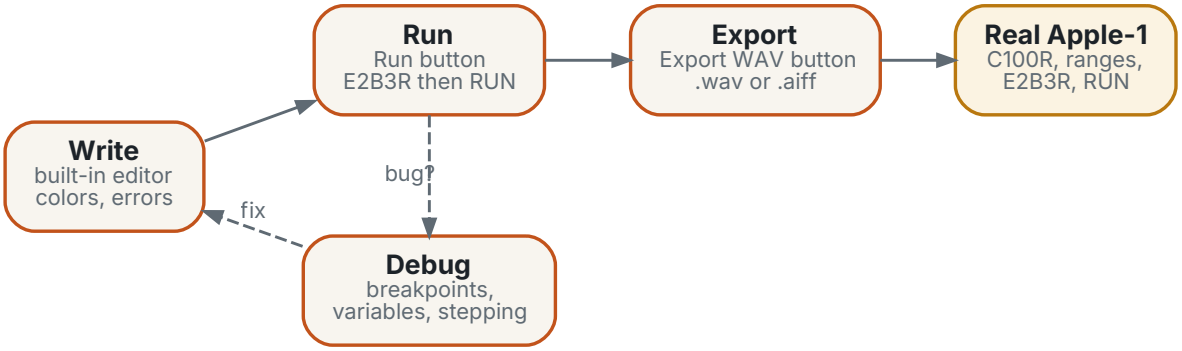



**Figure 23 —** From the editor to the real Apple-1: write, run, fix, export.

## 7.4 A 6502 source

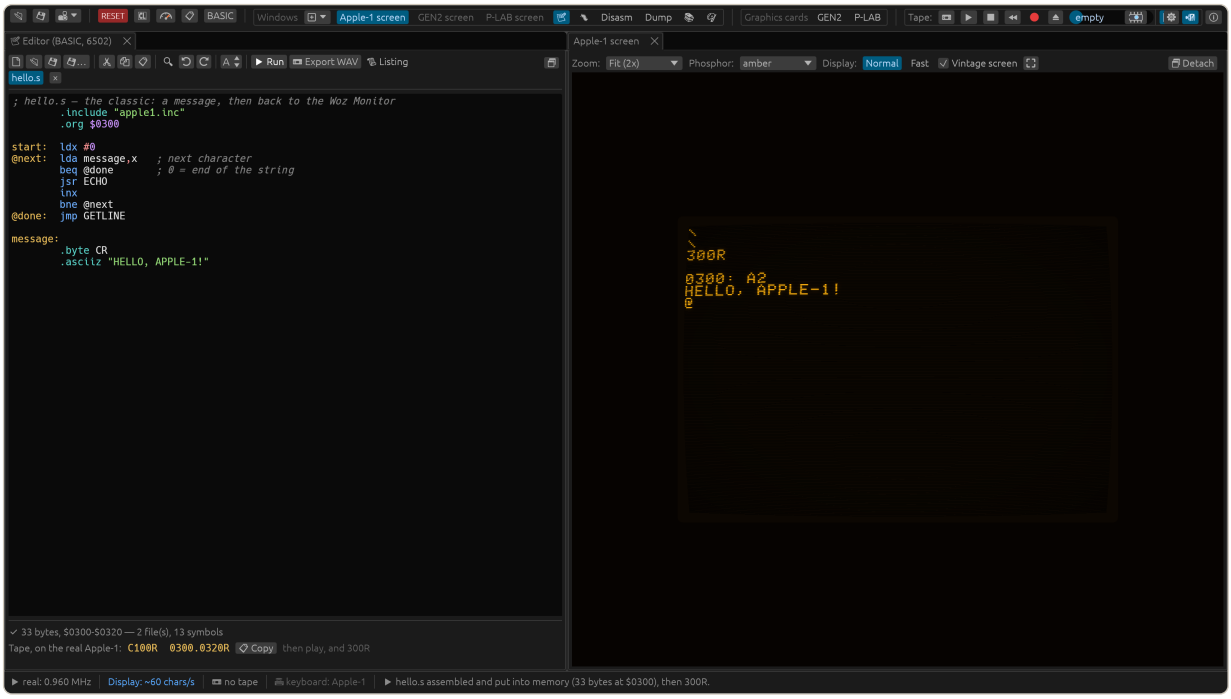



**Figure 24 —** A 6502 source in the editor: the colors, the assembly result and the line to type on the real Apple-1.

A source is assembled **at every change** by the built-in two-pass assembler, which accepts ca65 syntax (and xa syntax, translated automatically). The bottom of the panel shows the size and the addresses (`33 bytes, $0300-$0320 — 2 file(s), 13 symbols`), then the line for the real machine (`C100R` then `0300.0320R`, then `300R`).

- **Errors**: file, line and column; a click takes you there, opening the included file if necessary. Warnings (zero page encoded as absolute) are in yellow.
- 📃 **Listing**: address, bytes and source of each line, with included files and expanded macros, then the symbol table; a click takes you back to the line.
- ▶ **Run**: puts the bytes into memory at the origin (`.org`, `$0300` by default), adds RAM if needed, then types for example `300R`.
- 📼 **Export WAV**: the tape, as for BASIC.
- ● **Breakpoint** and ⏩ **To cursor**: the link with the debugger (chapter 8).
- **Link with the debugger**: when you select an instruction in the source (or place the cursor on its line), the bytes it produced are highlighted in the debugger's **disassembly** and in the **hex / ASCII dump**, when those panels are visible; if they are elsewhere in memory, the panels scroll to show them. The assembler's table of source lines and addresses makes this possible.

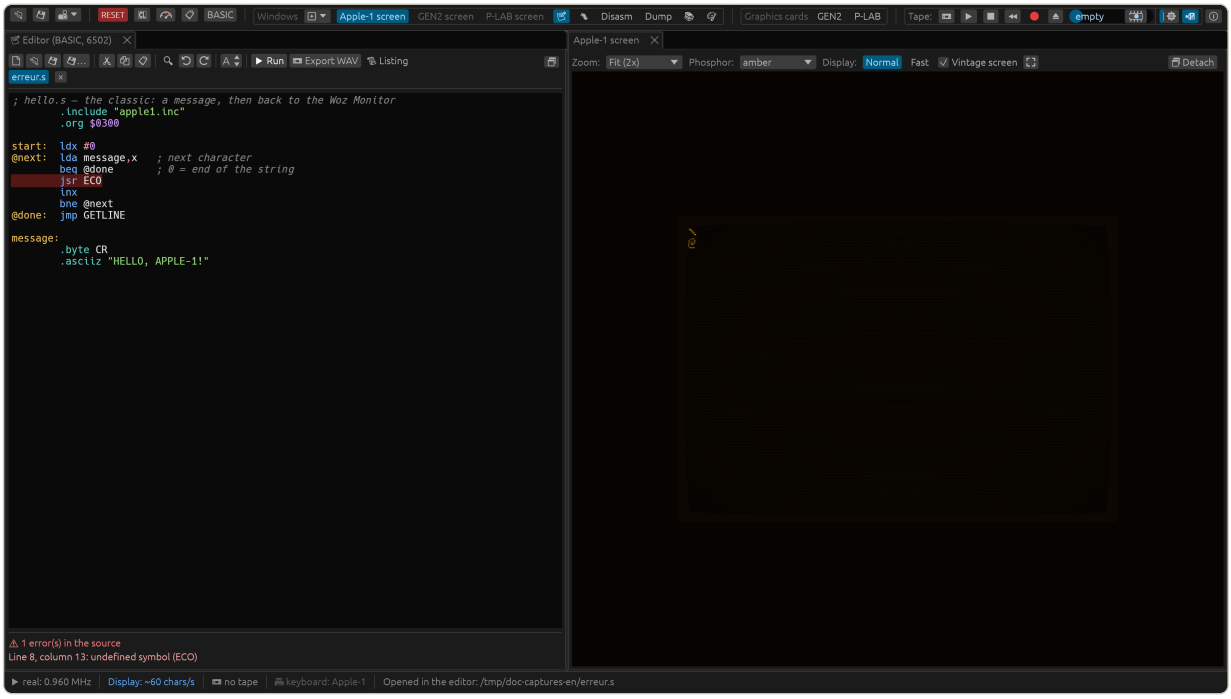



**Figure 25 —** An assembly error: ECO instead of ECHO, an unknown symbol, reported with its line and column.

Part III is entirely devoted to the assembler: syntax, directives, listing, examples.

CHAPTER 8

# The debugger, the disassembly and the dump

*The debugger stops the Apple-1 and steps through it, one instruction at a time in 6502 or one line at a time in BASIC, showing the registers, the variables and memory.*

## 8.1 The debugger panel

🐛 **Debugger** opens the panel, docked on the right or detachable like the editor. Its commands:

| Command | Effect |
|---|---|
| ⏸ **Pause** / ▶ **Continue** | stops or resumes the Apple-1 |
| ⬇ **Step** | executes one instruction |
| ↪ **Over** | executes a `JSR` in one go, and stops on return |
| ⤴ **Out** | runs until the `RTS` of the current subroutine |
| ⏭ **BASIC line** | advances by one line of the BASIC program |
| ⏩ **To cursor** | (in the editor) runs to the cursor line |

The **Processor** frame shows A, X, Y, S, PC and the flags N V B D I Z C; all of them can be changed by clicking the value. The number of cycles and instructions executed is shown below.

## 8.2 Breakpoints

A breakpoint can be set in four ways:

- ● **Breakpoint** in the editor (or `F9`, or the right-click menu), on the cursor line: in a `.s` source (thanks to the assembler's address table) or in a BASIC program (line number);
- a click in the margin of the 📃 **Listing**;
- a click on a line of the disassembly;
- typing an address or a label: `$0368`, `ASK`, `PUTS+3`, or `L190` for BASIC line 190. A label that does not exist yet is not refused: the breakpoint is kept and is set as soon as the label exists (when you assemble the editor's source, or run it), with a message that says so.

Breakpoints, watched variables and memory watches are remembered from one session to the next, but loading another program clears the breakpoints and labels of the previous one. When the built-in BASIC is loaded the BASIC section of the debugger is titled **RDK G2 Basic**; without any BA-

SIC at `$E000` it says so neutrally. The colours of the current line, the dump addresses and the ASCII column can be changed in ⚙ **Preferences** → **Interface** (“Interface colours”). Narrow panels drop the bytes column of the disassembly and wrap the poke row instead of cutting them.

## 8.3 Debugging a BASIC program

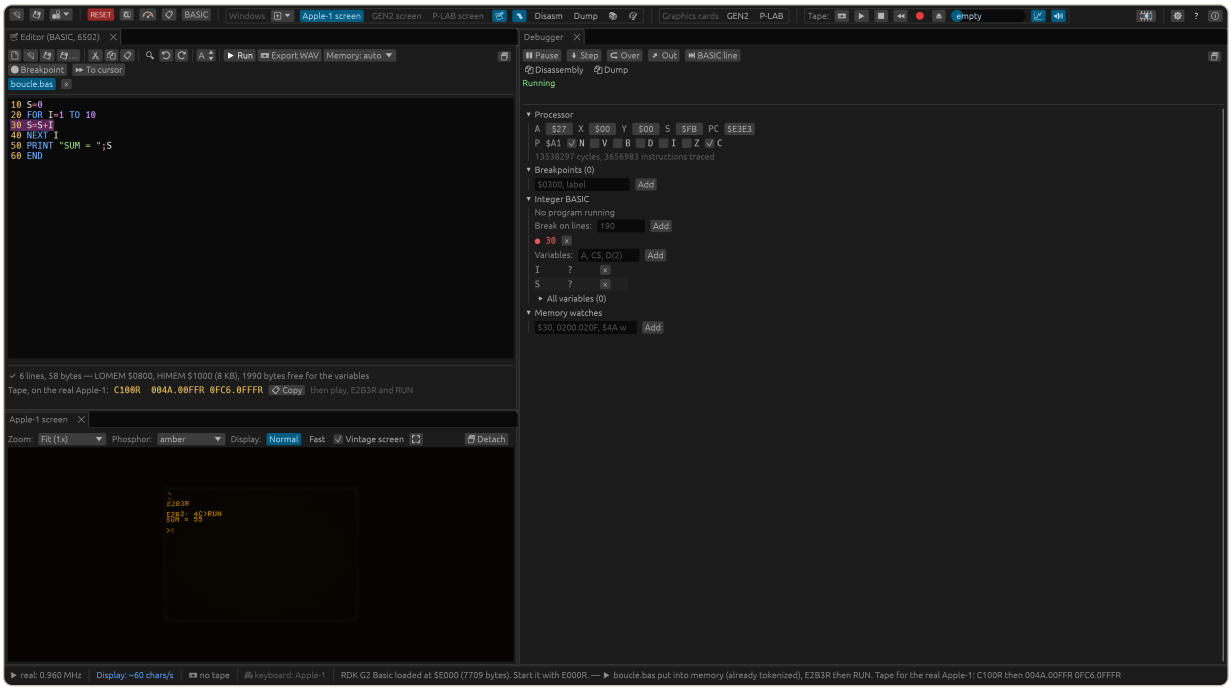



**Figure 26 —** boucle.bas stopped at a breakpoint on line 30: the current line in green in the editor, the variables I and S watched in the debugger.

The current line is highlighted in green in the editor, breakpoints in purple. The BASIC frame (**RDK G2 Basic** or **Integer BASIC**, depending on the one loaded) shows the current line, the breakpoints on line numbers and the **watched variables**: integers ( `A` , `B1` ), array elements ( `D(2)` ), whole arrays ( `D` ), strings ( `C$` ), plus **All variables**.

Debugging works with **RDK G2 Basic**, the BASIC built into the emulator, as well as with the original: the real Integer BASIC is no longer needed. The debugger notices the start of every line (the current line, the line breakpoints, ⏭ **BASIC line**) and reads the variables from the interpreter’s tables. For the original it reads the line from the zero-page pointers `$DC` / `$DD` ; G2 Basic keeps it elsewhere, and the debugger knows both layouts.

> **! A KEY PRESS STOPS THE PROGRAM**
>
> As on the real machine, a key pressed while the BASIC program runs stops it ( `STOPPED AT` ). During a debugging session, keep the keyboard focus in the debugger panel, not on the Apple-1 screen. A G2 Basic program can read the keyboard without being stopped with `KEY(0)` (chapter 15).

## 8.4 Debugging a 6502 program

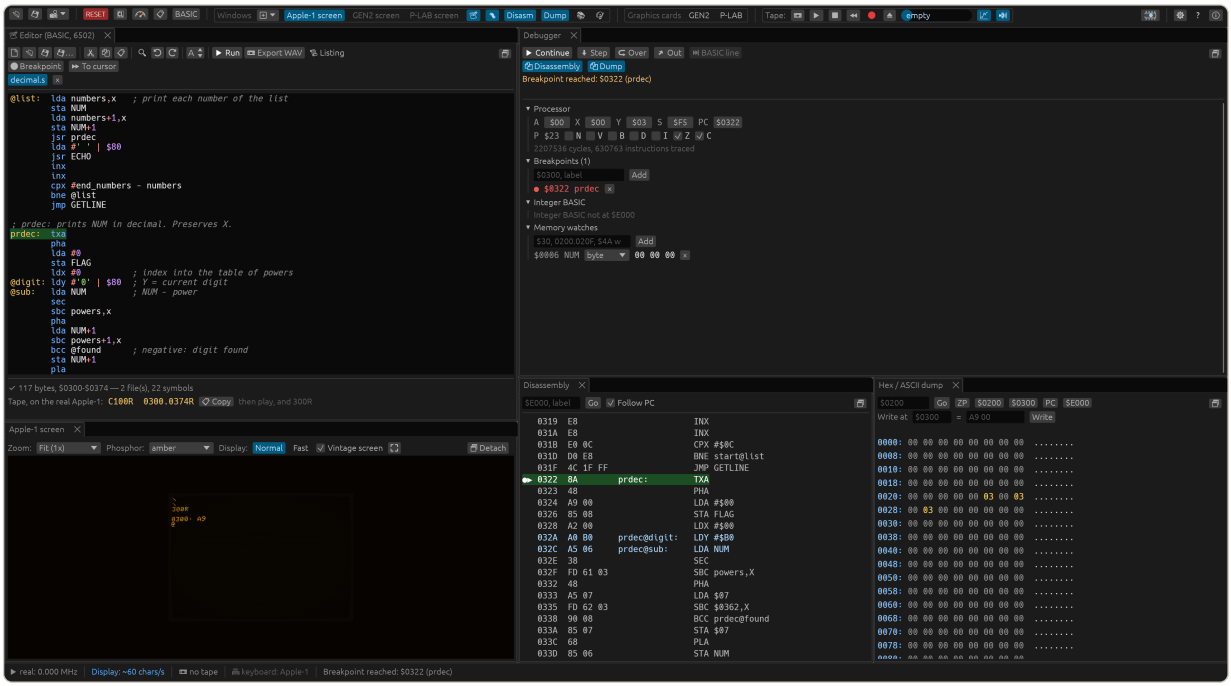


**Figure 27 —** decimal.s stopped at the label prdec: registers, disassembly with the program's labels, memory watch $06.$08.

The **disassembly** shows 24 lines around PC, with the labels of the assembled program in their own column. It only re-centers when PC leaves the visible area, so the display does not jump at every step. **Go** pins the list on an address or a label; **Follow PC** makes it follow execution again.

When you select an instruction in the editor's 6502 source, its bytes are also **highlighted** in the disassembly and in the dump (chapter 7), and both panels scroll to them if they are out of view; the disassembly then stops following PC until you press **Follow PC**.

The **memory watch** shows a byte, a word or ASCII ( `$30` , `$30 w` , `0200.020F` , a label), editable, in yellow when the value changes.

## 8.5 The dump

The **Dump** shows memory in hexadecimal and ASCII, 16 bytes per line (8 if the panel is narrow), with bit 7 stripped for the ASCII. **Go** jumps to an address, or to the shortcuts ZP, `$0200` , `$0300` , PC, `$E000` . Changed bytes are in yellow, the byte at PC on a green background. **Write** stores bytes ( `0300 = A9 00` ); clicking a byte fills in the write line.

The disassembly and the dump are panels in their own right: detached by default, they can be docked next to the debugger (chapter 10).

## 8.6 On the command line

For demonstrations and automated tests: `--debogueur` opens the panel, `--dbg-arret ASK,L190` sets breakpoints, `--dbg-surveiller 'A,C$,$30 w'` watches variables and memory, and `--dbg-commande 5:pas` sends a command after 5 seconds (`pause`, `continuer`, `pas`, `dessus`, `sortir`, `basic`). These option names and keywords are French.

CHAPTER 9

# Photo → ASCII

*The Photo → ASCII tool turns a photo into a 40 × 24-character Apple-1 picture, then into a BASIC or 6502 program, ready to run or to export to tape.*

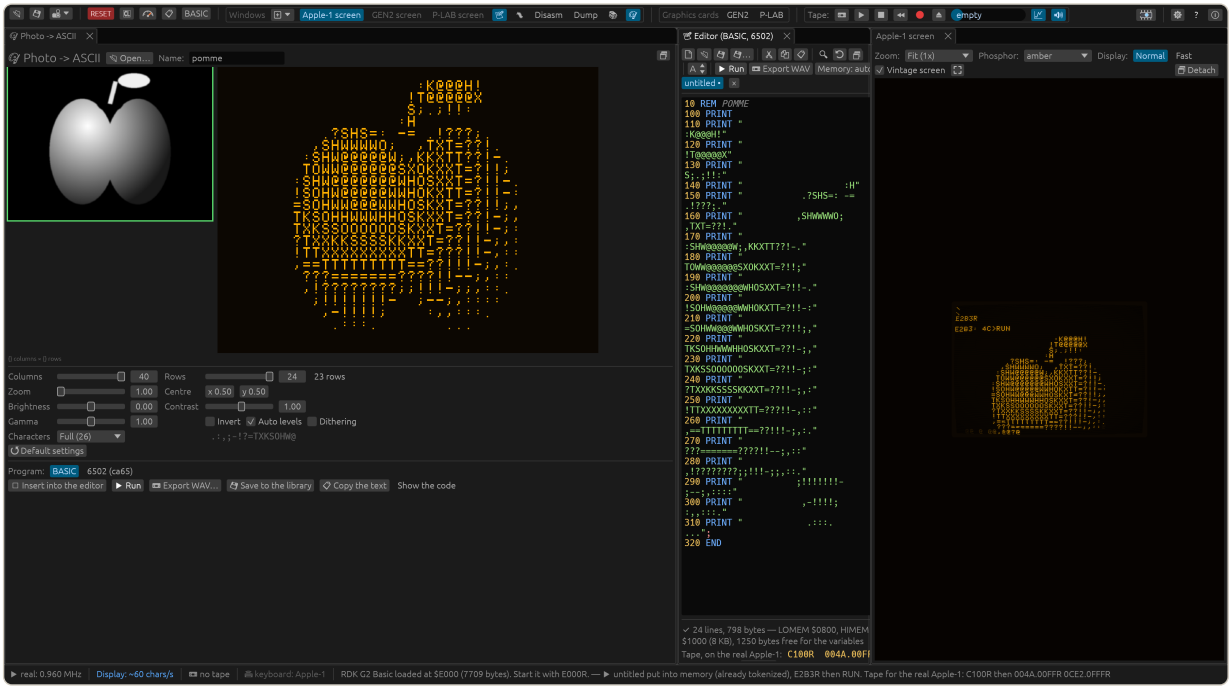


**Figure 28 —** The Photo → ASCII tool: the photo and its crop frame, the settings, and the preview drawn with the 2513 font.

## 9.1 Opening a picture

🎨 **ASCII** opens the tool, a dockable panel like the others. **Drag and drop** a `.jpg` or `.png` photo onto the window, or use 📂 **Open…** (a photo, or a finished ASCII art in `.txt`).

The green frame on the photo shows the part being converted: drag it to move it, the mouse wheel zooms.

## 9.2 Settings

| Setting | Role |
|---|---|
| **Columns**, **Rows** | 40 × 24 at most. **23 rows** leaves the last line to the prompt, so that the picture does not scroll. |
| **Zoom**, **Centre** | the part of the photo kept |
| **Brightness**, **Contrast**, **Gamma** | the usual settings (gamma > 1 lightens) |
| **Invert** | dark = a lot of ink |
| **Auto levels** | stretches the gray levels (2 % to 98 %) before the other settings |
| **Dithering** | Floyd-Steinberg: more shades, more noise |
| **Characters** | **Full** ramp (26 levels), **Simple** (10), the one from **Schmenk's slideshow**, or **Custom** |

Only the characters of the 2513 are possible: upper case, digits and symbols, with no lower case, no `"` and no `_`. They are ranked by the number of dots lit in the Apple-1 font: that number, not the shape, gives the gray level.

> ✦ FOR A PORTRAIT
>
> 40 × 24 characters is not much: crop on the face (zoom 1.3 to 1.6), raise the contrast a little, and try a gamma below 1 for a bright photo. A plain dark background gives the best results.

## 9.3 The program produced

- **BASIC**: one `PRINT "…"` line per line of the art. A full 40-character line ends with `;`, because the screen wraps by itself. The program is slow to type in (about 2 s per line).
- **6502**: a ca65 source at `$0300`, started with `300R`. After clearing the screen, a loop sends a compressed table to ECHO (runs of identical characters encoded as "count + character"). The display is instant.

With 24 rows, the program waits for a key before returning; with 23, the prompt lands on the last line and the picture stays whole.

The buttons: ↰ **Insert into the editor** (new BASIC or 6502 tab), ▶ **Run**, 📼 **Export WAV…**, 💾 **Save to the library** (a `.txt` in the "Art ASCII" folder), 📋 **Copy the text**, **Show the code**.

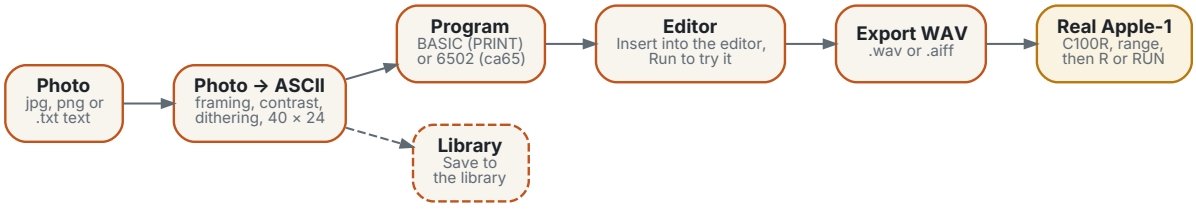



**Figure 29 —** From the photo to the real Apple-1: conversion, program, tape.

## 9.4 On the command line

TERMINAL

```
apple1-emu ascii photo.jpg                      # the art alone, on screen
apple1-emu ascii woz.jpg --zoom 1.3 --cx 0.45 --cy 0.32 \
          --contrast 1.3 --gamma 0.45 --rows 23 -o woz.txt
apple1-emu ascii woz.jpg -o woz.bas             # BASIC program
apple1-emu ascii woz.jpg -o woz.s               # 6502 source
apple1-emu ascii woz.jpg -o woz.wav             # 6502 tape, started with 300R
apple1-emu ascii woz.jpg -o woz.wav --format basic
```

Chapter 12 gives all the options.

▣ **DAVE SCHMENK'S SLIDESHOW**

In 2014, Dave Schmenk wrote a portrait slideshow for the Apple-1. The library adds it as a GitHub project ( `dschmenk/Apple1-Slideshow` ): it is downloaded, not supplied, because the repository has no license. ▶ **Test the latest build** assembles it (xa syntax, translated automatically; the same 3456 bytes as xa 2.4.1) and runs it with `280R` .

CHAPTER 10

# Panel layout

*Each panel is a tab that you place wherever you like: side by side, in tabs, or in its own window on a second screen.*

## 10.1 Moving a panel

The editor, the Apple-1 screen, the GEN2 color monitor, the debugger, the disassembly, the dump, the ASCII tool and the library are all **dockable tabs**. Drag a tab by its title: guides appear, and you drop it **left of**, **right of**, **above** or **below** another panel, or **into the same tab group**. Drag the dividers to resize. The layout is remembered in `gui.json`.

## 10.2 The initial layout

The editor is on the left; in the middle, the Apple-1 screen, with the color monitor below it when the GEN2 card is plugged in; on the right, the debugger with the disassembly below it, then the dump, and the library at the far right. At the very first launch the choice is made from the width of the **monitor** (the window opens small and is maximized a moment later): below 1400 points (a 1366 × 768 laptop, a 1024 × 768 screen) it starts as **editor and screen**, which is the most useful pair; from 1400 points up, with every panel. Choose another layout from the menu at any time. It follows the width of the window: each panel keeps its share, from a laptop to a wide monitor. The **⊞ Layout ▾** menu of the toolbar restores it at any time (**Default**, every panel open) and offers four more ready-made layouts: **Game** (Apple-1 screen and library), **BASIC** (editor and screen), **Assembly** (editor, screen and debugger) and **Debugging** (editor, debugger, disassembly and dump). **Save the current layout…** asks for a name and keeps the current layout, with its detached windows and the size of the main window (loading it opens those windows again where they were); your layouts are listed under a separator, in alphabetical order, and a right-click lets you **Rename** or **Delete** them. They are kept in `gui.json`.

## 10.3 The start-up guide

On first launch a three-screen guide replaces the old “Your workspace” window. It is drawn in amber on black, with the tube effect (scanlines, glow, vignette, curvature 0.07) on the frame; the small text and the thumbnails stay sharp. A small **×** at the top right closes it. **‹ Back** and **Next ›** move between screens; the last screen has **Done**.

1. **Welcome**: what the kit is for.
2. **ROM and BASIC**: the Woz Monitor, the cassette ROM and G2 Basic are built in. Nothing else is required. Purists install their own original Integer BASIC from an outside source; the button opens the guide (“Installing a ROM”), which has links only, never the files.

3. **Your workspace**: the five layouts (**Default**, **Game**, **BASIC**, **Assembly**, **Debugging**). Each thumbnail is clickable (there is no “Choose” button). The rectangles keep numbers 1 to 6, and the colour screen (GEN2 / P-LAB) is **7**, in its own colour, in the thumbnails that show it and in the legend.

**Show this guide at start-up** is on all three screens, ticked by default. The same box is in ⚙ **Preferences → Interface**. ⊞ **Layout ▾ → Start-up guide...** opens it again. It is an ordinary window: it does not stay above the others. It does not open when the command line sets a size ( `--taille` ), nor for captures.

## 10.4 Text size of each window

The editor, the assembler listing, the debugger, the disassembly, the dump and the library each have a compact **A 13 ▲▼** control in their bar: the triangles change the size, which is remembered for that window. Ctrl+wheel and Ctrl+plus / Ctrl+minus act on the window under the mouse pointer (over the Apple-1 screen, Ctrl+plus / Ctrl+minus change the zoom). **Reset preferences** puts every one of these sizes back to 13.

## 10.5 What is remembered between two launches

The layout is saved on its own when you quit and found again as it was: the position and size of every panel, the detached windows, the zoom and the layout you chose. The window is restored to its size and its place, then the panels are put back in the same proportions, whatever the screen: the layout was checked at 2880 × 1620, 2560 × 1440, 1920 × 1080 and 1366 × 768, with less than 1 % of the picture different after a relaunch. A layout saved under a name (**Save the current layout...**) keeps in addition the detached windows. The interface never zooms itself: only the setting “Icon and text size” changes the size of the buttons.

## 10.6 When the screen is bigger than its pane

With a **fixed zoom** (2x, 3x..., or Ctrl + +), the Apple-1 screen keeps the size you asked for, even if the pane is smaller: it then **scrolls** (wheel or bars) instead of being shrunk, and the status bar says so, with a hint to enlarge the pane. Only **Fit** adapts the zoom to the pane. In a narrow window (1024 × 768, for instance) the toolbar scrolls sideways rather than being cut off: the **»** button brings the Cassette group, and a second click goes back to the start. The editor’s **Run** button always stays visible.

## 10.7 Detaching a panel

Each panel has a **Detach** button at the top right (also from a right-click on the tab) that opens it in **its own window**, which you can move to a second screen; its size and place are remembered. ⊟ **Dock** puts it back in its initial place.

- The Apple-1 screen and the color monitor detach separately. The keyboard goes to the Apple-1 from either of their windows, and from the main window. Closing a screen’s window docks it again.

- Closing a tool's window (editor, debugger…) closes the tool; its toolbar button reopens it in the same place.
- Dialog boxes (Open, Save as, Export) always open in the main window. In the file dialogs, Esc closes the dialog (also from the file name field), Ctrl+L edits the path (its text is selected, so typing replaces it); Save and Export of a tab start in the folder of its file.
- Saved layouts keep the detached windows; the main window keeps the same size from one launch to the next whatever the zoom.
- The tape oscilloscope stays a small floating window inside the main window.

### To play

Close the editor and the debugger, keep the library. **Fit** enlarges the screen; F11 goes full screen, with the vintage screen.

### To program

Editor on the left, screen in the middle, debugger and dump on the right. Detach the editor to a second monitor to give it the full height.

### For the GEN2

Plug in the card: the color monitor goes below the Apple-1 screen. Detach it to get it large, like a second monitor.

### To give a talk

**Detach** the Apple-1 screen onto the projector, full screen, and the main window on your laptop.

CHAPTER 11

# Preferences

⚙ *Preferences gathers all the persistent settings. Everything is kept in ~/.config/apple1-emu/gui.json (or in the kit folder, in portable mode).*

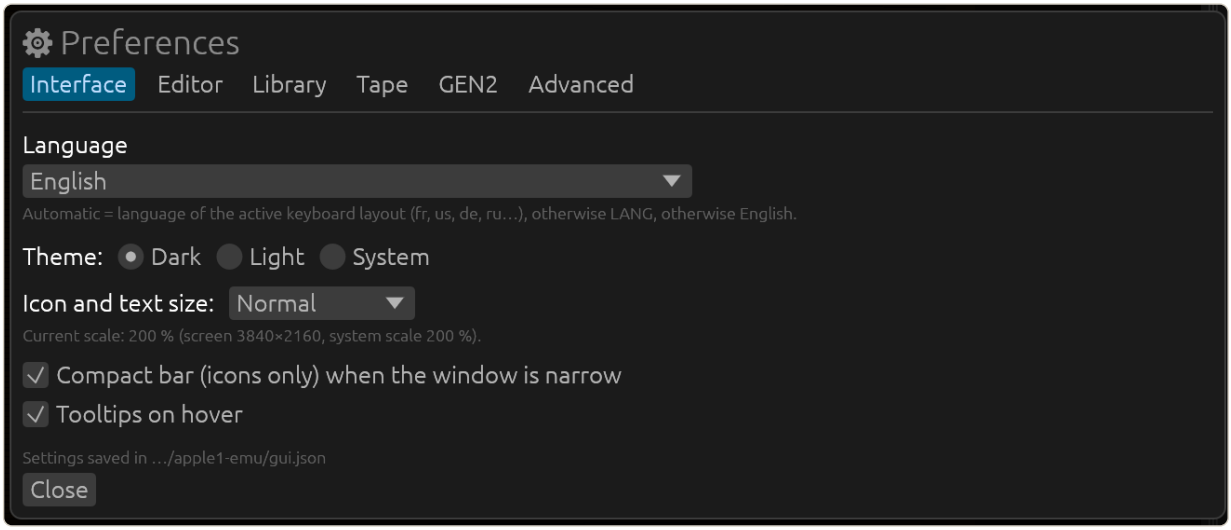



**Figure 30 —** The Interface tab of the preferences: language, theme, icon and text size, compact bar, tooltips, start-up window and interface colours.

The settings are spread over six tabs:

| Tab | Settings |
|---|---|
| **Interface** | language: automatic (from the keyboard layout), or forced: French, English, German, Russian, Spanish, Italian, Portuguese; the detected language and its source are shown. Theme: dark, light or system. Icon and text size: small, normal (the default), large, extra large. Compact bar (icons only when the window is narrow). Tooltips on hover. **Show this window at start-up** (the "Your workspace" window) and the button that opens it at once. **Interface colours**: the colours of the interface lists, chosen **separately for the dark and the light theme** (active buttons, current line of the debugger, highlighted line, selected cell, breakpoint line, addresses and ASCII column of the dump, changed values, labels); each colour has a swatch with a live preview, the defaults are readable (contrast of at least 4.5:1 on the theme background), and **Reset** buttons restore one theme or both. |
| **Editor** | syntax colours, for BASIC and for the 6502 assembler separately (chapter 7) |
| **Library** | the main folder (**Browse...**, **⟲ Default**, **Open folder**), the other folders, the GitHub projects and the optional GitHub token (only for private repositories and Actions artifacts; never displayed, and the `GITHUB_TOKEN` variable takes precedence); library position, left or right (right by default) |
| **Tape** | tape sound (muted at fast speed) and its volume, oscilloscope during playback or recording |
| **GEN2** | Uncle Bernie's color card, 60 or 50 Hz scan, LORES palette (chapter 6) |
| **Advanced** | display and speed: show the library, display limited to ~60 characters/s like the real terminal, fast speed; **Vintage screen: fine settings** (shared by both screens, chapter 3); the Integer BASIC file loaded at `$E000` (📂 **Choose...** / **Forget**) |

Each slider of the preferences (vintage screen intensity, afterglow and curvature, tape volume) shows its **numerical value** next to it. At the first launch the curvature of the vintage screen is set to **0.07** (a slight rounding of the tube); an existing installation keeps its own value. The preview of the **Editor** tab follows the interface language.

The zoom, the phosphor and the vintage screen checkbox are not in the preferences: they are in the display row of each screen (chapter 3).




**Figure 31 —** The Editor tab: a color scheme for BASIC and for the 6502 assembler, or your own colors.

**Automatic language.** The emulator follows the active keyboard layout: `fr` gives French, `us` or `gb` English, `de`, `ch` or `at` German, `ru` Russian, and so on. It asks GNOME, `setxkbmap`, Cinnamon (the Linux Mint case), `localectl`, then `/etc/default/keyboard`, and finally the `LANG` variables. On the command line: `--langue fr|en|de|ru|es|it|pt|auto`.

> ✦ **STARTING FROM SCRATCH**
>
> **Reset preferences**, at the bottom of the window, restores the original settings after a confirmation (language, theme, layout, vintage screen...). Your library files stay where they are; editor tabs that are not saved are closed. The first launch of a new kit version asks the same question, because `gui.json` survives when the program folder is erased to install another one.
>
> For a portable kit, put an empty file named `portable` at the root of the kit (next to `run.sh`), or start it with `APPLE1_PORTABLE=1`: the preferences (`config/gui.json`), the default library (`programmes`) and the cache then live in the kit folder.
>
> Closing the emulator and renaming `~/.config/apple1-emu/gui.json` does the same as the button, at the next launch. Program settings stay in each folder's `programs.json`.

CHAPTER 12

# The command line

*Two programs: apple1-emu-gui, the graphical interface, and apple1-emu, the terminal emulator and conversion toolbox. Everything the export buttons do can also be done from the command line, for your scripts.*

> **! THE COMMAND-LINE TOOLS SPEAK FRENCH**
>
> The graphical interface is available in English ( `--langue en` ), but `apple1-emu` prints its messages and help in French only, and the GUI options, the F2 commands and the debugger keywords are French words ( `--carte-couleur` , `charger` , `pas` ...). This chapter shows the real output and translates it where needed.

## 12.1 The terminal emulator

`apple1-emu` with no command runs the Apple-1 in the terminal (at least 42 columns × 28 lines): 8 KB of RAM plus `$E000` , ACI, real speed.

TERMINAL

```
apple1-emu                                  # 8 KB + E000, ACI, real speed
apple1-emu --fast                           # start in fast mode
apple1-emu --load samples/hello.txt --run 300
apple1-emu --tape samples/hello.wav         # tape in the virtual deck
apple1-emu --basic ~/apple1/integer_basic.bin
apple1-emu --ram 32 --no-e000               # 32 KB, no RAM at E000
```

| Option | Effect |
|---|---|
| `--ram KB` | RAM from `$0000`, 4 to 48 KB (8 by default) |
| `--no-e000` | no RAM at `$E000-$EFFF` (the board's "W → E" jumper) |
| `--no-aci` | no cassette card |
| `--basic FILE` | your Integer BASIC (4 KB binary or hex dump), loaded at `$E000` |
| `--load FILE[@ADDR]` | a program into memory ( `.txt`, `.bin@ADDR`, `.wav`, `.aif`); repeatable |
| `--tape FILE` | a tape in the deck |
| `--type FILE` | types a text file at startup |
| `--run ADDR` | types `ADDRR` after loading |
| `--fast`, `--no-throttle` | fast mode; display without the 60 characters/s limit |
| `--no-refresh` | without the DRAM refresh model (1.0227 MHz instead of 0.960) |
| `--carte-couleur`, `--couleur-50hz` | the GEN2 card (registers and memory; the color picture exists only in the graphical interface) |
| `--cursor-hz HZ`, `--cursor-duty F` | cursor blinking (2.18 Hz, 2/3 visible by default) |
| `--monitor-rom`, `--monitor-page` | another Woz Monitor image (256 or 512 bytes, page 0 or 1) |
| `--aci-rom`, `--aci-page` | another image of the ACI PROM |

**Terminal keys.** Ctrl+R RESET, Ctrl+L CLEAR SCREEN, F1 help, F2 command line, F3 speed, F4 display limit, F5/F6 tape play/stop, F10 or Ctrl+Q quit. Text pasted into the terminal is typed on the Apple-1 keyboard. Backspace types `_`, the Woz Monitor rub-out.

**F2 commands** (the command words are French).

| Command | Effect |
|---|---|
| charger `FILE[@ADDR]` | *load*: loads a `.txt`, a `.bin` ( `@ADDR` required), a `.wav` or an `.aiff` into memory |
| sauver `FILE START.END …` | *save*: saves memory; the extension selects the format. The ranges can be copied from the ACI line: `sauver st.aiff 004A.00FFR 0300.0FFFR` |
| cassette `FILE` | *tape*: inserts a tape, which starts at the ACI's first read |
| `lecture`, `stop`, `rembobiner`, `ejecter` | the deck: play, stop, rewind, eject |
| enregistrer `FILE` | *record*: arms the recorder; then type `C100R` and `0300.03FFW` |
| `basic FILE` | loads your Integer BASIC at `$E000` |
| `taper FILE` | *type*: types a text file on the keyboard |
| `reset`, `effacer`, `rapide`, `reel`, `debit`, `aide`, `quitter` | reset, clear, fast, real speed, display rate, help, quit |

## 12.2 Conversions

The output format is chosen by the **extension**: `.aif` / `.aiff` gives an AIFF (smartphone profile), `.wav` a WAV, `.txt` / `.hex` a Woz dump, `.bin` a binary.

export (alias of `export-wav`) produces a tape. The signal is created by the real ACI ROM running in the emulated 6502: the bit timings are those of an Apple-1 executing `W`.

TERMINAL

```
apple1-emu export samples/hello.txt -o hello.aiff
apple1-emu export prog.bin@300 -o prog.wav --rate 48000 --bits 16
apple1-emu export startrek.txt --range "004A.00FFR 0300.0FFFR" -o startrek.aiff
```

| Option | WAV (default) | AIFF (default) |
|---|---|---|
| `--rate` | 44,100 Hz | 48,000 Hz |
| `--bits 8\|16` | 16 | 16 |
| `--channels 1\|2` | 1 | 2, identical |
| `--amplitude` (0.05 to 1) | 0.8 | 0.89 (−1 dBFS) |
| `--silence S` | 0.5 s | 1.0 s |
| `--range` | one range, or a whole ACI line; repeatable | |
| `--no-refresh`, `--no-embed` | bits 6 % shorter; no addresses in the file | |

import (alias of `import-wav`) decodes a tape: `-o prog.txt` or `-o prog.bin`, `--addr ADDR` for a tape without addresses, `--range` to place several records.

info describes a file (records, addresses, length); with `--aci-input stock|mimeo|gen2`, `--volts` and `--series`, it simulates the analog input of the ACI (section 5.8).

convert `INPUT OUTPUT` converts between `.txt`, `.bin`, `.wav` and `.aiff`.

## 12.3 BASIC and assembler

TERMINAL

```
apple1-emu basic game.bas -o game.wav          # tokenized listing -> tape
apple1-emu basic game.bas -o game.txt --ram 16 # Woz dump, for 16 KB
apple1-emu basic game.bas                      # check only
apple1-emu asm game.s -o game.wav              # 6502 source -> tape
apple1-emu asm game.s -o game.bin -l game.lst  # binary + listing
apple1-emu asm game.s -I ../lib                # check, extra include path
```

`basic` tokenizes a listing as the interpreter does and writes the `$004A-$00FF` block, then the program; `--ram 8|12|16|32` selects the target machine (by default, the smallest that is enough). `asm` assembles a ca65 (or xa) source; without `-o`, it only checks. Both print the line to type on the real Apple-1.

Output of apple1-emu asm hello.s -o hello.txt

```
33 octets, $0300-$0320 ; 2 fichier(s), 13 symboles
Cassette (après C100R) : 0300.0320R   puis 300R
écrit : hello.txt
```

That is: “33 bytes, $0300-$0320; 2 file(s), 13 symbols / Tape (after C100R): 0300.0320R then 300R / written: hello.txt”.

## 12.4 ASCII art

ascii `IMAGE` : `--cols`, `--rows` (23 to keep the prompt), `--zoom`, `--cx`, `--cy`, `--brightness`, `--contrast`, `--gamma`, `--invert`, `--dither`, `--no-auto-level`, `--ramp simple|full|slideshow|LIST`, `--format asm|basic`, `--title`, `-o` ( `.txt`, `.bas`, `.s`, `.wav`, `.aif` ).

## 12.5 PROMs

prom prepares files for programming real 256 × 4 PROMs (82S129, 3601, 6301), in nibble-dump format:

TERMINAL

```
apple1-emu prom split woz -o proms/     # PROM-A2.bin (D7-D4) and PROM-A1.bin (D3-D0)
apple1-emu prom split aci -o proms/     # PROM-A3.bin (D7-D4) and PROM-A4.bin (D3-D0)
apple1-emu prom join A2.bin A1.bin      # reassemble a read-out and compare it
```

`--swap-a1-a7` is only for a board wired after the 1976 Apple schematic, which crosses address lines 1 and 7 of PROM A2.

To burn a 2716-to-27C512 EPROM or a 28C EEPROM from the graphical interface (a TL866II+, T48 or T56 programmer with `minipro`), see the EPROM programmer, section 2.6.

## 12.6 The graphical interface

`apple1-emu-gui [OPTIONS] [FILE…]` loads the given files at startup. Main options (the option names are French):

| Option | Effect |
|---|---|
| `--zoom N\|ajuster`, `--taille WxH` | screen zoom ( `ajuster` = fit), window size |
| `--phosphore vert\|blanc\|ambre` | phosphor (green, white, amber) |
| `--screening`, `--sans-screening`, `--screening-options LIST` | vintage screen of the Apple-1 and its settings |
| `--gen2-screening` | vintage screen on the GEN2 color monitor |
| `--theme clair\|sombre\|systeme` | light, dark or system theme |
| `--taille-icones auto\|petite\|normale\|grande\|tres-grande` | icon and text size |
| `--barre-complete` | never compact the toolbar to icons only |
| `--bibliotheque FOLDER`, `--basic FILE` | library folder, BASIC file |
| `--carte-couleur`, `--sans-carte-couleur`, `--couleur-50hz` | GEN2 card |
| `--langue CODE\|auto` | interface language ( `--langue en` for English) |
| `--mode-rapide`, `--mode-cassette` | library loading: instant (default), or through the tape at real speed |
| `--cassette FILE` | inserts a tape |
| `--ecran-separe`, `--debogueur` | screen in its own window, debugger open |
| `--editeur FILE`, `--ascii FILE` | opens a file in the editor or the ASCII tool |

For demonstrations and screenshots, some options replay actions after a few seconds: `--lancer-biblio S:NAME`, `--charger-biblio S:NAME`, `--parametres S:NAME`, `--preferences S` (and `--onglet-prefs interface|editeur|bibliotheque|cassette|gen2|avance`), `--editeur-lancer S`, `--editeur-recherche TEXT` (search bar open), `--editeur-menu` and `--ecran-menu` (right-click menus open), `--menu-etat` (the State menu open), `--eprom` (the EPROM programmer open), `--fenetre-fichiers` (the Load window open), `--editeur-exporter S:FILE`, `--ascii-reglages R`, `--ascii-lancer S`, `--taper TEXT`, `--taper-a S:TEXT`, `--dbg-arret`, `--dbg-surveiller`, `--dbg-commande S:CMD`, `--capture IMAGE.png` (after `--delai S`) and `--quitter`.

CHAPTER 13

# Troubleshooting and FAQ

## 13.1 The window does not open

The OpenGL driver is missing, or you are in a virtual machine. `run.sh` automatically retries with Mesa software rendering; by hand: `LIBGL_ALWAYS_SOFTWARE=1 ./apple1-emu-gui`. The vintage screen stays smooth with software rendering.

## 13.2 Keys do not reach the Apple-1

Click the Apple-1 screen. When a text field or a dialog box is active, the keyboard goes to that field; the status bar shows where the keyboard goes.

## 13.3 "Integer BASIC not found"

BASIC is not supplied. Put your `BASIC.wav` (or a 4096-byte `.bin`) in the library, or point to it with the **BASIC** button or ⚙ **Preferences** → **Integer BASIC**. The emulator then finds it by itself.

## 13.4 A BASIC program stops with STOPPED AT

A key was pressed while it was running: on the Apple-1, any key interrupts a BASIC program. Type `RUN` to start again, or `GOTO` followed by the line number to resume without clearing the variables.

## 13.5 The program does not start after loading

- BASIC program: you need `E2B3R` (warm entry), not `E000R`, which erases the program. BASIC must be in memory **before** the program.
- Machine code: check the run address in the ⚙ settings; it is not always `300R` (`280R`, `1000R`, `2000R`...).
- Program larger than 8 KB: the RAM is added by ▶, but not by a manual load. Start it with ▶, or use `--ram 16`.

## 13.6 The tape does not load on the real Apple-1

1. Volume at maximum, every audio effect off, no notifications.
2. RETURN during the header, not before playback starts nor after the header ends.
3. The exact ranges, with four digits: `004A.00FFR`, never `4A.FFR`.

4. Try another player (VLC), then `--rate 44100`, `--channels 1`, or `--no-refresh`.
5. Check the level with `apple1-emu info prog.aiff --aci-input stock` (section 5.8).
6. If loading never ends: RESET, then try again with more volume.

## 13.7 The screen is full of inverse @

That is the GEN2 color monitor: its memory starts at zero, and code `$00` is an inverse `@`. The program has not filled the text page yet. On the real card the contents would be random.

## 13.8 Characters come out slowly

This is normal: the Apple-1 terminal displays about 60 characters per second. Click **Display: ~60 chars/s** in the status bar (or **Fast** in the display row of the screen) for instant display, and switch to fast mode (F3) to speed everything up.

## 13.9 The tape sound is muted

It is off at fast speed, and when ALSA or the sound card is missing: the oscilloscope says so ("muted at full speed", "no audio output"). The volume is set in the oscilloscope; 🔊 in the toolbar turns the sound off or on.

## 13.10 The GitHub project does not build

"Test the latest build" builds with ca65/ld65 (package `cc65`) if it finds neither a Release nor an artifact. Install `cc65`, or add an `apple1.toml` that names the right file. Some sources need a recent ca65 (2.19 or later); ca65's error is then shown as is.

## 13.11 What the emulator does not do

- Undocumented 6502 opcodes are treated as NOPs of the right length, and JAM opcodes halt the processor; their exact effects are not reproduced (Apple-1 software does not use them).
- The ACI comparator is ideal during emulation; only the `info` command simulates it as an analog circuit.
- The emulation has been checked against the ROMs, the listings and independent tools (c2t, ca65, xa), but the audio files have not yet been played into a real Apple-1 by the author of this manual.

CHAPTER 14

# Installing a ROM or the original Integer BASIC

*Nothing else is required. The Woz Monitor, the ACI cassette ROM and RDK G2 Basic are already in the program. This page is for anyone who wants Apple's Integer BASIC, or another compatible ROM, from an outside source.*

The kit **does not redistribute any proprietary ROM file**. The Apple-1, the 1976 Integer BASIC and the commercial tapes stay with their rights holders. The links below point at sites that already exist: you fetch your own copy there. The kit does not contain it.

## 14.1 What is already built in

- **Woz Monitor**, at `$FF00`.
- **ACI cassette ROM**, at `$C100`.
- **RDK G2 Basic**, a free BASIC compatible with Integer BASIC, loaded by the **BASIC** button at `$E000`.

## 14.2 Original Integer BASIC

1. Get your own copy, for example the `BASIC.wav` tape of the "Apple1Cassettes" collection on archive.org: https://archive.org/details/Apple1Cassettes.
2. Put the file in your library, or keep it wherever you like.
3. Open ⚙ **Preferences** → **Advanced** → **BASIC**, choose **original Integer BASIC**, then 📂 **Choose...** and point at the file (a `.wav`, a `.txt`, or a 4,096-byte `.bin`).
4. The **BASIC** button then loads that file. **Forget** goes back to the built-in G2 Basic.

The Apple-1 community meets at https://www.applefritter.com/. The kit downloads nothing from these sites: the link is only a pointer.

PART II

# Integer BASIC

*Steve Wozniak's BASIC and its free clone RDK G2 Basic: every statement, memory, the error messages, the known bugs and their workarounds, and porting from Applesoft*

CHAPTER 15

# First steps in BASIC

*The Apple-1's Integer BASIC fits in 4 KB. It knows only integers, one-letter variable names, and has neither DATA nor string functions: it is small, fast, and has its traps.*

> ▣ **WHERE THIS BASIC COMES FROM**
>
> Steve Wozniak wrote it in 1976, by hand, in assembly language, without a computer to assemble it. It was sold on tape ($75) and loads at `$E000-$EFFF`. The emulator does not supply Apple's original: use your own `BASIC.wav` or 4 KB image (chapter 4) if you want it. Every test in this part was done in the emulator, with this original.

## 15.1 Two BASICs: RDK G2 Basic and the original

Since version 7.9 the emulator has its own BASIC, **RDK G2 Basic**, built into the program. It is a clean-room clone of Integer BASIC, written from scratch in 6502 assembly (it contains no code of Apple's interpreter) and distributed under the MIT licence. It is dedicated to Uncle Bernie.

- **Compatible**: same syntax, same tokens, same program format in memory, same tape format. Programs, listings and tapes written for the original run unchanged. It was checked against the original with 22 test sessions whose output is identical.
- **Fixed**: the known bugs of the original, described in chapter 18, are corrected.
- **Bigger**: 7,933 bytes instead of 4 KB. It needs RAM from `$E000` to `$FEFF`; the emulator provides it, but it would not fit in the 4 KB EPROM window of the original.
- **GEN2-aware**: at start it looks for the GEN2 card and, when it finds one, starts the card driver (and then puts LOMEM at `$2000`, above the graphics pages). The graphics statements themselves are planned for version 2 of G2 Basic.
- **Banner**: `RDK G2 BASIC 1.0`, then `DEDICATED TO UNCLE BERNIE`.
- **Extended** (version 7.9.2): `LOMEM=` and `HIMEM=` work as in the original, and a new function `KEY(n)` reads the keyboard without waiting, so that a game can look at the keys without `INPUT` blocking it and without a key press stopping the program (chapter 15). `LIST a,b` and `DEL a,b` use the original's ranges, and a listed line longer than the screen wraps as the original does.

The **BASIC** button loads G2 Basic. To use Apple's original instead, open ⚙ **Preferences → Advanced → BASIC**, choose **Original Integer BASIC (your own file, loaded at $E000)** and pick the file; the choice is remembered, and the button then loads it. The two behave alike for everything described in this part; the differences are noted where they matter (HIMEM at start, the bugs of chapter 18).

> **SOURCE AND TESTS**
> G2 Basic has its own repository, `Old-Wirehead/rdk-gen2basic-apple1`, with its manual, its source and its regression tests.

## 15.2 Starting BASIC

Click **BASIC** in the toolbar: the emulator loads the BASIC you chose (G2 Basic by default) and you start it with `E000R`. The BASIC prompt is `>`, the Woz Monitor's is `\`.

WOZ MONITOR

```
\
E000R
E000: 4C
>
```

- `E000R`: **cold start**, erases the program in memory.
- `E2B3R`: **warm start**, keeps the program. This is the address to use after loading a program from tape, and to return to BASIC after a trip through the monitor (RESET).

E000RE2B3R

## 15.3 Direct mode and program mode

A line that begins with a **number** (0 to 32767) is stored in the program; a line without a number is executed at once.

WOZ MONITOR

```
>PRINT 2*3+4*5
26
>10 PRINT "HELLO"
>20 GOTO 10
>RUN
HELLO
HELLO
…
```

Press a key to stop the program: BASIC prints `STOPPED AT 10`. Retyping a line with the same number replaces it; a number alone deletes it.

> **! DIRECT MODE IS LIMITED**
> In direct mode, BASIC refuses statements separated by `:` (`SYNTAX ERR`), as well as `FOR` and `GOSUB`. To try a loop, write it in a numbered line.

## 15.4 Correcting a line

The Apple-1 keyboard has no delete key: the emulator's **Backspace** types `_`, which cancels the previous character (the line on screen is not corrected, but BASIC receives the right one). Esc cancels the whole line: BASIC prints `\` and waits for the next line. For more comfortable editing, use the editor (chapter 7): it colors the syntax, reports errors before sending, and types the program for you.

## 15.5 A first program

boucle.bas — INTEGER BASIC

```
10 S=0
20 FOR I=1 TO 10
30 S=S+I
40 NEXT I
50 PRINT "SUM = ";S
60 END
```

WOZ MONITOR

```
>RUN
SUM = 55
>
```

In the editor, ▶ **Run** types the program and `RUN`; in the debugger you can follow it line by line (figure in chapter 8).

## 15.6 Commands

| Command | Effect |
|---|---|
| LIST `[n[,m]]` | lists the program, line `n`, or lines `n` to `m` |
| RUN `[n]` | clears the variables and runs the program, from line `n` if given |
| DEL `n[,m]` | deletes line `n`, or lines `n` to `m` |
| SCR | erases the program and the variables |
| CLR | clears the variables, keeps the program |
| AUTO `n[,step]` | automatic numbering from `n` (step 10 by default); Ctrl+D then RETURN leaves it, Esc does not |
| MAN | returns to manual numbering |
| HIMEM `=n`, LOMEM `=n` | the bounds of BASIC's memory (chapter 16) |

`GOTO n` in direct mode resumes a stopped program **without** clearing the variables, unlike `RUN n`.

✦ LIST AND THE SLOW DISPLAY

The Apple-1 terminal displays about 60 characters per second: listing a long program takes time. Turn off the display limit (**Fast** in the display row, or in the status bar) to read faster, or list in slices: `LIST 100,200` .

## 15.7 Three programs to type in

### Guess a number

devine.bas — INTEGER BASIC

```
10 REM GUESS A NUMBER FROM 1 TO 100
20 N=RND(100)+1:C=0
30 PRINT "I PICKED A NUMBER FROM 1 TO 100."
40 INPUT "YOUR GUESS",E
50 C=C+1
60 IF E<N THEN PRINT "HIGHER!"
70 IF E>N THEN PRINT "LOWER!"
80 IF E#N THEN 40
90 PRINT "WELL DONE, IN ";C;" TRIES."
100 END
```

WOZ MONITOR

```
>RUN
I PICKED A NUMBER FROM 1 TO 100.
YOUR GUESS?50
HIGHER!
YOUR GUESS?75
LOWER!
YOUR GUESS?62
LOWER!
YOUR GUESS?56
HIGHER!
YOUR GUESS?57
WELL DONE, IN 5 TRIES.
```

`RND(100)` gives an integer from 0 to 99; `#` means “not equal”.

### A multiplication table

table.bas — INTEGER BASIC

```
10 DIM T(10)
20 INPUT "TABLE OF",N
30 FOR I=1 TO 10:T(I)=N*I:NEXT I
40 FOR I=1 TO 10
50 PRINT I;" X ";N;" = ";T(I)
60 NEXT I
70 S=0:FOR I=1 TO 10:S=S+T(I):NEXT I
80 PRINT "TOTAL: ";S
90 END
```

### Playing with strings

chaines.bas — INTEGER BASIC

```
10 DIM N$(20),R$(20)
20 INPUT "YOUR NAME",N$
30 L=LEN(N$)
40 PRINT "INITIAL: ";N$(1,1)
50 PRINT "END: ";N$(L-1)
60 R$="":FOR I=L TO 1 STEP -1
70 R$(LEN(R$)+1)=N$(I,I)
80 NEXT I
90 PRINT "BACKWARDS: ";R$
100 IF R$=N$ THEN PRINT "IT'S A PALINDROME!"
110 END
```

WOZ MONITOR

```
>RUN
YOUR NAME?KAYAK
INITIAL: K
END: AK
BACKWARDS: KAYAK
IT'S A PALINDROME!
```

Strings are sliced with `N$(start,end)` and extended with `R$(LEN(R$)+1)=…` : there is neither `MID$` nor `+` for strings (chapter 16).

The four programs are in `docs/exemples/en/basic/` : open them in the editor, or make a tape with `apple1-emu basic devine.bas -o devine.aiff`.

CHAPTER 16

# Statement and function reference

*Every statement recognized by the Apple-1's Integer BASIC, taken from the interpreter's keyword table, with what it really does. Every example was tried.*

## 16.1 Statements

```
PRINT list [;]
```

PRINT displays numbers and strings. `;` joins items with no space (`PRINT A;B` prints `12` for A=1, B=2); `,` moves to the next column, in steps of 8 (`PRINT 1,2,3` prints `1 2 3`). A final `;` suppresses the line break. `PRINT` alone moves to the next line.

```
INPUT ["prompt",] variable [,variable…]
```

INPUT reads a number or a string. BASIC always prints `?` after the prompt: `INPUT "NAME",N$` prints `NAME?`. A string too long for its `DIM` gives `STR OVFL ERR`.

```
LET variable = expression
```

LET is optional: `A=5` is enough.

```
IF expression THEN statement | number
```

IF executes the statement when the expression is not zero (`IF A THEN …` is allowed). `THEN 100` jumps to line 100. **Caution**: only the first statement depends on the condition (chapter 18, bug 4).

```
GOTO expression — GOSUB expression — RETURN
```

GOTO and GOSUB accept an **expression**: `GOTO K*100`, `GOSUB 300+K` (the "computed GOTO" stands in for the missing `ON … GOTO`). A nonexistent line gives `BAD BRANCH ERR`. At most eight nested GOSUBs (`>8 GOSUBS ERR` on the ninth); `RETURN` without `GOSUB` gives `BAD RETURN ERR`.

```
FOR var = start TO end [STEP step] … NEXT var
```

FOR/NEXT: the body is executed **at least once**, even if start is past end (`FOR I=5 TO 1` goes through once). `STEP` may be negative. At most eight nested loops (`>8 FORS ERR`). `NEXT` must name the variable; a `NEXT` without `FOR` gives `BAD NEXT ERR`.

```
DIM var(n) [,var$(n)…]
```

DIM creates an integer array of `n` elements, numbered **from 1 to n** (`A(0)` gives `RANGE ERR`), or reserves `n` characters (at most 255) for a string. Every string must be dimensioned before it is assigned.

```
REM text
```

REM: a comment, up to the end of the line (a `:` after REM is part of the comment).

```
END
```

END ends the program. A program that ends without `END` prints `*** END ERR` : this is harmless, but end your programs with `END` .

`TAB n`

TAB is a **statement** (not a PRINT function as in Applesoft): `TAB 10:PRINT "*"` prints the star in column 10.

`POKE address , value`

POKE writes a byte (0 to 255, otherwise `>255 ERR` ). Addresses above 32767 are written as negative numbers: address − 65536.

`CALL address`

CALL calls a machine-language program, which returns with `RTS` . Example: `POKE 768,96 : CALL 768` (96 = `RTS` ). `CALL -256` returns cleanly to the Woz Monitor ( `$FF00` ).

## 16.2 Functions

| Function | Result | Verified example |
|---|---|---|
| PEEK `(addr)` | the byte at the address | `PEEK(76)+PEEK(77)*256` gives HIMEM (4096) |
| RND `(n)` | random integer from 0 to n−1; `RND(0)` : `>32767 ERR` ; `RND(-5)` gives a negative number | `RND(6)+1` : a die |
| SGN `(x)` | −1, 0 or 1 (also for a number of 256 or more) | `SGN(-5);SGN(0)` prints `-10` |
| ABS `(x)` | absolute value | `ABS(-9)` = 9 |
| `KEY(n)` (G2 Basic only) | code of the key pressed, without waiting (see below) | `K=KEY(0)` |
| LEN `(a$)` | length of the string | `LEN("ABC")` = 3 |
| USR `(x)` | calls `$0000` : **crashes** without a prepared program | — |
| ASC `(` , SCRN `(` | present in the table, but `SYNTAX ERR` | — |

## 16.3 Operators

| Precedence | Operators |
|---|---|
| 1 | unary `-` and `+` , `NOT` |
| 2 | `*` `/` `MOD` |
| 3 | `+` `-` |
| 4 | `=` `#` (not equal, also `<>` ) `<` `>` `<=` `>=` |
| 5 | `AND` |
| 6 | `OR` |

MOD, AND, OR and NOT:

- Division **truncates toward zero**: `7/2` = 3, `-7/2` = −3. `MOD` takes the sign of the dividend: `-7 MOD 3` = −1.
- Comparisons give 1 (true) or 0 (false): `PRINT 5>3;5=3` prints `10 . NOT 0` = 1, `NOT 5` = 0.
- `AND` and `OR` are logical, not bitwise: `3 AND 0` = 0, `3 OR 0` = 1.
- `3*-2` is accepted and equals −6.
- `^` exists in the table, but **hangs the machine** (chapter 18, bug 2).
- Strings can only be compared with `=` and `#` ; `"ABC"<"ABD"` is refused ( `SYNTAX ERR` ).

## 16.4 Extensions of RDK G2 Basic

These work only with **RDK G2 Basic** (the original Integer BASIC answers `SYNTAX ERR` ).

`KEY(n)`

`KEY(0)` returns the **code of the key** that was pressed (the 7-bit ASCII code: 65 for `A` , 13 for Return, 32 for the space bar) and consumes it, or `0` if no key was pressed. It never waits, so a program can run while it watches the keyboard. `KEY(1)` (any other non-zero argument) **waits** for a key.

Wait for a key without INPUT

INTEGER BASIC

```
10 K=KEY(0)
20 IF K=0 THEN 10
30 PRINT K
```

In the original, a key pressed while a program runs **stops it** ( `STOPPED AT n` ), and `INPUT` blocks until Return. With G2 Basic, once a program has called `KEY` , a key press no longer stops it: only `KEY` sees the keyboard, and Ctrl+C (code 3) read by `KEY` stops the program at the next call. Call `KEY` in the main loop of the program, then.

`LOMEM: n` — `HIMEM: n`

Immediate commands that take an expression ( `LOMEM=2048+512` ); inside a program they give `SYNTAX ERR` , as in the original. They are described with the memory in chapter 16.

`LIST [a[,b]]` — `DEL a[,b]`

`LIST` lists everything, `LIST n` one line, `LIST a,b` the lines from `a` to `b` . `DEL n` deletes one line, `DEL a,b` the lines from `a` to `b` (if `b` is below `a` , only line `a` ). A listed line longer than the screen is wrapped like the original's (continuation lines are indented by six spaces).

String slices follow the original: `A$(I)` with `I` of 256 or more, or negative, gives `>255 ERR` , and `A$(I,J)` with `J` below `I-1` gives `STRING ERR` .

## 16.5 Keywords that do nothing

The table contains `COLOR=` , `PLOT` , `HLIN` , `VLIN` and `VTAB` , inherited from the BASIC project for the Apple II. On the Apple-1, `COLOR=` and `PLOT` are accepted and do nothing, `VTAB` gives `SYNTAX ERR` . Keep them out of your programs.

CHAPTER 17

# Variables, strings, arrays and memory

## 17.1 Numbers

Integer BASIC knows only **signed 16-bit integers**, from −32767 to 32767. A calculation that leaves this range stops with `*** >32767 ERR` (`200*200`, `32767+1`, and also `10/0` and `RND(0)`). The value −32768 cannot be used (chapter 18, bug 5). There is no decimal point and no `INT`: `7/2` is 3.

## 17.2 Variable names

This is the limit that surprises people most: a variable name is **one letter, optionally followed by one digit**.

| Name | Result (tried) |
|---|---|
| `A`, `Z`, `B1`, `Z9` | accepted |
| `AB`, `CNT`, `SCORE`, `COUNTER` | `SYNTAX ERR` |
| `A12` | `SYNTAX ERR` |
| `A$`, `N$` | string: **one letter** only |
| `B9$`, `Z9$` | `SYNTAX ERR` |

That makes 26 × 11 = 286 possible integer variables, and 26 strings. Get into the habit of noting the role of each variable in a `REM`.

## 17.3 Strings

A string must be **dimensioned** before use, with its maximum length (1 to 255): `DIM N$(20)`. `DIM B$(256)` gives `>255 ERR`.

| Operation | Written as | Example (N$ = “KAYAK”) |
|---|---|---|
| length | `LEN(N$)` | 5 |
| substring | `N$(i,j)` : characters i to j | `N$(1,3)` = “KAY” |
| tail | `N$(i)` : from character i to the end | `N$(4)` = “AK” |
| concatenate | `A$(LEN(A$)+1)=B$` | “APPLE” + “-1” → “APPLE-1” |
| compare | `=` and `#` only | `IF N$="WOZ" THEN …` |

Assigning a string longer than its `DIM` gives `*** STR OVFL ERR`, as does an answer to `INPUT` that is too long. There is no `MID$`, `LEFT$`, `CHR$`, `STR$`, `VAL` or `ASC` (the latter is in the table but does not work).

> ✦ **REPLACING CHR$ AND VAL**
>
> To print a character from its code, keep a reference string and slice it: `DIM C$(26):C$="ABCDEFGHIJKLMNOPQRSTUVWXYZ"` then `PRINT C$(K,K)`. To read a number, use `INPUT N` rather than `INPUT N$`.

## 17.4 Arrays

`DIM T(10)` creates an array of 10 integers, **numbered from 1 to 10**. `T(0)` and `T(11)` give `*** RANGE ERR`. There is only one dimension: for an 8 × 8 grid, use `G((L-1)*8+C)`. In the emulator, elements are 0 after `DIM`; on a real machine memory is not necessarily cleared: initialize your arrays. An array too large for memory gives `*** MEM FULL ERR` (`DIM A(3000)` in 8 KB).

## 17.5 BASIC's memory

| Addresses | Contents | Notes |
|---|---|---|
| FF00–FFFF | **Woz Monitor** (256-byte PROM) | NMI, RESET, IRQ vectors |
| E000–EFFF | **Integer BASIC** (4 KB of RAM) | loaded from tape |
| D010–D013 | **PIA 6821**: keyboard, display | KBD, KBDCR, DSP, DSPCR |
| C250–C257 | **GEN2 card**: soft switches | optional, read access |
| C100–C1FF | **ACI PROM** (C100R) | C000–C0FF: flip-flop |
| 1000–BFFF | expansion RAM (up to 48 KB) | HIRES 2000–5FFF (GEN2) |
| 0800–0FFF | BASIC programs (LOMEM–HIMEM) | text/LORES 2: 0800 |
| 0300–07FF | your machine-language programs | text/LORES 1: 0400 |
| 0200–027F | monitor input buffer | keep clear |
| 0100–01FF | 6502 stack | |
| 0000–00FF | zero page | monitor 24–2B, BASIC 4A–FF |

**Figure 32 —** Apple-1 memory: BASIC at $E000, the program below HIMEM, the variables above LOMEM.

BASIC stores the **program at the top** of memory, growing down from HIMEM, and the **variables at the bottom**, growing up from LOMEM. The free space lies between the two.

| Pointer | Address | Value in 8 KB | Role |
|---|---|---|---|
| LOMEM | `$4A-$4B` (74-75) | `$0800` (2048) | start of the variables |
| HIMEM | `$4C-$4D` (76-77) | `$1000` (4096) | end of BASIC's memory |
| PP | `$CA-$CB` (202-203) | moves down with the program | start of the program |
| PV | `$CC-$CD` (204-205) | moves up with the variables | end of the variables |

`PRINT PEEK(202)+PEEK(203)*256` gives the start of the program and `PRINT PEEK(204)+PEEK(205)*256` the end of the variables: the free space is the difference between the two. An integer variable such as `A` takes 6 bytes.

▣ **WHY A BASIC TAPE STARTS AT $004A**

These pointers are in zero page, from `$4A` to `$FF`. A BASIC tape therefore records **two ranges**: `004A.00FF` (the pointers), then the program from PP to HIMEM, for example `0F2C.0FFF`. The `apple1-emu basic` command computes them for you (its output is in French: "10 lines, 212 bytes... 1836 bytes free for the variables... ACI ranges (after C100R)... then E2B3R and RUN"):

```
10 lignes, 212 octets ; LOMEM $0800 HIMEM $1000 PP $0F2C ; 1836 octets libres pour
les variables ; RAM 8 Ko
Plages ACI (après C100R) : 004A.00FFR 0F2C.0FFFR   puis E2B3R et RUN
```

## Changing LOMEM and HIMEM

The syntax is `LOMEM=n` and `HIMEM=n` (with `=`, **not** `:` as on the Apple II, which gives `SYNTAX ERR`).

- `LOMEM=2304` moves the start of the variables and **clears the variables** (the program stays). Useful to reserve `$0800-$08FF` for a machine-language program.
- `HIMEM=` **higher** (with more RAM, for example `HIMEM=8192` in 16 KB) works and keeps the program.
- `HIMEM=` **lower** is **silently ignored** by the original, even after `SCR` (chapter 18, bug 11). **RDK G2 Basic accepts it**: the program in memory is moved down with the limit, or `MEM FULL ERR` is given if it would no longer fit above the variables (`RANGE ERR` for a negative value; `LOMEM=` gives `RANGE ERR` if it is not below the start of the program, and `>32767 ERR` above 32767). With the original, there is a verified workaround: write the pointer directly, then `SCR`:

Lowering HIMEM to 3072 ($0C00) — INTEGER BASIC

```
POKE 76,0 : POKE 77,12 : SCR
```

At the start of G2 Basic, HIMEM is the top of the RAM found and LOMEM is `$0800` (`$2000` with a GEN2 driver). (`76` / `77` = `$4C` / `$4D`, 12 × 256 = 3072.) Type these three statements on a numbered line or separately in direct mode, **before** entering the program: `SCR` erases it.

## 17.6 Mixing BASIC and machine language

| Area (8 KB) | Usable for 6502 code? |
|---|---|
| `$0000-$00FF` | zero page: the Woz Monitor uses `$24-$2B`, BASIC at least `$4A-$FF`; avoid it, or save and restore it |
| `$0100-$01FF` | 6502 stack: no |
| `$0200-$027F` | Woz Monitor line buffer; overwritten by every line typed |
| `$0280-$07FF` | free as long as BASIC does not touch it: the best place |
| `$0800-…` | BASIC variables (unless LOMEM is raised) |
| `… -$0FFF` | BASIC program |

The machine-language program is loaded from tape or by POKE, and called with `CALL 768` (for `$0300`). It must end with `RTS` and preserve BASIC's zero page. Minimal example:

WOZ MONITOR

```
>POKE 768,96
>CALL 768
>
```

(96 = `$60` = `RTS`: the call returns at once.)

CHAPTER 18

# Error messages and limits

## 18.1 Messages

An error prints `*** MESSAGE ERR`, followed, when a program is running, by `STOPPED AT n`. A line refused on entry is not stored: retype it.

| Message | Cause | Example tried |
|---|---|---|
| SYNTAX ERR | unknown statement, invalid variable name, `:` in direct mode | `CNT=7`, `GET A$`, `X=5:PRINT X` in direct mode |
| >32767 ERR | overflow, division by zero, line number > 32767 | `200*200`, `10/0`, `RND(0)`, `32768 PRINT` |
| TOO LONG ERR | line too complex (expression nested too deeply) or too long | 13 nested parentheses |
| MEM FULL ERR | no more room between variables and program | `DIM A(3000)` in 8 KB |
| TOO MANY PARENS ERR | parentheses (provided for by the interpreter; masked by bug 3) | — |
| STRING ERR | string misused | — |
| END ERR | the program ends without `END` | `10 PRINT "X"` then `RUN` |
| BAD BRANCH ERR | `GOTO` / `GOSUB` to a nonexistent line | `GOTO 500` |
| >8 GOSUBS ERR | more than 8 nested `GOSUB`s | recursion: 9 calls |
| BAD RETURN ERR | `RETURN` without `GOSUB` | `RUN 30` on a `RETURN` |
| >8 FORS ERR | more than 8 nested `FOR`s, or bug 1 | 9 loops on one line |
| BAD NEXT ERR | `NEXT` without a matching `FOR` | `20 NEXT I` alone |
| STOPPED AT | a key was pressed while running | — |
| >255 ERR | value > 255: `POKE`, `DIM` of a string | `POKE 768,300`, `DIM B$(256)` |
| RANGE ERR | array index out of bounds | `A(0)`, `A(3)` after `DIM A(2)` |
| DIM ERR | array or string dimensioned again | — |
| STR OVFL ERR | string longer than its `DIM` | `A$="ABCDE"` with `DIM A$(3)` |
| RETYPE LINE | invalid answer to `INPUT` (letters for a number): retype | — |

The "—" rows come from the interpreter's message table; they were not provoked in our tests.

## 18.2 Limits

| Limit | Value |
|---|---|
| integers | −32767 to 32767 |
| line numbers | 0 to 32767 |
| variable names | letter + optional digit |
| string names | one letter + `$` |
| string length | 255 |
| array dimensions | 1, indices from 1 to n |
| nested `FOR` s | 8 |
| nested `GOSUB` s | 8 |
| nested parentheses | 12 |
| length of a typed line | 120 characters get through, 141 are refused |

## 18.3 Saving memory

In 8 KB, BASIC has about 2 KB for the program and the variables. A few ways to gain some:

1. **Lower LOMEM** if you have no machine code: `LOMEM=768` (`$0300`) gives the variables 1280 more bytes. Tried: `DIM T(1200)` fails with `MEM FULL` with the default LOMEM, and succeeds after `LOMEM=768`. The ports of *101 BASIC Computer Games* made during development do just that.
2. **Write tersely**: BASIC keeps spaces and `REM` s. `LIST` re-indents by itself, and one long line holding several statements costs less than several lines.
3. **Reuse arrays and strings** rather than dimensioning new ones.
4. **Add RAM**: a program's ⚙ settings, `--ram 16`, or a real expansion. A program over 8 KB requires a 12, 16 or 32 KB machine: `apple1-emu basic --ram 16` places it at the top of that memory.

CHAPTER 19

# Known bugs and their workarounds

*The Apple-1's Integer BASIC was written without an assembler, in 1976, and was never fixed. Here are its known defects, each reproduced in the emulator with the real BASIC, and how to avoid them.*

VERIFIED: reproduced in the emulator with the real Integer BASIC, with a transcript to back it up. NOT VERIFIED: reported by a source, not reproduced. Bug n corresponds to section 18.n. Since the emulator runs the original code instruction by instruction, a bug reproduced here also exists on a real Apple-1.

## 19.1 Leaving a FOR loop with GOTO VERIFIED

A loop left with `GOTO` stays "open". If the program goes through the `FOR` again, a new entry is pushed; on the ninth pass: `>8 FORS ERR`.

INTEGER BASIC

```
10 N=0
20 FOR I=1 TO 10
30 IF I=2 THEN GOTO 50
40 NEXT I
50 N=N+1:IF N<20 THEN 20
60 PRINT "DONE ";N
70 END
```

WOZ MONITOR

```
>RUN
*** >8 FORS ERR
STOPPED AT 20
```

✦ WORKAROUND

Never leave a `FOR` with `GOTO`: give the variable its final value and go through `NEXT`. Here: `30 IF I=2 THEN I=10:GOTO 40`. The program then prints `DONE 20`. The ports of 101 BASIC *Computer Games* follow this rule.

## 19.2 The ^ operator hangs the machine VERIFIED

`^` (power) is in the interpreter's table, but `PRINT 2^3` never returns: only RESET gets you out.

✦ WORKAROUND

Multiply: `2*2*2`, or a loop `P=1:FOR I=1 TO N:P=P*X:NEXT I`.

## 19.3 The “PPED AT” message VERIFIED

When the interpreter’s operand stack overflows (an expression nested too deeply in direct mode), BASIC should print an error message, but it points **3 bytes too far** into the message table (Eric Smith’s disassembly notes it: `ldy #erri_stopped_at+3 ; The "+3" is a bug!`). What you read is the end of `STOPPED AT`:

WOZ MONITOR

```
>PRINT 1+(1+(1+(1+(1+(1+(1+(1+(1+(1+(1+(1+(1))))))))))))
14
*** PPED AT  ERR
STOPPED AT 65535
```

✦ WORKAROUND

Break long expressions up with intermediate variables. Beyond 12 levels of parentheses, a program line is refused anyway (`TOO LONG ERR`).

## 19.4 IF … THEN governs only the first statement VERIFIED

In `IF A=2 THEN PRINT "X":PRINT "Y"`, only `PRINT "X"` depends on the condition: `PRINT "Y"` is **always** executed (with A=1, the program prints `Y`). This is the most common trap when porting a program written for another BASIC.

✦ WORKAROUND

Invert the condition and jump over the block:

INTEGER BASIC

```
80 IF A#2 THEN 90
85 PRINT "X":PRINT "Y"
90 REM CONTINUE
```

## 19.5 −32768 VERIFIED

`X=-32767-1` prints a `-` then `*** >32767 ERR`. The smallest usable value is −32767.

✦ WORKAROUND

Stay within −32767 … 32767. To read a 16-bit word with `PEEK`, split it: `PEEK(A)+PEEK(A+1)*256` overflows above 32767; test the high byte first (`IF PEEK(A+1)>127 THEN …`).

## 19.6 ASC( does not work VERIFIED

`ASC("A")` is in the keyword table, but any line that uses it is refused: `SYNTAX ERR`. The same goes for `SCRN(`.

✦ WORKAROUND (TRIED)

Look the character up in a reference string. Line 60 closes the loop ( `K=26:NEXT K` ) to avoid bug 1:

The position of a letter, without ASC — INTEGER BASIC

```
10 DIM C$(26),L$(1)
20 C$="ABCDEFGHIJKLMNOPQRSTUVWXYZ"
30 INPUT "LETTER",L$
40 FOR K=1 TO 26:IF C$(K,K)=L$ THEN 60:NEXT K
50 PRINT "NOT A LETTER":END
60 PRINT "POSITION ";K:K=26:NEXT K
70 GOTO 30
```

WOZ MONITOR

```
LETTER?W
POSITION 23
LETTER?E
POSITION 5
LETTER?5
NOT A LETTER
```

## 19.7 Graphics keywords that do nothing VERIFIED

`COLOR=`, `PLOT` and `HLIN` are accepted and do nothing; `VLIN` and `VTAB` give `SYNTAX ERR`. They are leftovers of the BASIC intended for the Apple II.

✦ WORKAROUND

To position text, use `TAB n` and empty `PRINT`s. To draw in color, the GEN2 card is driven with `POKE` (chapter 6).

## 19.8 The Apple II 251-character bug ABSENT

A later version of Integer BASIC (Apple II) crashes on certain lines of 251 characters or more. On the Apple-1 the typed line is limited well before that (about 128 characters): the bug cannot occur.

## 19.9 STOPPED AT 65535 VERIFIED

An error caused by a line **refused while it is being entered** (nested too deeply) or by bug 3 in direct mode is followed by `STOPPED AT 65535` : BASIC believes it is in a program and prints an absurd line number. It has no consequences; the program in memory is not touched.

## 19.10 USR crashes the machine VERIFIED

`X=USR(1)` jumps to `$0000`, where there is normally no code: the machine hangs. RESET then `E2B3R` return to BASIC without losing the program.

> ✦ **WORKAROUND**
> Call your machine-language programs with `CALL address`; pass values with `POKE` and `PEEK`.

## 19.11 HIMEM= cannot go down VERIFIED

`HIMEM=3072` (and any value below the current one) is **silently ignored**, even after `SCR`, because the interpreter compares the new value with the program pointer before accepting it.

> ✦ **WORKAROUND (VERIFIED)**
> `POKE 76,0 : POKE 77,12 : SCR` sets HIMEM to 3072; the next program is stored below it. See chapter 16.

## 19.12 The colon refused in direct mode VERIFIED

`X=5:Y=X*2:PRINT Y` typed without a number gives `SYNTAX ERR`; the same line with a number works. `FOR` and `GOSUB` are also refused in direct mode.

> ✦ **WORKAROUND**
> Type one statement per line, or write a temporary line `0` and `RUN 0` ... without forgetting to delete it (`DEL 0`).

## 19.13 INPUT always adds "?" VERIFIED

`INPUT "AGE ? ",N` prints `AGE ? ?`: BASIC adds its own question mark. Write the prompt without `?`: `INPUT "AGE",N` prints `AGE?`.

## 19.14 The FOR loop runs at least once VERIFIED

`FOR I=5 TO 1 … NEXT I` executes the body once (with I=5), where other BASICs skip it.

> ✦ **WORKAROUND**
> Test first: `IF N<1 THEN 100` before `FOR I=1 TO N`.

## 19.15 A key press stops the program VERIFIED

Any key pressed while the program runs stops it (`STOPPED AT`), except during an `INPUT`. This is not a bug but a design choice: there is no BREAK key. Consequence: a real-time game cannot wait for a key without stopping. Reading the keyboard with `PEEK(-12271)` (`$D011`) and `PEEK(-12272)`

( `$D010` ) looks possible, but the interpreter also tests the keyboard between statements and may stop first: we have not tried it (NOT VERIFIED). For a real-time game, move to machine language (Part III).

## 19.16 Summary

| # | Bug | Status | Workaround |
|---|---|---|---|
| 1 | leaving FOR with GOTO → `>8 FORS` | **verified** | end the loop with NEXT |
| 2 | `^` hangs | **verified** | multiply |
| 3 | “PPED AT” (+3) | **verified** | intermediate variables |
| 4 | `IF…THEN a:b` executes b | **verified** | inverted condition + jump |
| 5 | −32768 | **verified** | stay ≥ −32767 |
| 6 | `ASC(` , `SCRN(` | **verified** | reference string |
| 7 | COLOR=, PLOT, HLIN do nothing; VLIN, VTAB refused | **verified** | TAB, POKE (GEN2) |
| 8 | 251 characters (Apple II) | **absent** | — |
| 9 | STOPPED AT 65535 | **verified** | harmless |
| 10 | USR crashes | **verified** | CALL |
| 11 | HIMEM= does not go down | **verified** | POKE 76/77 + SCR |
| 12 | `:` refused in direct mode | **verified** | one statement per line |
| 13 | INPUT adds “?” | **verified** | prompt without “?” |
| 14 | FOR runs once | **verified** | test first |
| 15 | reading the keyboard with PEEK during RUN | **not verified** | machine language |

CHAPTER 20

# Porting an Applesoft or Microsoft BASIC program

*Most listings from the 1970s and 1980s are written in Microsoft BASIC (Applesoft, the BASIC of "101 BASIC Computer Games"). They can be ported to the Apple-1, provided you know the equivalents. Every "refused" line below was typed into the real BASIC.*

## 20.1 Equivalents

| Applesoft / Microsoft | Apple-1 Integer BASIC | Note |
|---|---|---|
| `ON X GOTO 100,200,300` | `GOTO 100*X` (if the line numbers allow it) or a series of `IF X=1 THEN 100` | refused |
| `GET A$` | `INPUT A$` | refused |
| `DATA` / `READ` / `RESTORE` | array filled by assignments, or a sliced string | refused |
| `MID$(A$,I,N)` | `A$(I,I+N-1)` | refused |
| `LEFT$(A$,N)` / `RIGHT$(A$,N)` | `A$(1,N)` / `A$(LEN(A$)-N+1)` | — |
| `A$+B$` | `A$(LEN(A$)+1)=B$` | — |
| `CHR$(N)` , `ASC(A$)` | reference string (chapter 18, bug 6) | refused |
| `STR$(N)` , `VAL(A$)` | `PRINT N` directly; `INPUT N` | refused |
| `INT(X)` | unnecessary: everything is an integer | refused |
| `RND(1)` (real from 0 to 1) | `RND(N)` (integer from 0 to N−1) | — |
| `X^2` | `X*X` | hangs (bug 2) |
| `HOME` | 24 × `PRINT` , or CLEAR SCREEN (Ctrl + L) by hand | refused |
| `PRINT TAB(10);"X"` | `TAB 10:PRINT "X"` | — |
| `STOP` | `END` | refused |
| `DEF FN` | `GOSUB` | — |
| `A<>B` | `A#B` ( `<>` is also accepted) | — |
| `IF A THEN X:Y` | inverted condition + jump (bug 4) | trap |
| `DIM A(10,10)` | `DIM A(121)` and `A(I*11+J+1)` | — |
| `A(0)` | indices start at 1 | `RANGE ERR` |
| `SCORE` , `NAME$` | `S` , `N$` : one letter (+ digit) | `SYNTAX ERR` |
| real numbers | integers ×10 or ×100 (fixed point) | — |

## 20.2 The method

1. **List the variables** of the original program and assign each one a letter (+ digit). Record the table in `REM` s at the top.
2. **Replace the DATA** with assignments in an initialization subroutine, or with a string you slice.
3. **Find every** `IF … THEN … : …` and rewrite it with a jump (bug 4).
4. **Find every exit from a** `FOR` **by** `GOTO` and replace it with “final value + `NEXT` ” (bug 1).

5. **Shift array indices** by 1 if the program uses index 0.
6. **Check in the editor**: it flags words unknown to Integer BASIC as you type. ▶ **Run** types the program in: any refused line appears on screen with `SYNTAX ERR`.
7. **Measure the memory**: `apple1-emu basic prog.bas` gives the size and the RAM needed. Too big for 8 KB? `LOMEM=768` (chapter 17), or a 16 KB machine.
8. **Export** the tape (📼 **Export WAV**) and note the ranges for the real machine.

## 20.3 A porting example

A typical fragment in Microsoft BASIC:

Original (Microsoft BASIC)

```
100 READ N$,S
110 IF S>HI THEN HI=S:BEST$=N$
120 IF N$<>"END" THEN 100
130 PRINT "BEST: ";BEST$;" (";HI;")"
140 DATA "ANNE",12,"BOB",17,"CLAIRE",9,"END",0
```

On the Apple-1: no `READ` / `DATA`, no long names, and the `IF` on line 110 would assign `BEST$` on every pass (bug 4).

Integer BASIC port (tried) — INTEGER BASIC

```
10 DIM N$(10),B$(10),S(3)
20 S(1)=12:S(2)=17:S(3)=9
30 H=0
40 FOR I=1 TO 3
50 GOSUB 200
60 IF S(I)<=H THEN 80
70 H=S(I):B$=N$
80 NEXT I
90 PRINT "BEST: ";B$;" (";H;")"
100 END
200 IF I=1 THEN N$="ANNE"
210 IF I=2 THEN N$="BOB"
220 IF I=3 THEN N$="CLAIRE"
230 RETURN
```

WOZ MONITOR

```
>RUN
BEST: BOB (17)
```

## 20.4 Lessons from the ports

Eight games from 101 *BASIC Computer Games* (DEC, 1975) were ported this way during the emulator's development: BAGLES, MUGWMP, HEX, BULCOW, STARS, TRAP, CHOMP and WAR. Each was played in the emulator, exported to tape and reloaded through the ACI ROM. These ports are not distributed (the original listings are not free), but they set the rules of this chapter: one-letter variables, arrays starting at 1, no `READ` / `DATA`, no `GOTO` out of a `FOR`, a single statement governed by `IF`, and `LOMEM=768` for the larger ones.

PART III

# 6502 assembly

*The processor, the built-in assembler and its tools, input/output and the Woz Monitor routines, annotated examples and tips*

CHAPTER 21

# The 6502 in brief

*The Apple-1's 6502 is an 8-bit processor running at 1.023 MHz, with three registers, 56 instructions and 64 KB of address space. This chapter gives the essentials needed to read and write the examples in this part.*

## 21.1 Registers

| Register | Size | Role |
|---|---|---|
| **A** | 8 bits | the accumulator: arithmetic, comparisons, reads and writes |
| **X**, **Y** | 8 bits | the index registers: loop counters, offset into a table |
| **S** | 8 bits | the stack pointer; the stack occupies page `$0100-$01FF` |
| **PC** | 16 bits | the program counter: the address of the next instruction |
| **P** | 8 bits | the flags N V – B D I Z C |

The flags are updated by almost every instruction:

`N (negative)`
: copy of bit 7 of the result; tested by `BMI` / `BPL`.

`V (overflow)`
: signed overflow of an addition or subtraction; `BVS` / `BVC`.

`Z (zero)`
: the result is zero; `BEQ` / `BNE`.

`C (carry)`
: carry out of an addition, no borrow from a subtraction, bit shifted out; `BCS` / `BCC`.

`D (decimal)`
: `ADC` and `SBC` work in BCD. The Woz Monitor does `CLD` at start-up.

`I (interrupts)`
: masks IRQ. The stock Apple-1 has none.

## 21.2 Addressing modes

| Mode | Syntax | Bytes | Example | Effect |
|---|---|---|---|---|
| implied | `INX` | 1 | `inx` | X ← X + 1 |
| accumulator | `ASL A` | 1 | `asl a` | shifts A |
| immediate | `#val` | 2 | `lda #$8D` | A ← $8D |
| zero page | `zp` | 2 | `lda $06` | A ← memory $0006 |
| zero page, X | `zp,X` | 2 | `lda $06,x` | |
| absolute | `addr` | 3 | `lda $D010` | |
| absolute, X / Y | `addr,X` | 3 | `lda message,x` | indexed table |
| indirect indexed | `(zp),Y` | 2 | `lda ($06),y` | 16-bit pointer in zero page, plus Y |
| indexed indirect | `(zp,X)` | 2 | `lda ($06,x)` | table of pointers |
| relative | `label` | 2 | `bne loop` | branch of −128 to +127 bytes |
| indirect | `(addr)` | 3 | `jmp ($FFFC)` | `JMP` only |

**Zero page** ( `$0000-$00FF` ) is the most precious: its instructions are shorter and faster, and only this mode allows `(zp),Y` pointers. The Woz Monitor uses only `$24-$2B` ; Integer BASIC uses much more (chapter 16).

## 21.3 Instructions by family

| Family | Instructions |
|---|---|
| load, store | `LDA LDX LDY STA STX STY` |
| transfers | `TAX TAY TXA TYA TSX TXS` |
| stack | `PHA PLA PHP PLP` |
| arithmetic | `ADC SBC` (with carry: `CLC` before `ADC` , `SEC` before `SBC` ) |
| increments | `INC INX INY DEC DEX DEY` |
| logic | `AND ORA EOR BIT` |
| shifts | `ASL LSR ROL ROR` |
| comparisons | `CMP CPX CPY` (C = 1 if register ≥ value, Z = 1 if equal) |
| branches | `BEQ BNE BCC BCS BMI BPL BVC BVS` |
| jumps | `JMP JSR RTS RTI BRK` |
| flags | `CLC SEC CLD SED CLI SEI CLV` |
| nothing | `NOP` |

> ✦ **UNSIGNED COMPARISON**
> After `CMP #n`, `BCC` branches if A < n and `BCS` if A ≥ n. For "A > n", compare with n + 1: that is what the examples do with `cmp #'9' | $80 + 1`.

## 21.4 An NMOS 6502, not a 65C02

The Apple-1 has an original (NMOS) 6502, and so does the emulator. Undocumented opcodes execute as `NOP`s of the right length, and the `JAM` opcodes halt the processor. The assembler accepts 65C02 instructions after `.setcpu "65C02"` (`BRA`, `STZ`, `PHX`, `PLX`, `PHY`, `PLY`, `INC A`...), for a board fitted with a 65C02, but they **do not work** in the emulator:

c02.s (test) 6502

```
        .setcpu "65C02"
        .org $0300
        lda #$C1        ; "A"
        bra next        ; on an NMOS 6502: 2-byte NOP
        lda #$D8        ; "X"
next:   jsr $FFEF
        jmp $FF1F
```

This program prints `A` on a 65C02, and `X` in the emulator: there `BRA` ($80) is a `NOP` that skips its operand, and the `LDA #$D8` is executed.

## 21.5 The memory map at a glance

| Addresses | Contents |
|---|---|
| `$0000-$00FF` | zero page (Woz Monitor: `$24-$2B`; BASIC: much more) |
| `$0100-$01FF` | stack |
| `$0200-$027F` | Woz Monitor line buffer |
| `$0280-$1FFF` | free: your programs, often at `$0280` or `$0300` (8 KB of RAM by default in the emulator; 4 KB, up to `$0FFF`, on a stock Apple-1) |
| `$C000-$C0FF` | ACI card (cassette); `$C100-$C1FF`: its program |
| `$C250-$C257` | GEN2 card switches |
| `$D010-$D013` | PIA: keyboard and display |
| `$E000-$EFFF` | Integer BASIC (if loaded) |
| `$FF00-$FFFF` | Woz Monitor |

Appendix A gives the complete map.

CHAPTER 22

# The devkit assembler

*The emulator contains a complete two-pass 6502 assembler, compatible with ca65. It is used in the editor, which assembles on every keystroke, on the command line, and for GitHub projects. There is nothing to install.*

## 22.1 The work cycle

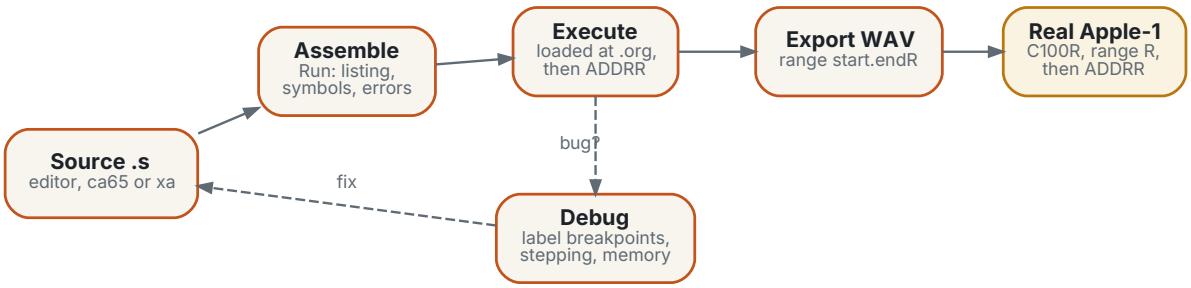



**Figure 33 —** The assembly work cycle: write, assemble, run, debug, then export to the real Apple-1.

1. **Write**: 📝 **Editor**, then **New** or **Open** a `.s`, `.asm` or `.a65` file. Syntax coloring distinguishes instructions, directives, labels, numbers and comments.
2. **Assemble**: nothing to do, assembly follows every change. The bottom of the panel gives the size, the addresses and the line to type on the real machine; errors are shown with their line and column.
3. **Run**: ▶ **Run** puts the bytes in memory at the `.org` address and types the Woz Monitor command (`300R`).
4. **Debug**: • **Breakpoint** on the cursor line, then step through in the debugger (chapter 8), with your labels in the disassembly.
5. **Export**: 📼 **Export WAV** writes the tape; on the real Apple-1, type `C100R`, the range shown (`0300.0320R`), then `300R`.

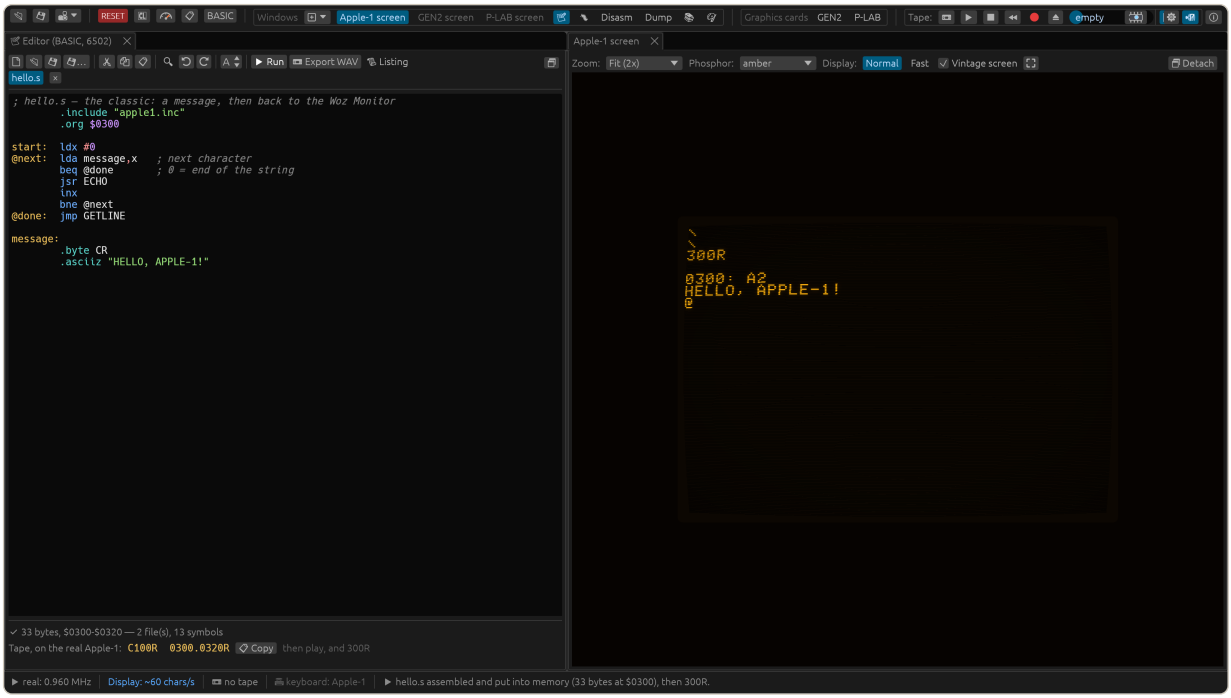



**Figure 34 —** A source in the editor: on the left the colored text, at the bottom the assembly result; on the right the Apple-1 has run the program.

## 22.2 The listing and the symbols

📃 **Listing** shows, for each line, the address, the bytes produced and the source; lines produced by a macro are marked `+`, included files are expanded, and the symbol table follows. Clicking a line takes you back to the source; clicking in the margin sets a breakpoint. Here is the listing of `hello.s` (chapter 23), produced by the command line:

hello.lst

```
0300  A2 00       start:  ldx #0
0302  BD 10 03    @next:  lda message,x   ; next character
0305  F0 06               beq @done       ; 0 = end of the string
0307  20 EF FF            jsr ECHO
030A  E8                  inx
030B  D0 F5               bne @next
030D  4C 1F FF    @done:  jmp GETLINE

                  message:
0310  8D                  .byte CR
0311  48 45 4C 4C         .asciiz "HELLO, APPLE-1!"
0315  4F 2C 20 41
…
; ---- Symboles (13)
; ECHO        $FFEF
; GETLINE     $FF1F
; message     $0310
; start       $0300
; start@done  $030D
; start@next  $0302
```

The local labels `@next` and `@done` belong to `start`: the table names them `start@next` and `start@done`, and the debugger accepts these names as breakpoint addresses. (The listing written by the command line keeps its French heading, *Symboles*.)

## 22.3 The command line

`apple1-emu asm` does the same without a window:

TERMINAL

```
apple1-emu asm hello.s                  # check only
apple1-emu asm hello.s -l hello.lst     # with the listing
apple1-emu asm hello.s -o hello.wav     # tape (.wav or .aif)
apple1-emu asm hello.s -o hello.bin     # raw binary
apple1-emu asm hello.s -o hello.hex     # Woz dump, to paste into the monitor
apple1-emu asm jeu.s -I ../common       # another folder for .include
```

Each command summarizes the assembly and gives the line for the real Apple-1. The summary is printed in French (“33 bytes … 2 file(s), 13 symbols / Tape (after C100R): 0300.0320R then 300R / written: hello.wav”):

```
33 octets, $0300-$0320 ; 2 fichier(s), 13 symboles
Cassette (après C100R) : 0300.0320R   puis 300R
écrit : hello.wav
```

Without `.org`, the program is placed at `$0300`. The `.hex` dump has the form the Woz Monitor accepts from the keyboard, ending with the run command:

t1.hex (excerpt)

```
0280: 34 12 34 12 10 05 FF 34
0288: 12 FF FF A8 02 EA EA EA
…
0280R
```

## 22.4 Syntax

A line contains, in this order and all optional: a label, an instruction or a directive, and a comment after `;`. Mnemonics and directives may be written in lower or upper case; symbols are case-sensitive.

6502

```
PTR     = $06           ; a constant (no byte produced)
start:  lda #0          ; a global label, followed by a colon
@loop:  inx             ; a local label: visible up to the next global one
        bne @loop
:       dex             ; an anonymous label
        bne :-          ; :- the previous one, :+ the next one, :-- the one before
```

**Numbers**: `255` in decimal, `$FF` in hexadecimal, `%1010` in binary, `'A'` for a character ($41). The keyboard returns its codes with **bit 7 set**: to compare a key, write `'A' | $80`; `apple1.inc` likewise defines `CR = $8D`. `*` stands for the current address.

**Operators**, from lowest to highest precedence:

| Precedence | Operators | Meaning |
|---|---|---|
| 1 | `\|\|` | logical or (result 0 or 1) |
| 2 | `&&` | logical and |
| 3 | `=` `<>` `<` `>` `<=` `>=` | comparisons (0 or 1) |
| 4 | `+` `-` `\|` | addition, subtraction, bitwise or |
| 5 | `*` `/` `&` `^` `<<` `>>` | multiplication, division, and, exclusive or, shifts |
| 6 (unary) | `<` `>` `-` `~` `!` | low byte, high byte, negation, complement, logical not |

So `lda #<message` loads the low byte of the address and `lda #>message` the high byte; `#>S+1` is `(>S)+1`. There is no modulo operator (`%` or `.mod` give a syntax error).

**Zero page or absolute.** An address below `$100` that is known at assembly time is encoded as zero page ( `lda $0006` gives `A5 06` , as with ca65). To force the absolute form, write `a:` ( `lda a:$06` gives `AD 06 00` ); to force zero page, `z:` .

> **! DEFINE ZERO-PAGE CONSTANTS BEFORE USING THEM**
>
> If `PTR = $06` is written **after** `lda PTR` , the assembler does not yet know the value and encodes the address on 2 bytes. It flags this in yellow: `warning: zero page address coded on 2 bytes (symbol defined further down; define it earlier, or write` The program works, but it is longer, and `(zp),Y` modes become impossible.

## 22.5 Directives

| Directive | Effect | Example |
|---|---|---|
| `.org addr` | assembly (and load) address | `.org $0300` |
| `.byte` ( `.byt` ) | bytes, strings included | `.byte CR, "OK", 0` |
| `.word` ( `.addr` ) | 16-bit words, low byte first | `.word 10000, end` |
| `.asciiz` | zero-terminated string | `.asciiz "HELLO"` |
| `.res n[, val]` | reserves n bytes (0, or val) | `.res 3, $EA` |
| `.include "f"` | inserts a file | `.include "apple1.inc"` |
| `.macro name args` … `.endmacro` | defines a macro ( `.mac` , `.endmac` ) | see `macros.s` |
| `.local a, b` | labels private to each macro call | |
| `.repeat n[, var]` … `.endrep` | repeats a block, var = 0 … n−1 | `.repeat 16, I` |
| `.if` `.elseif` `.else` `.endif` | conditional assembly | `.if WIDTH > 30` |
| `.ifdef` `.ifndef` | depending on whether a symbol exists | `.ifdef APPLE1` |
| `.ifblank` `.ifnblank` | macro argument empty or not | |
| `.defined(sym)` | 1 if the symbol exists (in an expression) | `.if .defined(d)` |
| `.define name value` | textual substitution ( `.undefine` ) | `.define CHARACTER '*'` |
| `.setcpu "65C02"` | accepts 65C02 instructions ( `.pc02` , `.p02` ) | |
| `.segment` `.code` `.data` `.rodata` `.bss` | accepted for compatibility | |
| `.feature string_escapes` | `\n` , `\"` … in strings | |

> **! SEGMENTS DO NOT MOVE ANYTHING**
>
> With ca65 and ld65, `.code` , `.data` and `.bss` send code to areas defined by the linker configuration. The built-in assembler produces **a single block**, in source order: segment directives are accepted but have no effect, and `.bss` produces zeros. For a project that needs a real `.cfg` file, the library uses ca65 and ld65 (chapter 4).

## 22.6 Macros, .repeat and conditional assembly

A macro is a template of lines, copied at each call with its arguments. `.local` gives each call its own labels, without conflicts. `.repeat` unrolls a block at assembly time, which is also handy to **compute tables**:

```6502
squares:
.repeat 16, I
        .byte I * I     ; 0, 1, 4, 9 … 225
.endrep
```

Chapter 23 presents `macros.s`, which uses all of this, and `gen2-lores.s`, which computes the addresses of the 24 screen rows with `.repeat`.

## 22.7 Error messages

Each error gives the file, the line, the column and the offending text; no byte is written while an error remains. The editor shows the messages in the interface language (below, English); the command line prints them in French.

| Message | Usual cause |
|---|---|
| `syntax error` | unknown operator, missing parenthesis |
| `unknown instruction, directive or macro` | typo; 65C02 instruction without `.setcpu`; label without a colon |
| `addressing mode not possible for this instruction` | `sta #5`, `ldx ($06),y`… |
| `undefined symbol` | `jsr ECO` instead of `ECHO`; forgotten `.include` |
| `symbol already defined` | two labels with the same name |
| `value out of range` | `lda #300`, address beyond `$FFFF` |
| `branch too far (-128 to +127 bytes)` | replace `bne far` with `beq :+` / `jmp far` / `:` |
| `file for .include not found` | name, or folder: add `-I` |
| `badly closed block (.macro/.endmacro, .if/.endif)` | forgotten `.endif` or `.endmacro` |
| `division by zero` | in an expression |
| `string or character without closing quote` | `"HELLO` |
| `value not known at this point (.org, .res, .if, .repeat)` | these directives need a value that is already defined |

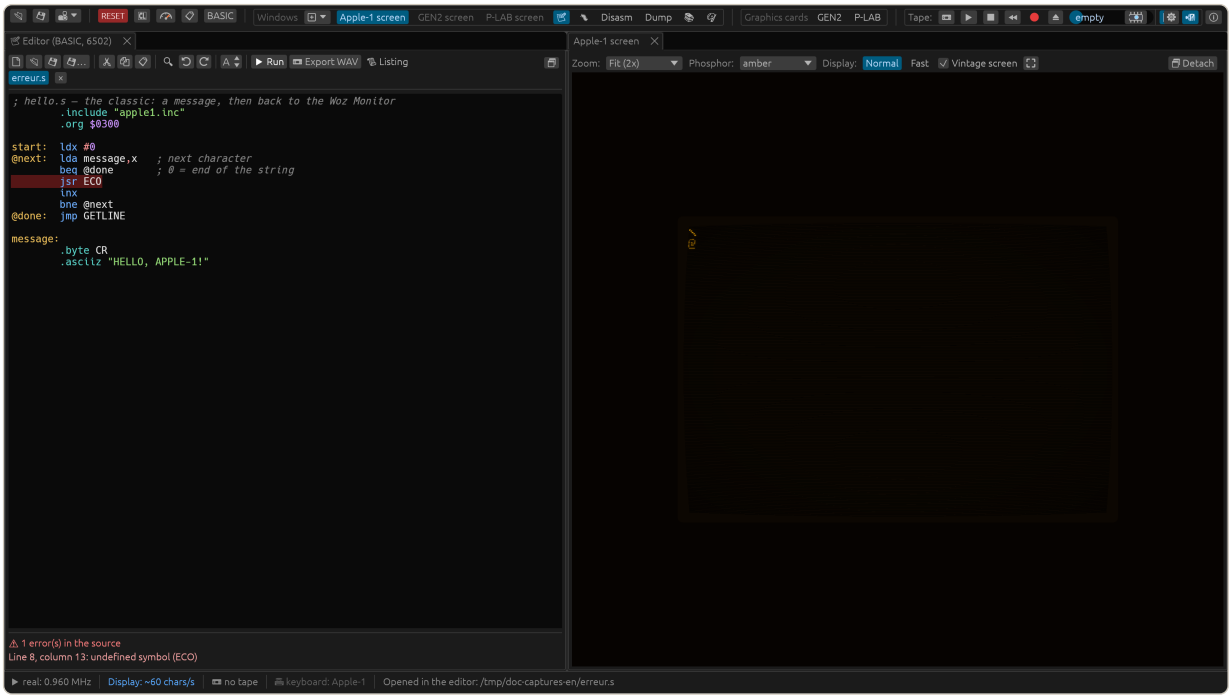



**Figure 35 —** An error in the editor: ECO instead of ECHO. Clicking the message takes you to the line.

## 22.8 Compatibility with ca65 and xa

The built-in assembler follows the syntax of **ca65** (cc65 package) for everything a single-block program needs. The eight examples of chapter 23 were assembled with both tools and compared byte for byte:

| Source | Bytes | Built-in = ca65 |
|---|---|---|
| `hello.s` | 33 | identical |
| `clavier.s` | 32 | identical |
| `hexa.s` | 64 | identical |
| `decimal.s` | 117 | identical |
| `jeu.s` | 222 | identical |
| `macros.s` | 135 | identical |
| `gen2-lores.s` | 110 | identical |
| `gen2-hires.s` | 94 | identical |

Sources written for **xa** (xa65) are recognized by their `#include` or `#define` lines, and translated line by line before assembly (error line numbers stay correct): `#define`, `#include`, `*=`, `.BYT`, `.ASC`, `/* */` comments and labels in column 0 without a colon. Dave Schmenk's slideshow (chapter 9) is assembled this way, to the same 3456 bytes as xa 2.4.1.

For **GitHub projects**, the library prefers ca65 and ld65 if they are installed (or if the project supplies an ld65 configuration), with `-D APPLE1` by default; otherwise it uses the built-in assembler. The repository's `apple1.toml` file specifies the addresses and symbols (chapter 4).

CHAPTER 23

# The keyboard, the display and the Woz Monitor

*The Apple-1 has only four input/output registers, in a 6820 (or 6821) PIA, and a 256-byte monitor. That is not much, but it is enough: this chapter shows how to read a key, write a character and use Steve Wozniak's routines.*

## 23.1 The four PIA registers

| Address | Name | Read | Write |
|---|---|---|---|
| `$D010` | `KBD` | key code, bit 7 set; reading **acknowledges** the key | — |
| `$D011` | `KBDCR` | bit 7 = 1: a key is waiting | configuration (done by the monitor) |
| `$D012` | `DSP` | bit 7 = 1: the display is busy | character to display (7 bits) |
| `$D013` | `DSPCR` | — | configuration (done by the monitor) |

These names are defined in `apple1.inc`, included by all the examples:

apple1.inc — 6502

```
; apple1.inc — I/O addresses and Woz Monitor routines
; (checked in the ROM: FFDC PRBYTE, FFE5 PRHEX, FFEF ECHO, FF1F GETLINE)
KBD     = $D010         ; keyboard data (bit 7 = 1)
KBDCR   = $D011         ; bit 7 = 1: a key is waiting
DSP     = $D012         ; write: character to the display (bit 7 read = busy)
DSPCR   = $D013
PRBYTE  = $FFDC         ; prints A as 2 hex digits (A destroyed)
PRHEX   = $FFE5         ; prints the low nibble of A (A destroyed)
ECHO    = $FFEF         ; prints the character in A (A, X, Y preserved)
GETLINE = $FF1F         ; back to the monitor: new line, then prompt
CR      = $8D           ; carriage return (bit 7 set, as the monitor uses it)
```

## 23.2 Reading a key

Wait for bit 7 of `KBDCR` to become 1, then read `KBD` :

```6502
read:   lda KBDCR       ; a key?
        bpl read        ; bit 7 = 0: not yet
        lda KBD         ; ASCII code + $80
```

The keyboard produces only upper case (the emulator converts lower case), with bit 7 set: `A` gives `$C1` , RETURN `$8D` , ESC `$9B` . To find out whether a key has been pressed **without waiting**, test `KBDCR` alone:

```6502
        lda KBDCR
        bpl nokey       ; no key: carry on working
        lda KBD         ; a key: read it (and acknowledge it)
```

`jeu.s` (chapter 23) uses this to pick a random number: it counts in a loop until the player presses a key.

## 23.3 Writing a character

The Apple-1's terminal takes one character at a time, and only when it is ready: bit 7 of `DSP` stays at 1 until the previous character has been displayed. The monitor's `ECHO` routine does the waiting for you. Here are its four instructions, read in the emulator's monitor:

```WOZ MONITOR
\
FFEF.FFFFR

FFEF: 2C
FFF0: 12 D0 30 FB 8D 12 D0 60
FFF8: 00 00 00 0F 00 FF 00 00
```

That is `BIT DSP` / `BMI ECHO` / `STA DSP` / `RTS` . `ECHO` preserves A, X and Y. Since the original terminal displays only **one character per frame**, about 60 per second, a program that prints a lot is limited by the display, not by the processor; the emulator respects this limit by default (F4 lifts it, chapter 2).

What the display does with each code:

- **RETURN** ( `$8D` , or `$0D` ) starts a new line; at the bottom of the screen, everything scrolls up one line.
- The **other control characters** ( `$00-$1F` , with or without bit 7) are ignored: no clear, no backspace, no bell.
- **Lower case** `$60-$7F` is displayed like upper case `$40-$5F` : the terminal keeps only 6 bits.
- After the 40th column, the cursor moves to the next line by itself.
- There is no code to clear the screen: it is a button on the machine (**Clear screen**, Ctrl+L).

## 23.4 The Woz Monitor routines

| Address | Name | Input | Effect | Registers |
|---|---|---|---|---|
| `$FFEF` | `ECHO` | A = character | displays it | A, X, Y preserved |
| `$FFDC` | `PRBYTE` | A = byte | displays it as 2 hex digits | A destroyed; X, Y preserved |
| `$FFE5` | `PRHEX` | A (low nibble) | displays one hex digit | A destroyed; X, Y preserved |
| `$FF1F` | `GETLINE` | — | back to the monitor: new line, then waits for a command | — |
| `$FF1A` | `ESCAPE` | — | like `GETLINE`, printing `\` first | — |
| `$FF00` | `RESET` | — | initializes the PIA and the monitor | — |

The monitor uses zero page `$24-$2B` and the line buffer `$0200-$027F`; everything else is yours (unless Integer BASIC must survive, chapter 16). Its commands (`300R`, `0300.031F`, `300: A9 8D`...) are summarized in Appendix B.

## 23.5 Starting and ending a program

A program is started by the monitor's `R` command (`300R`), which does a `JMP` to the address, not a `JSR`. There is therefore **no return address** on the stack: end with `jmp GETLINE` (`$FF1F`), not with `RTS`.

> **! RTS AND BRK DO NOT RETURN CONTROL**
>
> An `RTS` at the end of a program started by `R` pops whatever is on the stack. A `BRK` jumps through the ROM's interrupt vector, which is `$0000` (the last three words of the monitor, above: NMI `$0F00`, RESET `$FF00`, IRQ/BRK `$0000`): the processor then executes zero page. In both cases, press RESET (Ctrl+R).

A program called from BASIC with `CALL 768`, on the other hand, is called by a `JSR`: it must end with `RTS` to return to BASIC. Chapter 16 shows how to fit machine code below the BASIC program.

WOZ MONITOR

```
>POKE 768,96
>CALL 768
>
```

(`96` = `$60`, the `RTS` opcode: the smallest possible subroutine.)

## 23.6 The PIA, the tape and the color card

The ACI card occupies `$C000-$C0FF` (every access toggles the output) and its program `$C100-$C1FF`; chapter 5 explains how it works and Appendix C its timings. The GEN2 card is controlled by simply **reading** `$C250-$C257` (`bit SS_GR`); its addresses are in `gen2.inc`:

gen2.inc

6502

```
; gen2.inc — soft switches of Uncle Bernie's GEN2 color card
; (the Apple II $C050-$C057, moved to $C25x; a READ flips the switch)
SS_GR    = $C250        ; graphics (leaves text mode)
SS_TEXT  = $C251        ; text
SS_FULL  = $C252        ; full screen
SS_MIX   = $C253        ; 4 lines of text at the bottom
SS_PAGE1 = $C254
SS_PAGE2 = $C255
SS_LORES = $C256
SS_HIRES = $C257
```

CHAPTER 24

# Annotated examples

*Eight programs, from the simplest to the most complete. They are in the `docs/exemples/en/asm/` folder of the distribution; each was assembled by the built-in assembler and by ca65 (same bytes), then run in the emulator. The outputs shown here are the emulator's.*

To try them: open the file in the editor and click ▶ **Run**, or, on the command line, `apple1-emu asm hello.s -o hello.wav` then play the tape (chapter 5). Keep `apple1.inc` (and `gen2.inc`) in the same folder as the sources.

| File | Bytes | Address | What it shows |
|---|---|---|---|
| `hello.s` | 33 | `$0300` | a string, `ECHO`, local labels, back to the monitor |
| `clavier.s` | 32 | `$0300` | reading the keyboard, comparing characters |
| `hexa.s` | 64 | `$0300` | zero-page pointer, `(zp),Y`, 16-bit addition, `PRBYTE` |
| `decimal.s` | 117 | `$0300` | 16-bit numbers, subtractions, word tables |
| `jeu.s` | 222 | `$0300` | a complete game: randomness, subroutines, messages |
| `macros.s` | 135 | `$0300` | macros, `.local`, `.repeat`, `.if`, computed tables |
| `gen2-lores.s` | 110 | `$0C00` | the GEN2 card in LORES |
| `gen2-hires.s` | 94 | `$0C00` | the GEN2 card in HIRES |

## 24.1 `hello.s`: printing a message

hello.s 6502

```
; hello.s — the classic: a message, then back to the Woz Monitor
        .include "apple1.inc"
        .org $0300

start:  ldx #0
@next:  lda message,x   ; next character
        beq @done       ; 0 = end of the string
        jsr ECHO
        inx
        bne @next
@done:  jmp GETLINE

message:
        .byte CR
        .asciiz "HELLO, APPLE-1!"
```

X walks along the string up to the 0 byte placed by `.asciiz`. The `bne @next` after `inx` is never false for a string shorter than 256 characters: it is a short, safe jump. The program ends with `jmp GETLINE`, never with `RTS` (chapter 22).

WOZ MONITOR

```
\
300R
0300: A2
HELLO, APPLE-1!
```

The monitor first prints `0300: A2`, the byte at the start address, then the program takes over.

## 24.2 `clavier.s`: echoing keys

clavier.s 6502

```
; clavier.s — reads the keyboard and echoes each key to the screen.
; Digits are printed twice; ESC returns to the monitor.
        .include "apple1.inc"
        .org $0300

read:   lda KBDCR       ; a key?
        bpl read        ; bit 7 = 0: not yet
        lda KBD         ; ASCII code + $80; reading clears the key strobe
        cmp #$9B        ; ESC
        beq done
        jsr ECHO
        cmp #'0' | $80
        bcc read        ; < '0'
        cmp #'9' | $80 + 1
        bcs read        ; > '9'
        jsr ECHO        ; a digit: print it a second time
        jmp read
done:   jmp GETLINE
```

`cmp #'0' | $80` compares with the code of the digit 0, with bit 7 set as the keyboard does. `bcc` (“lower”) and `bcs` (“higher or equal”) bracket the digits. With the keys `A`, `B`, `1`, `2` then ESC, the screen shows:

```
0300: ADAB1122
\
```

The `AD` comes from the monitor (`lda KBDCR` starts with opcode `$AD`); `AB` then `11` and `22` come from the program; ESC returns control to the monitor, which prints `\`.

## 24.3 `hexa.s`: a monitor-style dump

hexa.s 6502

```
; hexa.s — monitor-style memory dump with PRBYTE:
; 2 lines of 8 bytes starting at $FF00
        .include "apple1.inc"
        .org $0300
PTR     = $06           ; zero-page pointer (not used by the monitor)

start:  lda #<$FF00
        sta PTR
        lda #>$FF00
        sta PTR+1
        ldx #2          ; number of lines
line:   lda #CR
```

```
        jsr ECHO
        lda PTR+1       ; address: high byte, then low byte
        jsr PRBYTE
        lda PTR
        jsr PRBYTE
        lda #':' | $80
        jsr ECHO
        ldy #0
byte:   lda #' ' | $80
        jsr ECHO
        lda (PTR),y
        jsr PRBYTE      ; PRBYTE destroys A but keeps X and Y
        iny
        cpy #8
        bne byte
        clc             ; PTR += 8
        lda PTR
        adc #8
        sta PTR
        bcc :+
        inc PTR+1
:       dex
        bne line
        jmp GETLINE
```

`PTR` and `PTR+1` form a **16-bit pointer** in zero page, low byte first; `lda (PTR),y` reads the byte at address PTR + Y. The addition `PTR += 8` shows the carry at work: `clc`, `adc #8`, and if a carry comes out (`bcc` not taken), `inc PTR+1`. The result is the beginning of the Woz Monitor:

```
FF00: D8 58 A0 7F 8C 12 D0 A9
FF08: A7 8D 11 D0 8D 13 D0 C9
```

## 24.4 `decimal.s`: printing a number in decimal

decimal.s 6502

```
; decimal.s — prints an unsigned 16-bit integer in decimal
; (repeated subtraction of powers of ten, no leading zeros)
        .include "apple1.inc"
        .org $0300
NUM     = $06           ; number to print (2 bytes)
FLAG    = $08           ; 1 once a non-zero digit has been printed

start:  lda #CR
        jsr ECHO
        ldx #0
@list:  lda numbers,x   ; print each number of the list
        sta NUM
        lda numbers+1,x
```

```
        sta NUM+1
        jsr prdec
        lda #' ' | $80
        jsr ECHO
        inx
        inx
        cpx #end_numbers - numbers
        bne @list
        jmp GETLINE

; prdec: prints NUM in decimal. Preserves X.
prdec:  txa
        pha
        lda #0
        sta FLAG
        ldx #0          ; index into the table of powers
@digit: ldy #'0' | $80  ; Y = current digit
@sub:   lda NUM         ; NUM - power
        sec
        sbc powers,x
        pha
        lda NUM+1
        sbc powers+1,x
        bcc @found      ; negative: digit found
        sta NUM+1
        pla
        sta NUM
        iny
        bne @sub
@found: pla
        tya
        cmp #'0' | $80
        bne @print
        ldy FLAG        ; leading zero? skip it
        beq @next
@print: jsr ECHO
        inc FLAG
@next:  inx
        inx
        cpx #8          ; 4 powers: 10000 down to 10
        bne @digit
        lda NUM         ; remainder: the units, always printed
        ora #'0' | $80
        jsr ECHO
        pla
        tax
        rts

powers: .word 10000, 1000, 100, 10
numbers:
```

```
        .word 0, 7, 42, 1976, 6502, 65535
end_numbers:
```

The 6502 cannot divide: `prdec` subtracts 10000 as many times as possible (the count gives the ten-thousands digit), then 1000, 100 and 10. The 16-bit subtraction is done in two steps, `sbc` on the low byte then on the high byte, with the carry linking them. `FLAG` suppresses leading zeros. Output:

```
0 7 42 1976 6502 65535
```

✦ **PRESERVING X IN A SUBROUTINE**

`prdec` uses X for itself, while the main loop needs it: it saves it with `txa` / `pha` at the start and restores it with `pla` / `tax` at the end. The NMOS 6502 has no `PHX`.

## 24.5 `jeu.s`: guess the digit

6502

jeu.s

```
; jeu.s — guess the digit (0 to 9): a simple game loop.
; The randomness comes from how long the player takes to press a key.
        .include "apple1.inc"
        .org $0300
SECRET  = $06
COUNT   = $07

start:  ldx #<title
        ldy #>title
        jsr prmsg
        ldx #0          ; X spins while we wait: the seed
@wait:  inx
        lda KBDCR
        bpl @wait
        lda KBD
        txa
@mod:   cmp #10         ; X modulo 10
        bcc @ok
        sbc #10         ; C = 1 here
        bcs @mod
@ok:    sta SECRET
        lda #0
        sta COUNT

turn:   ldx #<question
        ldy #>question
        jsr prmsg
        jsr getdig      ; A = digit 0..9 (other keys are ignored)
        inc COUNT
        cmp SECRET
        beq won
```

```
        bcc @higher
        ldx #<lower
        ldy #>lower
        jsr prmsg
        jmp turn
@higher:
        ldx #<higher
        ldy #>higher
        jsr prmsg
        jmp turn

won:    ldx #<bravo
        ldy #>bravo
        jsr prmsg
        lda COUNT
        ora #'0' | $80  ; < 10 tries: a single digit
        cmp #'9' | $80 + 1
        bcc :+
        lda #'+' | $80
:       jsr ECHO
        ldx #<tries
        ldy #>tries
        jsr prmsg
        jmp GETLINE

; getdig: waits for a digit, echoes it, returns it in binary in A
getdig: lda KBDCR
        bpl getdig
        lda KBD
        cmp #'0' | $80
        bcc getdig
        cmp #'9' | $80 + 1
        bcs getdig
        jsr ECHO
        and #$0F
        rts

; prmsg: prints the zero-terminated string whose address is in Y:X
MSG     = $08
prmsg:  stx MSG
        sty MSG+1
        ldy #0
:       lda (MSG),y
        beq :+
        jsr ECHO
        iny
        bne :-
:       rts

title:    .byte CR
```

```
          .asciiz "I PICKED A DIGIT. PRESS ANY KEY."
question: .byte CR
          .asciiz "YOUR DIGIT? "
higher:   .asciiz " HIGHER."
lower:    .asciiz " LOWER."
bravo:    .byte " WON IN ", 0
tries:    .asciiz " TRIES!"
```

The randomness comes from the player: X counts non-stop ( `@wait` ) until the first key, then the remainder of the division by 10 gives the secret digit. `prmsg` receives the address of a message in **Y:X** (high byte in Y), stores it in the pointer `MSG` , and prints it with `(MSG),y` . One game, typing `x` then the digits in order:

```
I PICKED A DIGIT. PRESS ANY KEY.
YOUR DIGIT? 0 HIGHER.
YOUR DIGIT? 1 HIGHER.
…
YOUR DIGIT? 7 HIGHER.
YOUR DIGIT? 8 WON IN 9 TRIES!
```

## 24.6 `macros.s`: letting the assembler do the work

6502

macros.s

```
; macros.s — macros, .repeat and conditional assembly
        .include "apple1.inc"
        .org $0300

WIDTH = 20              ; try 10 or 40
.define CHARACTER '*'

; print a constant character (the macro expands to lda/jsr)
.macro putc c
        lda #(c) | $80
        jsr ECHO
.endmacro

; print a string: local labels for each call
.macro print text
        .local loop, done, string
        ldx #0
loop:   lda string,x
        beq done
        jsr ECHO
        inx
        bne loop
        beq done
string: .asciiz text
done:
```

```
.endmacro

start:  putc 13
        print "MACROS AND .REPEAT"
        putc 13
.repeat 3               ; three rows of stars, unrolled at assembly time
        ldx #WIDTH
:       putc CHARACTER
        dex
        bne :-
        putc 13
.endrep
.if WIDTH > 30
        print "LONG LINES"
.else
        print "SHORT LINES"
.endif
        jmp GETLINE

; a table computed by the assembler: the squares of 0 to 15
squares:
.repeat 16, I
        .byte I * I
.endrep
```

`putc` and `print` generate code at each call; `.local` prevents the two calls of `print` from defining `loop` twice. `.repeat 3` unrolls the line of stars three times (each with its own anonymous label), `.if` chooses the message according to `WIDTH`, and the `squares` table is computed at assembly time. Output:

```
MACROS AND .REPEAT
********************
********************
********************
SHORT LINES
```

Change `WIDTH = 20` to `40`: the editor reassembles at once, the lines get longer and the message becomes `LONG LINES`.

## 24.7 gen2-lores.s: low-resolution color

These two programs need the GEN2 card (⚙ **Preferences ▸ Uncle Bernie GEN2 colour card**, chapter 6) and start at `$0C00`, above the LORES page.

gen2-lores.s 6502

```
; gen2-lores.s — color bars in LORES (40 x 48, 16 colors)
; Needs the GEN2 card (Settings ▸ Color card). Any key: back to text.
        .include "apple1.inc"
```

```
        .include "gen2.inc"
        .org $0C00      ; above LORES page 1 ($0400-$07FF)
PTR     = $06

start:  ldx #0          ; X = text row (0..23)
row:    lda rows_lo,x
        sta PTR
        lda rows_hi,x
        sta PTR+1
        ldy #39
column:
        tya             ; color = column / 2 (0..15, then again)
        lsr a
        and #$0F
        sta $00         ; one byte = 2 blocks: low nibble on top, high below
        asl a
        asl a
        asl a
        asl a
        ora $00
        sta (PTR),y
        dey
        bpl column
        inx
        cpx #24
        bne row

        bit SS_LORES    ; LORES, full screen, page 1, then graphics
        bit SS_FULL
        bit SS_PAGE1
        bit SS_GR
wait:
        lda KBDCR
        bpl wait
        lda KBD
        bit SS_TEXT
        jmp GETLINE

; addresses of the 24 rows: $400 + (r & 7) * $80 + (r >> 3) * $28
rows_lo:
.repeat 24, R
        .byte <($0400 + (R & 7) * $80 + (R >> 3) * $28)
.endrep
rows_hi:
.repeat 24, R
        .byte >($0400 + (R & 7) * $80 + (R >> 3) * $28)
.endrep
```

The two tables `rows_lo` and `rows_hi` are computed by `.repeat` with the row-address formula from chapter 6. Each byte carries two blocks, hence the nibble copied to the top and bottom halves. The `bit SS_…` instructions are **reads**: that is how the card's modes are switched.

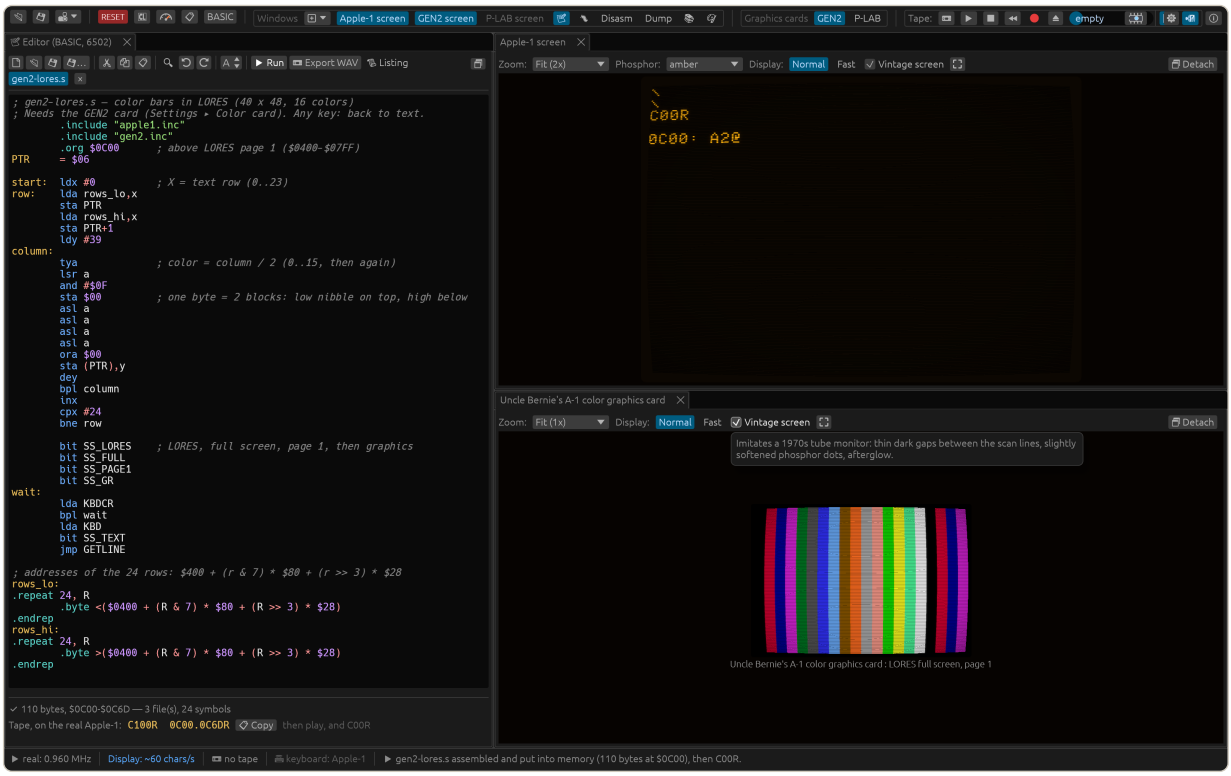



**Figure 36 —** gen2-lores.s: the sixteen LORES colors, two columns each, on the color monitor.

## 24.8 `gen2-hires.s`: high resolution

gen2-hires.s 6502

```
; gen2-hires.s — three color bands in HIRES (280 x 192)
; In each 128-byte block of the $2000 page, bytes 0-39 go to the top
; third of the screen, 40-79 to the middle, 80-119 to the bottom (120-127: holes).
        .include "apple1.inc"
        .include "gen2.inc"
        .org $0C00
PTR     = $06

start:  lda #<$2000
        sta PTR
        lda #>$2000
        sta PTR+1
block:  ldy #0
        ldx #0          ; X = pattern: 0 top, 2 middle, 4 bottom
third:  lda #40
        sta $08
byte:   tya             ; even or odd column?
        and #1
        ora pattern_idx,x
```

```
        sty $09
        tay
        lda patterns,y
        ldy $09
        sta (PTR),y
        iny
        dec $08
        bne byte
        inx
        cpx #3
        bne third
        clc             ; next block (+128)
        lda PTR
        adc #$80
        sta PTR
        bcc block
        inc PTR+1
        lda PTR+1
        cmp #$40        ; end of page 1 ($2000-$3FFF)
        bne block

        bit SS_HIRES
        bit SS_FULL
        bit SS_PAGE1
        bit SS_GR
wait:
        lda KBDCR
        bpl wait
        lda KBD
        bit SS_TEXT
        jmp GETLINE

pattern_idx: .byte 0, 2, 4
; two bytes per pattern (even column, odd column):
; violet, green, then blue and orange (bit 7 = 1: the other palette)
patterns: .byte $55, $2A        ; violet
        .byte $2A, $55          ; green
        .byte $D5, $AA          ; blue
```

In HIRES, the `$2000-$3FFF` page is divided into 128-byte blocks whose first 40 bytes go to the top third of the screen, the next 40 to the middle and the 40 after that to the bottom. The program fills each third with a different pattern, alternating two bytes according to the parity of the column: the result is three color bands.

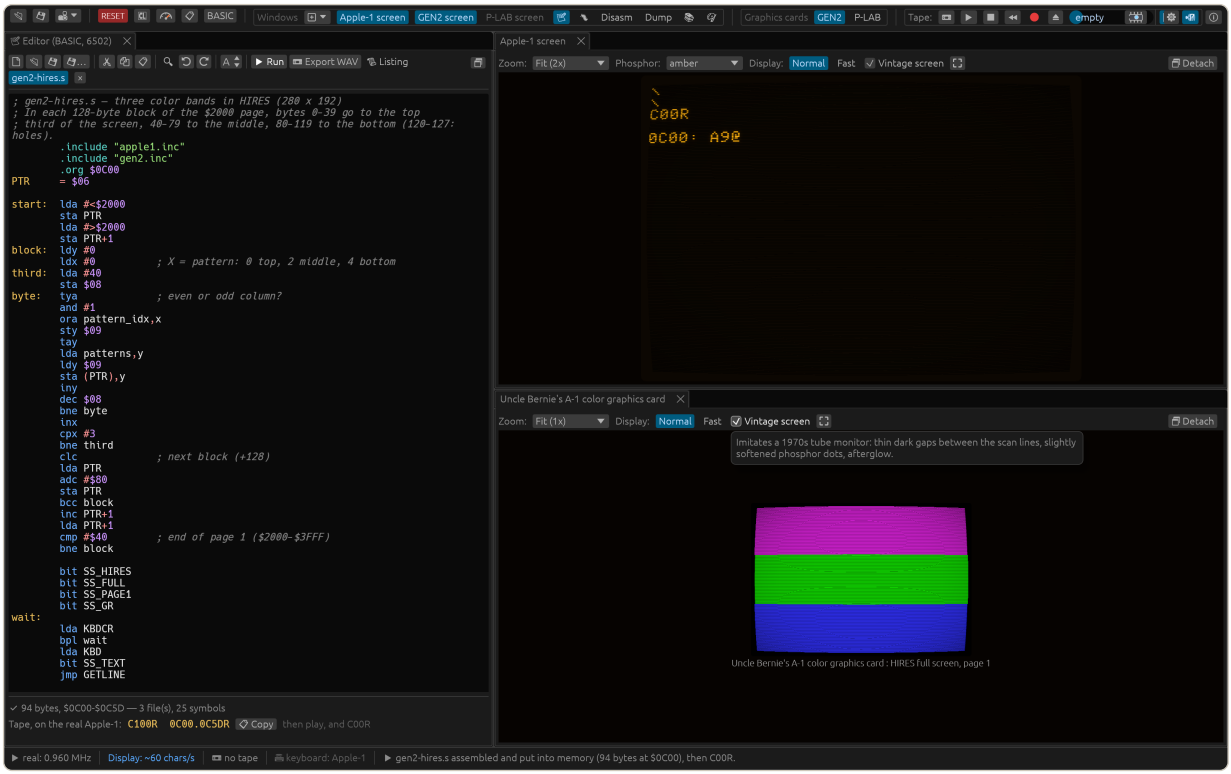



**Figure 37 —** gen2-hires.s: three color bands in HIRES: violet, green and blue.

CHAPTER 25

# Tips, traps and limits

*What you learn writing programs for the Apple-1, first in the emulator and then on the real machine.*

## 25.1 Where to put a program

| Address | For |
|---|---|
| `$0280` | as low as possible: just after the monitor's buffer |
| `$0300` | the usual choice (and the assembler's default address) |
| `$0C00` | with the GEN2 card in LORES or text (pages `$0400-$0BFF` left free for the display) |
| below HIMEM | with Integer BASIC: reserve the space (chapter 16) |

For zero page, the examples use `$06-$09`, which the monitor leaves free. If the program must coexist with BASIC, choose addresses BASIC does not use, or save and restore them.

## 25.2 Living within the branch range

A branch reaches only 127 bytes forward or 128 back. When the assembler answers `branch too far`, invert the condition and jump over a `JMP`:

```6502
        beq :+          ; instead of: bne far
        jmp far
:
```

## 25.3 Strings and bit 7

The keyboard returns codes with bit 7 set, and the monitor compares against those codes. The display, on the other hand, ignores bit 7: `.asciiz "HELLO"` displays perfectly well. To **compare** a key, put bit 7 in the constant: `cmp #'Q' | $80`. Lower-case letters in a string are displayed as upper case.

## 25.4 Waiting a little

At 1.023 MHz (0.96 MHz effective), a `dex` / `bne` loop costs 5 cycles per iteration:

```6502
delay:
        ldy #78         ; 78 x 256 iterations of 5 cycles: about 0.1 s
:       ldx #0
:       dex
        bne :-
        dey
        bne :--
        rts
```

The emulator at real speed respects these durations; at fast speed (F3), they vanish. The ACI's timing loops are counted in cycles in the same way (Appendix C).

## 25.5 Debugging efficiently

- Set breakpoints on a **label** (`start`, `prdec`, `start@done`) rather than on an address: it stays correct when the code moves.
- Step over `jsr ECHO` with ↪ **Over**: otherwise you follow the display's wait loop, instruction by instruction.
- If the Apple-1 stops responding after your program, look at PC: in zero page, it is almost always a stray `BRK` or `RTS` (chapter 22).
- The **dump** shows zero page: your pointers are visible there, low byte first.

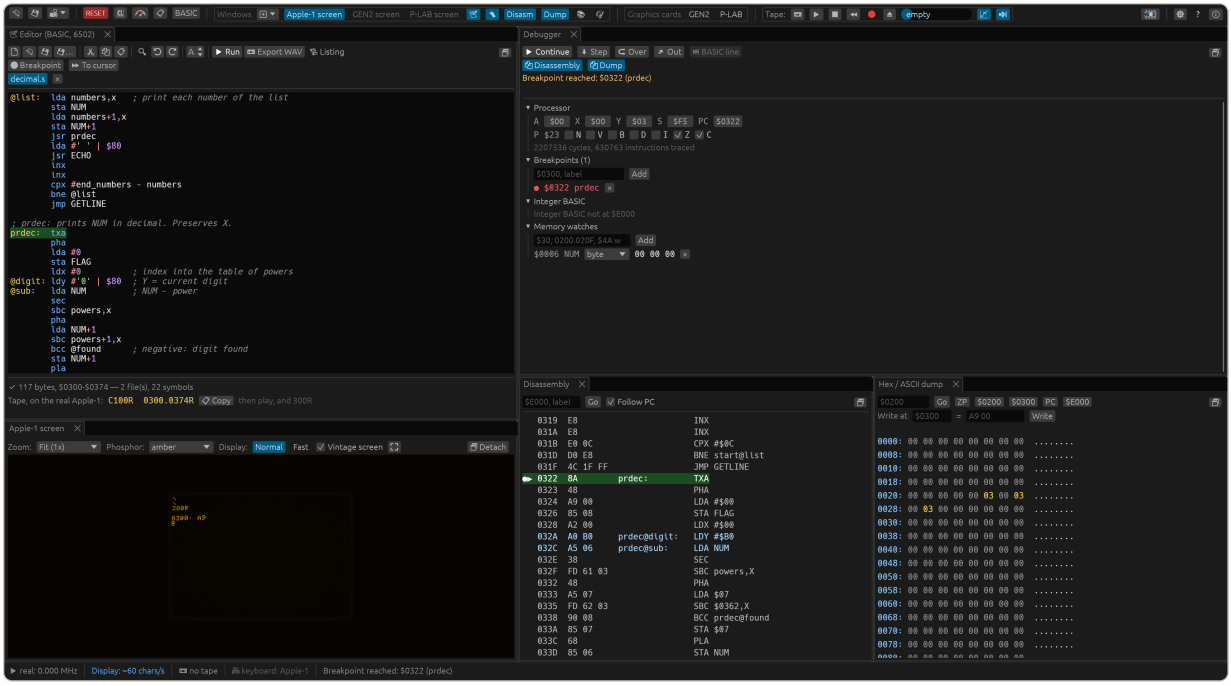


**Figure 38 —** The debugger stopped in a 6502 source: registers, flags, disassembly with labels, and memory.

## 25.6 On to the real Apple-1

1. Assemble and try it in the emulator, **at real speed** and with the display limited to 60 characters per second.
2. Export the tape (📼 **Export WAV**, or `-o prog.aif`): AIFF is the safest format when playing from a phone.
3. On the Apple-1: `C100R`, then the range shown (`0300.0320R`), start playback, then `300R`.
4. Without an ACI: the `.hex` dump can be typed (or pasted into a serial terminal) straight into the Woz Monitor; its last line runs the program.

## 25.7 Limits of the built-in assembler

- A single output block, in source order: segments have no effect, and there is no linker. For code spread over several areas, use ca65 and ld65.
- No modulo operator, no `.proc`, `.scope`, `.struct` or `.enum`: these are ca65 directives the built-in assembler does not know.
- 65C02 instructions are assembled, but the emulated processor is an NMOS 6502: they do not work in the emulator (chapter 20).
- In the **listing**, a line produced by a macro sometimes loses the space between the instruction and the operand (`ldastring, x` for `lda string,x`). This is a display defect of the listing only: the bytes are correct.

APPENDICES

# Appendices

*Memory map, Woz Monitor commands, ACI timings, character set, keyboard shortcuts, pitfalls, licence and credits*

APPENDIX A

# Memory map

| Addresses | Contents | Notes |
|---|---|---|
| FF00–FFFF | **Woz Monitor** (256-byte PROM) | NMI, RESET, IRQ vectors |
| E000–EFFF | **Integer BASIC** (4 KB of RAM) | loaded from tape |
| D010–D013 | **PIA 6821**: keyboard, display | KBD, KBDCR, DSP, DSPCR |
| C250–C257 | **GEN2 card** : soft switches | optional, read access |
| C100–C1FF | **ACI PROM**(C100R) | C000–C0FF: flip-flop |
| 1000–BFFF | expansion RAM (up to 48 KB) | HIRES 2000–5FFF (GEN2) |
| 0800–0FFF | BASIC programs (LOMEM–HIMEM) | text/LORES 2: 0800 |
| 0300–07FF | your machine-language programs | text/LORES 1: 0400 |
| 0200–027F | monitor input buffer | keep clear |
| 0100–01FF | 6502 stack | |
| 0000–00FF | zero page | monitor 24–2B, BASIC 4A–FF |

**Figure 39 —** Apple-1 memory in the emulator, from the monitor vectors down to zero page.

| Addresses | Contents | Details |
|---|---|---|
| `$0000-$00FF` | zero page | Woz Monitor `$24-$2B`; Integer BASIC `$4A-$FF` (pointers LOMEM `$4A`, HIMEM `$4C`, PP `$CA`, PV `$CC`) |
| `$0100-$01FF` | 6502 stack | |
| `$0200-$027F` | monitor line buffer | 128 characters |
| `$0280-$0FFF` | base RAM (4 KB) | machine-language programs; BASIC between LOMEM (`$0800`) and HIMEM (`$1000`) |
| `$1000-$BFFF` | expansion RAM | 8 KB by default in the emulator (up to `$1FFF`), up to 48 KB with `--ram` |
| `$C000-$C0FF` | ACI: output toggle, tape input | any access toggles the output |
| `$C100-$C1FF` | ACI: PROM (the ACI program) | started with `C100R` |
| `$4000-$7FFF` | P-LAB CodeTank | 16 KB EPROM, read-only, when a ROM is plugged in (over the RAM) |
| `$C250-$C257` | GEN2 card: switches | read; text/LORES pages `$0400`, `$0800`, HIRES `$2000`, `$4000` |
| `$CC00-$CC01` | P-LAB card: TMS9918A | `$CC00` data, `$CC01` control and status; repeated over `$CC00-$CFFF` |
| `$D010-$D013` | PIA: `KBD`, `KBDCR`, `DSP`, `DSPCR` | chapter 22 |
| `$E000-$EFFF` | BASIC | RDK G2 Basic (7,933 bytes, needs RAM up to `$FEFF`), or Apple's Integer BASIC (4 KB of RAM, loaded from tape or an image) |
| `$FF00-$FFFF` | Woz Monitor | vectors: NMI `$0F00`, RESET `$FF00`, IRQ/BRK `$0000` |

APPENDIX B

# Woz Monitor commands

The monitor waits for a line ended by RETURN. Addresses and bytes are written in hexadecimal, without `$` ; a line may contain several commands separated by spaces.

| Command | Effect | Example |
|---|---|---|
| `addr` | displays the byte at that address | `FFEF` |
| `addr.addr` | displays a range, 8 bytes per line | `FF00.FF1F` |
| `.addr` | continues the display up to that address | `.FF3F` |
| `addr: byte byte …` | stores bytes starting at the address | `300: A9 C1 20 EF FF` |
| `: byte …` | continues storing | `: 4C 1F FF` |
| `addrR` | runs the program at that address ( `JMP` ) | `300R` |
| Esc | cancels the line (prints `\` ) | |
| `_` (backspace) | cancels the last character | |

A line longer than 127 characters is cancelled as if by Esc. After `R` , the monitor first prints the byte at the address ( `0300: A2` ), then the program takes over.

| Address | Command | Role |
|---|---|---|
| `E000R` | BASIC cold start | clears the program |
| `E2B3R` | BASIC warm start | keeps the program |
| `C100R` | the ACI program | then `start.endR` (read) or `start.endW` (write) |
| `FF00R` | software RESET of the monitor | |

APPENDIX C

# ACI timings

The durations are produced by the ACI program, by counting cycles between two toggles of the output. Values measured in the emulator, which runs this program on its 6502 (1.022727 MHz clock, with the 4-cycle stall every 65 cycles for memory refresh):

| Element | Cycles | Duration | Frequency |
|---|---|---|---|
| header half-period | 592 | 618 µs | ≈ 809 Hz |
| first half of the start bit | 180 | 188 µs | |
| half-period of a “0” | 238 | 248 µs | ≈ 2.0 kHz |
| half-period of a “1” | 473 | 494 µs | ≈ 1.0 kHz |
| complete header | 16,390 half-periods | ≈ 10.1 s | |

Format of a recording:

1. the **header**: about 10 seconds of 618 µs half-periods;
2. the **start bit**: one short half-period (188 µs), then a “0” half-period;
3. the **data**: each byte, most significant bit first, one full period per bit;
4. nothing else: no length, no address, no checksum. The read command supplies the range.

When reading, the ACI program measures half-periods with a 12-cycle loop. It takes as the start bit any half-period shorter than 31 iterations (≈ 372 cycles, 388 µs), and separates “0” from “1” at about 0.74 ms per full period. The emulator’s file decoder accepts headers from 0.40 to 0.90 ms per half-period, which covers the c2t tool (1 kHz) and tape speed variations.

APPENDIX D

# The 2513 character set




**Figure 40 —** The 64 characters of the Signetics 2513, as the emulator draws them: codes $20 to $5F (with bit 7, $A0 to $DF).

The character generator knows only 64 symbols: space, punctuation, digits and upper-case letters. The terminal keeps only 6 bits of the code it receives:

| Code sent | Display |
|---|---|
| `$20-$5F` (or `$A0-$DF`) | the character from the table |
| `$60-$7F` (lower case, or `$E0-$FF`) | the corresponding upper-case letter (`$40-$5F`) |
| `$0D` (or `$8D`) | new line |
| other `$00-$1F` | nothing: ignored |

Each character is a 5 × 7 dot matrix in a 7 × 8 cell; the screen is 40 × 24 characters, or 280 × 192 dots.

APPENDIX E

# Keyboard shortcuts

| Key | Effect |
|---|---|
| Ctrl + R | Apple-1 RESET |
| Ctrl + L | CLEAR SCREEN |
| Ctrl + V | paste: types the clipboard on the Apple-1 keyboard |
| F1 | help mode (detailed tooltips) |
| F3 | real or fast speed |
| F11 | full screen |
| Ctrl + + / Ctrl + − / Ctrl + 0 | screen zoom (or editor text zoom, when the editor has the keyboard focus) |
| Ctrl + S | save the editor's file |
| Ctrl + F / Ctrl + H / Ctrl + G | editor: find, replace, go to line |
| Ctrl + / | editor: comment / uncomment (6502), REM / un-REM (BASIC) |
| F5 / F9 / F12 | editor: run, breakpoint on the line, go to the definition (6502) |
| Ctrl +letter | control character ( `Ctrl+D` = `$84` ) |
| right-click on a screen | Copy, Paste (type the text), Clear screen, Reset |
| Esc | ESC ( `$9B` ): cancels the line in the monitor and in BASIC |
| backspace | types `_` , the monitor's and BASIC's rubout |

In the **terminal** ( `apple1-emu` without a window, chapter 12):

| Key | Effect |
|---|---|
| F2 | emulator command line |
| F5 / F6 | tape: play, stop |
| F10 or Ctrl + Q | quit |

APPENDIX F

# Pitfalls and behaviours to know

Small things that surprise people the first time. None of them is a bug; each one has a reason.

- **BASIC and RUN.** The editor's ▶ **Run** starts BASIC, loads the listing and types `RUN` for you. A program that asks for input waits for your keys on the Apple-1 screen: click the screen first.
- **RND.** The first `RND` of a run would always be the same on a machine that has just been reset; ▶ **Run** waits a random moment before `RUN`, so a game does not always begin the same way.
- **Tape commands.** The line "Tape, on the real Apple-1" under the editor is the Woz Monitor command to type on a real machine (`C100R`, then the `R` commands of the ranges) before you play the tape. It is a hint, not something to type in the emulator.
- **The keyboard goes to one place at a time.** The line at the bottom of the window says where: the Apple-1, the editor, a field. Clicking the Apple-1 screen sends the keys to the machine; clicking the editor takes them back. After an error is clicked in the editor's list, the keys return to the editor.
- **Paused debugger.** Keys typed while the machine is paused are not lost, but only the **last** one is kept (the real keyboard latch holds one character); it is delivered when you continue.
- **Breakpoints go with the program.** Loading another program (file, library, assembler) clears the breakpoints and labels of the previous one. The breakpoints you set stay from one session to the next only as long as you do not load something else.
- **Pasting with the middle button.** In the editor, a middle click moves the cursor there and pastes the X11/Wayland **primary selection** (the text you just highlighted in any application). The editor does not yet *write* its own selection to the primary selection: to copy out of the editor use Ctrl+C.
- **Small windows.** On a monitor narrower than 1400 points the first-launch layout is editor and screen; from the ⊞ **Layout** ▾ menu you can choose another. On a 1024 × 768 screen the toolbar scrolls sideways: the **»** button reaches the Cassette group, instead of cutting it off on the right. A fixed zoom (Ctrl+**+**) larger than the screen pane is kept, and the screen scrolls in its pane; the status bar tells you so.
- **Saving an untitled tab. Save as** (icon, Ctrl+S or the right-click menu of the tab) never leaves two tabs for the same file: a tab already open on that file is closed.
- **First launch and the network.** The first start contacts GitHub to fetch the demo project (chapter 4); remove the project from the "GitHub projects" list in ⚙ **Preferences** to stop that for good.
- **Reading the keyboard in G2 Basic.** A key pressed while a program runs stops it, as on the original, and `INPUT` waits. A game that needs to watch the keys calls `KEY(0)` in its main loop (chapter 15); once it has, a key no longer stops the program.
- **Labels and breakpoints.** A breakpoint on a label that does not exist yet (typed before you assemble or run) is kept and set when the label appears.

- **Layouts and windows.** A saved layout remembers the docked panels, the detached windows (place and size) and the size of the main window. It does not remember which window has the keyboard.

APPENDIX G

# Licence and credits

**Retro Dev Kit Apple 1** is free software, released under the **GNU General Public License, version 3** (GPL-3.0), without any warranty. The full text is in the `LICENSE` file of the archive and of the repository.

Some parts come from other projects and keep their own licence, the **MIT** licence, which is compatible with the GPL; they are listed in the `NOTICE` file:

- **RDK G2 Basic**, the built-in BASIC, written for this project (repository `Old-Wirehead/rdk-gen2basic-apple1`, MIT);
- **Uncle Bernie's GEN2 tests** (`gen2-test1` to `gen2-test4`, the bars) and the **GEN2 glyphs**, supplied by Uncle Bernie.

The games of the library written by **Claude IA** (Hangman, Matches, Battleship, Tunnel and Tic-tac-toe) are distributed with the project.

What the kit does **not** contain: Apple's Integer BASIC, commercial tapes, and the demos of the P-LAB card. Those stay with their owners; you bring your own copies.

The P-LAB card is a creation of P-LAB; the TMS9918A is a Texas Instruments chip. This project is not affiliated with Apple, Texas Instruments or P-LAB.

# Index


## #

📼 Export WAV 43
▶ Run 43
>255 ERR 79
>32767 ERR 79
>8 FORS ERR 79
>8 GOSUBS ERR 79

## A

ABS 73
addr.addr 124
addrR 124
AND 74
anonymous labels 97
ASC 73
ascii 61
.asciiz 98
asm (command line) 60, 96
AUTO 69

## B

BAD BRANCH ERR 79
BAD NEXT ERR 79
BAD RETURN ERR 79
BASIC 13
basic (command line) 60
.byte 98

## C

C100R 32
$C250-$C257 35
CALL 73
cassette 59
charger 59
CLR 69
convert 60

## D

DEL 69
DIM 72
DIM ERR 79
double-click 25
DSP 101

## E

E000R 68
E2B3R 68
ECHO 103
END 73
END ERR 79
enregistrer 59
export 60

## F

FOR 72

## G

GETLINE 103
GOSUB 72
GOTO 72

## H

HIMEM 69, 77

## I

IF 72
.if 98
import 60
.include 98
info 60
INPUT 72
instructions (6502) 92

## K

KBD 101
KBDCR 101

## L

LEN 73
LET 72
LIST 69
Load 13
local labels 97
LOMEM 69, 77

## M

.macro 98
MAN 69
MEM FULL ERR 79
MOD 74

## N

NEXT 72
NOT 74

## O

OR 74
.org 98
Out 45
Over 45

## P

PEEK 73
POKE 73
porting 87
PP (program pointer) 77
PRBYTE 103
PRHEX 103
PRINT 72
prom 61

## R

R (read) 32
RANGE ERR 79
registers (6502) 91
REM 72
.repeat 98
.res 98
RETYPE LINE 79
RND 73
RUN 69

## S

sauver 59
Save 13
SCR 69
SCRN 73
.setcpu 98
SGN 73
Step (debugger) 45
STOPPED AT 79
STR OVFL ERR 79
STRING ERR 79
SYNTAX ERR 79

## T

TAB 73
Test the latest build 27
TOO LONG ERR 79
TOO MANY PARENS ERR 79

## U

USR 73

## W

W (write) 32
.word 98